

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI THÀNH PHỐ HỒ CHÍ MINH
VIỆN CÔNG NGHỆ THÔNG TIN VÀ ĐIỆN, ĐIỆN TỬ



HUỲNH NGUYỄN QUỐC BẢO - PHÍ NGỌC THÁI BÌNH

ỨNG DỤNG LLM ĐỂ PHÁT TRIỂN
ỨNG DỤNG TRẢ LỜI EMAIL TỰ ĐỘNG
DỰA TRÊN NỘI DUNG EMAIL ĐÃ GỬI

BÁO CÁO THỰC TẬP TỐT NGHIỆP

THÀNH PHỐ HỒ CHÍ MINH, 2025

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI THÀNH PHỐ HỒ CHÍ MINH
VIỆN CÔNG NGHỆ THÔNG TIN VÀ ĐIỆN, ĐIỆN TỬ



HUỲNH NGUYỄN QUỐC BẢO – 22H1120002
PHÍ NGỌC THÁI BÌNH – 22H1120095

ỨNG DỤNG LLM ĐỂ PHÁT TRIỂN
ỨNG DỤNG TRẢ LỜI EMAIL TỰ ĐỘNG
DỰA TRÊN NỘI DUNG EMAIL ĐÃ GỬI

BÁO CÁO THỰC TẬP TỐT NGHIỆP

GIÁO VIÊN HƯỚNG DẪN:
TS. HÀN TRUNG ĐỊNH

NIÊN KHÓA 2022 - 2026

BẢN NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN
THỰC TẬP TỐT NGHIỆP

1. Họ và tên sinh viên:

<i>Họ và tên</i>	<i>MSSV</i>	<i>Lớp</i>	<i>Chuyên Ngành</i>
Huỳnh Nguyễn Quốc Bảo	22H1120002	CN22CLCA	CNTT
Phí Ngọc Thái Bình	22H1120095	CN22CLCD	CNTT

2. Tên đề tài: Ứng dụng LLM để phát triển ứng dụng trả lời Email tự động dựa trên nội dung email đã gửi.

3. Nhận xét:

a) Về tinh thần, thái độ làm việc của sinh viên:

.....

.....

.....

b) Những kết quả đạt được của Thực tập tốt nghiệp:

.....

.....

.....

c) Những hạn chế của Thực tập tốt nghiệp:

.....

.....

.....

4. Đề nghị:

Được bảo vệ (hoặc nộp BCTTTN để chấm) ☐

Không được bảo vệ ☐

5. Điểm thi (nếu có):

TP. HCM, ngày ... tháng ... năm 20...

Giảng viên hướng dẫn

(Ký và ghi rõ họ tên)

BẢN NHẬN XÉT CỦA GIẢNG VIÊN PHẢN BIỆN

THỰC TẬP TỐT NGHIỆP

1. Họ và tên sinh viên:

<i>Họ và tên</i>	<i>MSSV</i>	<i>Lớp</i>	<i>Chuyên Ngành</i>
Huỳnh Nguyễn Quốc Bảo	22H1120002	CN22CLCA	CNTT
Phí Ngọc Thái Bình	22H1120095	CN22CLCD	CNTT

2. Tên đề tài: Ứng dụng LLM để phát triển ứng dụng trả lời Email tự động dựa trên nội dung email đã gửi.

3. Nhận xét:

a) Những kết quả đạt được của Thực tập tốt nghiệp:

.....
.....
.....

b) Những hạn chế của Thực tập tốt nghiệp:

.....
.....
.....

4. Đề nghị:

Được bảo vệ ☐

Bổ sung thêm để bảo vệ ☐

Không được bảo vệ ☐

5. Các câu hỏi sinh viên cần trả lời trước Hội đồng:

- (1)
- (2)
- (3)

6. Điểm thi (nếu có):

TP. HCM, ngày ... tháng ... năm 20...

Giảng viên phản biện

(Ký và ghi rõ họ tên)

LỜI CAM ĐOAN

Nhóm xin cam đoan đây là đề tài nghiên cứu của chính chúng em. Các số liệu, kết quả nêu trong báo cáo là trung thực, được xây dựng dựa trên quá trình tìm hiểu, phân tích và triển khai của hai thành viên và chưa từng được công bố trong bất kỳ một công trình nghiên cứu nào khác. Tất cả các tài liệu tham khảo, ý tưởng từ các công trình nghiên cứu khác được sử dụng trong báo cáo này đều được trích dẫn và ghi nhận rõ ràng. Chúng em xin hoàn toàn chịu trách nhiệm về đề tài của mình.

Hồ Chí Minh, ngày 20 tháng 12 năm 2025

Tác giả

Huỳnh Nguyễn Quốc Bảo, Phí Ngọc Thái Bình

LỜI CẢM ƠN

Để hoàn thành Đồ án thực tập tốt nghiệp này, chúng em xin trân trọng gửi lời cảm ơn đến TS. Hàn Trung Định đã hướng dẫn, định hướng và hỗ trợ tận tình nhóm trong suốt quá trình thực hiện đề tài.

Chúng em cũng xin gửi lời cảm ơn chân thành đến Quý Thầy Cô tại Viện Đào tạo Chất lượng cao, Trường Đại học Giao thông Vận tải TP. Hồ Chí Minh. Những kiến thức và nền tảng vững chắc mà Thầy Cô đã truyền đạt trong suốt những năm học là hành trang không thể thiếu để nhóm có thể tự tin thực hiện dự án này.

Bên cạnh đó, chúng em cảm ơn gia đình và bạn bè đã luôn đồng hành, động viên và tạo điều kiện tốt nhất để nhóm có thể hoàn thành tốt báo cáo.

Mặc dù đã cố gắng hoàn thiện sản phẩm trong phạm vi kiến thức và thời gian cho phép, đồ án này chắc chắn không tránh khỏi những thiếu sót cũng như những điểm chưa thực sự hoàn thiện. Chúng em mong sẽ nhận được những ý kiến đóng góp quý báu từ Quý Thầy Cô & các bạn sinh viên để nhóm có thể phát triển và hoàn thiện hơn nữa trong tương lai.

Chúng em xin chân thành cảm ơn!

Hồ Chí Minh, ngày 20 tháng 12 năm 2025

Tác giả

Huỳnh Nguyễn Quốc Bảo, Phí Ngọc Thái Bình

MỤC LỤC

LỜI CAM ĐOAN	i
LỜI CẢM ƠN	ii
DANH MỤC BẢNG BIỂU	vi
DANH MỤC HÌNH ẢNH	vii
DANH MỤC TỪ VIẾT TẮT	ix
MỞ ĐẦU	1
1. Lý do chọn đề tài.....	1
2. Mục đích nghiên cứu của đề tài	1
3. Đối tượng và phạm vi nghiên cứu.....	2
4. Phương pháp nghiên cứu.....	2
5. Bố cục của báo cáo.....	3
CHƯƠNG 1. CƠ SỞ LÝ THUYẾT & CÔNG NGHỆ SỬ DỤNG	4
1.1. Cơ sở lý thuyết	4
1.1.1. Hệ thống trả lời email tự động	4
1.1.2. Xử lý ngôn ngữ tự nhiên (Natural Language Processing – NLP).....	4
1.1.3. Mô hình ngôn ngữ lớn (Large Language Model – LLM).....	6
1.1.4. Kiến trúc Tác tử (Agentic Architecture)	6
1.1.5. Kỹ thuật Sinh Tăng cường Truy xuất (Retrieval-Augmented Generation - RAG)	8
1.2. Khảo sát các hệ thống đã có	9
1.3. Công nghệ sử dụng.....	11
1.3.1. Frontend	11
1.3.2. Backend.....	12
1.3.3. Cơ sở dữ liệu và lưu trữ vector	13
1.3.4. Mô hình ngôn ngữ lớn & Công nghệ hỗ trợ	14
1.3.5. Xác thực người dùng.....	17
1.3.6. Công cụ phát triển.....	18

CHƯƠNG 2. PHÂN TÍCH YÊU CẦU VÀ THIẾT KẾ HỆ THỐNG	19
2.1. Phân tích yêu cầu	19
2.1.1. Yêu cầu chức năng	19
2.1.2. Yêu cầu phi chức năng	21
2.2. Thiết kế hệ thống.....	23
2.2.1. Kiến trúc tổng thể.....	23
2.2.2. Biểu đồ use case của hệ thống	23
2.2.3. Biểu đồ hoạt động (Activity diagram)	24
2.2.4. Biểu đồ tuần tự (Sequence diagram).....	31
2.2.5. Thiết kế cơ sở dữ liệu.....	37
CHƯƠNG 3. CÀI ĐẶT, TRIỂN KHAI & KIỂM THỬ	40
3.1. Môi trường phát triển	40
3.1.1. Phần cứng.....	40
3.1.2. Phần mềm.....	40
3.1.3. Kết nối Internet	41
3.2. Cài đặt hệ thống	41
3.2.1. Cấu hình backend.....	41
3.2.2. Cấu hình frontend.....	43
3.2.3. Cài đặt cơ sở dữ liệu.....	44
3.3. Triển khai chức năng.....	45
3.3.1. Đăng nhập với Google	45
3.3.2. Đồng bộ và quản lý email	46
3.3.3. Tạo AI Reply với LangGraph.....	46
3.3.4. Chỉnh sửa và gửi Draft	47
3.3.5. Soạn email mới.....	48
3.3.6. Quản lý thông tin cá nhân	48
3.3.7. Vector Synchronization - Học văn phong người dùng.....	48
3.3.8. Dashboard Workspace.....	48
3.4. Minh họa giao diện.....	49

3.5. Kiểm thử.....	55
3.5.1. Kiểm thử hiệu năng.....	55
3.5.2. Kiểm thử trên nhiều trình duyệt.....	59
3.5.3. Kiểm thử chức năng.....	60
CHƯƠNG 4. KẾT QUẢ ĐẠT ĐƯỢC VÀ HƯỚNG PHÁT TRIỂN.....	66
4.1. Kết quả đạt được	66
4.4.1. Các chức năng đã hoàn thành.....	66
4.4.2. So sánh với mục tiêu ban đầu	67
4.2. Đánh giá hệ thống	69
4.2.1. Ưu điểm.....	69
4.2.2. Hạn chế.....	71
4.3. Hướng phát triển	73
KẾT LUẬN	76
TÀI LIỆU THAM KHẢO	79

DANH MỤC BẢNG BIỂU

Bảng 1.1. So sánh giữa mô hình Llama 3.3 70B với một số mô hình hỗ trợ Text-to-text và multilingual của Groq.....	14
Bảng 2.1. Bảng users (Người dùng).....	38
Bảng 2.2. Bảng documents (Kho dữ liệu Vector)	38
Bảng 2.3. Bảng email_drafts (Bản nháp Email)	39
Bảng 3.1. Kiểm thử chức năng.....	60

DANH MỤC HÌNH ẢNH

Hình 1.1. Các thành phần chính của kiến trúc tác tử LLM.....	8
Hình 1.2. Logo Next.js.....	11
Hình 1.3. Logo Typescript.....	12
Hình 1.4. Logo TailwindCSS	12
Hình 1.5. Logo Python.....	12
Hình 1.6. Logo FastAPI & Uvicorn.....	13
Hình 1.7. Logo Supabase & PostgreSQL	13
Hình 1.8. Logo Llama3.3 & GroqAPI	14
Hình 1.9. Logo HuggingFace.....	16
Hình 1.10. Logo LangChain & LangGraph	16
Hình 1.11. Logo Gmail	17
Hình 1.12. Logo Git & GitHub	18
Hình 1.13. Logo Visual Studio Code	18
Hình 2.1. Sơ đồ kiến trúc tổng thể	23
Hình 2.2. Sơ đồ use case toàn hệ thống	23
Hình 2.3. Biểu đồ hoạt động Đăng nhập tài khoản.....	24
Hình 2.4. Biểu đồ hoạt động Đăng xuất tài khoản.....	24
Hình 2.5. Biểu đồ hoạt động Quét email đã gửi	25
Hình 2.6. Biểu đồ hoạt động Tạo gợi ý trả lời	26
Hình 2.7. Biểu đồ hoạt động Xem và gửi email trả lời.....	27
Hình 2.8. Biểu đồ hoạt động Đồng bộ email	28
Hình 2.9. Biểu đồ hoạt động Gửi email mới.....	29
Hình 2.10. Biểu đồ hoạt động Ngắt kết nối tài khoản.....	30
Hình 2.11. Biểu đồ tuần tự Đăng nhập tài khoản.....	31
Hình 2.12. Biểu đồ tuần tự Đăng xuất tài khoản.....	31
Hình 2.13. Biểu đồ tuần tự Quét email đã gửi	32
Hình 2.14. Biểu đồ tuần tự Đồng bộ email	32
Hình 2.15. Biểu đồ tuần tự Xem và gửi email trả lời.....	33

Hình 2.16. Biểu đồ tuần tự Tạo gợi ý trả lời	34
Hình 2.17. Biểu đồ tuần tự Gửi email mới.....	35
Hình 2.18. Biểu đồ hoạt động Ngắt kết nối tài khoản.....	36
Hình 2.19. Cơ sở dữ liệu	37
Hình 3.1. Giao diện trang Đăng nhập	49
Hình 3.2. Giao diện trang xác thực, quét email đã gửi & lấy ngữ cảnh.....	50
Hình 3.3. Giao diện trang trả lời email với AI	50
Hình 3.4. Giao diện xem chi tiết email và chọn email để trả lời với AI	51
Hình 3.5. Giao diện AI hoàn tất tạo trả lời - Ảnh 1.....	51
Hình 3.6. Giao diện AI hoàn tất tạo trả lời - Ảnh 2.....	52
Hình 3.7. Giao diện đã gửi email trả lời.....	53
Hình 3.8. Giao diện trang soạn email & xem chi tiết email đã gửi.....	53
Hình 3.9. Giao diện soạn email mới	54
Hình 3.10. Giao diện soạn email mẫu & danh sách cập nhật sau gửi.....	54
Hình 3.11. Giao diện Header, menu và trang cài đặt	55
Hình 3.12. Kiểm thử hiệu năng trang mở đầu.....	57
Hình 3.13. Kiểm thử hiệu năng trang trả lời email với AI.....	57
Hình 3.14. Kiểm thử hiệu năng trang soạn email mới	57
Hình 3.15. Kiểm thử hiệu năng trang cài đặt	58
Hình 3.16. Giao diện trên trình duyệt Brave.....	59
Hình 3.17. Giao diện trên trình duyệt Chrome	59
Hình 3.18. Giao diện trên trình duyệt Microsoft Edge	60

DANH MỤC TỪ VIẾT TẮT

STT	TỪ VIẾT TẮT	GIẢI NGHĨA
1	ACID	Atomicity, Consistency, Isolation, Durability: Tính Nguyên tử, Nhất quán, Cô lập và Bền vững
2	API	Application Programming Interface: Giao diện lập trình ứng dụng
3	CORS	Cross-Origin Resource Sharing: Chia sẻ tài nguyên chéo nguồn (Cơ chế bảo mật trình duyệt)
4	CRUD	Create, Read, Update, Delete: Tạo, Đọc, Cập nhật, Xóa
5	JWT	JSON Web Token: Mã thông báo web dạng JSON
6	LLM	Large Language Model: Mô hình ngôn ngữ lớn
7	LSTM	Long Short-Term Memory: Bộ nhớ ngắn-dài hạn
8	NLP	Natural Language Processing: Xử lý ngôn ngữ tự nhiên
9	OCR	Optical Character Recognition: Nhận dạng ký tự quang học
10	RAG	Retrieval-Augmented Generation: Kỹ thuật Truy xuất - Tăng cường - Sinh
11	SEO	Search Engine Optimization: Tối ưu hóa công cụ tìm kiếm
12	SPA	Single Page Application: Ứng dụng trang đơn
13	SSG	Static Site Generation: Tạo trang tĩnh
14	SSR	Server-Side Rendering: Render phía máy chủ
15	TC	Test Case: Ca kiểm thử
16	TF-IDF	Term Frequency-Inverse Document Frequency: Tần suất xuất hiện của từ - Nghịch đảo tần suất văn bản

MỞ ĐẦU

1. Lý do chọn đề tài

Trong xã hội hiện đại ngày nay, email không chỉ là phương tiện giao tiếp mà còn là xương sống của hoạt động công việc chuyên nghiệp, ảnh hưởng đến mọi cá nhân trong cuộc sống hằng ngày. Tuy nhiên, sự gia tăng không ngừng của lượng email hàng ngày đã dẫn đến hiện tượng quá tải email (high email load), dẫn đến áp lực thời gian và gián đoạn công việc, làm suy giảm khả năng tập trung, giảm năng suất tổng thể. Nghiên cứu chỉ ra rằng một nhân viên tri thức có thể dành tới 28% thời gian làm việc chỉ để đọc và trả lời email [1], một con số đáng báo động về hiệu quả sử dụng thời gian.

Trước thực trạng đó, sự trỗi dậy của các mô hình ngôn ngữ lớn (LLM) đã mở ra một hướng tiếp cận đột phá. Thay vì các giải pháp trả lời tự động ngắn gọn và thiếu tính cá nhân, công nghệ LLM cho phép xây dựng các hệ thống thông minh có khả năng học hỏi và bắt chước văn phong của người dùng. Việc phát triển một trợ lý email cá nhân hóa không chỉ giúp giải quyết bài toán về thời gian mà còn nâng cao chất lượng và tính nhất quán trong giao tiếp, qua đó giảm bớt căng thẳng và cho phép con người tập trung vào các công việc đòi hỏi tư duy sáng tạo hơn.

2. Mục đích nghiên cứu của đề tài

Mục đích chính của đề tài là nghiên cứu và xây dựng một ứng dụng “Trợ lý Email Thông minh” có khả năng tự động gợi ý các bản nháp trả lời cho email mới đến. Hệ thống này sẽ được cá nhân hóa bằng cách học hỏi từ chính những email mà người dùng đã gửi trong quá khứ, từ đó tạo ra các câu trả lời không chỉ phù hợp về mặt nội dung mà còn nhất quán về văn phong và giọng điệu. Các mục tiêu cụ thể bao gồm: nghiên cứu kiến trúc của một Tác tử LLM (LLM Agent), áp dụng kỹ thuật Truy xuất-Tăng cường-Sinh (Retrieval-Augmented Generation - RAG) để khai thác kho dữ liệu email đã gửi, và xây dựng một ứng dụng hoàn chỉnh để chứng minh tính khả thi và hiệu quả của giải pháp. Cuối cùng, đề tài hướng đến việc giảm thiểu thời gian soạn thảo email cho người dùng, đồng thời nâng cao tính nhất quán và chuyên nghiệp trong giao tiếp điện tử của họ. Sau hết, nhóm nghiên cứu cũng hướng đến việc học

tập & bổ sung cho mình các kiến thức liên quan đến LLM cũng như kỹ thuật RAG – điều mà trước giờ nhóm chưa có cơ hội tiếp cận, thông qua việc tìm hiểu, học hỏi và thực hiện đề tài này.

3. Đối tượng và phạm vi nghiên cứu

Về đối tượng nghiên cứu: Hệ thống được thiết kế hướng đến người dùng cá nhân, bao gồm nhân viên văn phòng, người làm nghề tự do, sinh viên... phải thường xuyên phải xử lý một lượng lớn email công việc hoặc học thuật và mong muốn tối ưu hóa quy trình này.

Về phạm vi nghiên cứu: Bao gồm toàn bộ vòng đời phát triển phần mềm, từ giai đoạn nghiên cứu lý thuyết, phân tích yêu cầu, đến thiết kế hệ thống, lập trình, kiểm thử và đánh giá sản phẩm. Về chức năng, hệ thống sẽ tập trung vào các tính năng cốt lõi bao gồm xác thực người dùng an toàn qua tài khoản Google, đọc và phân tích email đã gửi để xây dựng "bộ nhớ" văn phong, và tự động tạo email nháp (draft) cho các email mới đến. Trong phiên bản đầu tiên, đề tài sẽ không đi sâu vào các tính năng nâng cao như tự động gửi email, xử lý các file đính kèm không phải ở dạng văn bản, phân tích các chuỗi hội thoại email phức tạp...

4. Phương pháp nghiên cứu

Để đảm bảo tính khoa học và hệ thống cho đề tài, nhóm thực hiện áp dụng các phương pháp nghiên cứu sau:

Nghiên cứu tài liệu: Tổng hợp và phân tích thông tin từ một số bài báo khoa học, tài liệu kỹ thuật về kiến trúc Tác tử LLM, kỹ thuật RAG, và API liên quan (Google) để xây dựng cơ sở lý thuyết vững chắc.

Phân tích và thiết kế hệ thống: Sử dụng các sơ đồ và mô hình để trực quan hóa kiến trúc hệ thống, luồng hoạt động của người dùng và dữ liệu, đảm bảo thiết kế có cấu trúc và logic.

Phát triển lặp lại và tạo mẫu (Prototyping/Iterative Development): Xây dựng hệ thống theo từng giai đoạn chức năng, liên tục kiểm thử và tinh chỉnh sản phẩm trong suốt quá trình phát triển, đảm bảo sản phẩm cuối cùng đáp ứng đúng nhu cầu.

5. Bố cục của báo cáo

Báo cáo được cấu trúc thành các chương cùng với nội dung chính như sau:

Chương 1 - Cơ sở lý thuyết & Công nghệ sử dụng: Trình bày các nền tảng lý thuyết cốt lõi về Mô hình Ngôn ngữ Lớn (LLM), kiến trúc Tác tử (Agent Architecture) và kỹ thuật Truy xuất-Tăng cường-Sinh (RAG). Chương này cũng khảo sát các hệ thống gợi ý email thông minh tương tự và giới thiệu các công nghệ được lựa chọn để xây dựng hệ thống.

Chương 2 - Phân tích yêu cầu và Thiết kế hệ thống: Đặc tả chi tiết các yêu cầu chức năng và phi chức năng của ứng dụng. Đồng thời chương này cũng trình bày thiết kế kiến trúc tổng thể, các mô hình UML và thiết kế cơ sở dữ liệu vector để quản lý bộ nhớ của Tác tử.

Chương 3 - Cài đặt & Triển khai: Mô tả về môi trường phát triển, các bước cài đặt và cấu hình hệ thống, cũng như quá trình hiện thực hóa các chức năng chính, kèm theo các hình ảnh minh họa giao diện người dùng của ứng dụng.

Chương 4 - Kết quả đạt được và Hướng phát triển: Tổng kết các kết quả mà hệ thống đã đạt được, đánh giá các ưu điểm và hạn chế của giải pháp hiện tại, dựa vào đó đề xuất các hướng cải tiến và phát triển tiềm năng trong tương lai.

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT & CÔNG NGHỆ SỬ DỤNG

1.1. Cơ sở lý thuyết

1.1.1. Hệ thống trả lời email tự động

Hệ thống trả lời email tự động có thể được định nghĩa theo cách thông thường là một hệ thống phần mềm (hoặc module) được lập trình để tự động gửi phản hồi đến các email nhận được, dựa trên các luật, mẫu (template) định trước, hoặc model xử lý ngôn ngữ/tự động, nhằm cung cấp phản hồi mà không cần sự can thiệp trực tiếp của con người. Các loại hệ thống trả lời email tự động bao gồm: Trả lời dựa trên quy tắc (Rule-based responders), Trả lời dựa trên thời gian (Time-based responders), Trả lời dựa trên sự kiện (Event-based responders), và Trả lời sử dụng trí tuệ nhân tạo (AI-powered responders) [2].

Trong đề tài này, khái niệm về trả lời email tự động được hiểu theo nghĩa rộng là hệ thống sử dụng trí tuệ nhân tạo, cụ thể là mô hình ngôn ngữ lớn LLM, để tự động tạo ra nội dung trả lời email. Tuy nhiên, để đảm bảo tính chính xác, phù hợp ngữ cảnh & văn phong của người dùng cũng như tránh rủi ro, hệ thống được thiết kế theo cơ chế bán tự động (semi-automatic), nghĩa là LLM chỉ sinh bản nháp trả lời, người dùng buộc phải xem xét, chỉnh sửa (nếu cần) và xác nhận gửi email thì email đó mới được gửi đi. Như vậy, khác với chức năng auto-reply truyền thống (gửi ngay lập tức không cần can thiệp), hệ thống trong đề tài tập trung vào việc hỗ trợ người dùng soạn thảo nhanh và thông minh hơn, nhưng vẫn giữ quyền kiểm soát cuối cùng cho con người.

1.1.2. Xử lý ngôn ngữ tự nhiên (Natural Language Processing – NLP)

Natural Language Processing là một nhánh của trí tuệ nhân tạo, tập trung vào việc giúp máy tính có khả năng hiểu, diễn giải, và tạo ra ngôn ngữ của con người một cách có ý nghĩa. Đây là sự kết hợp của nhiều lĩnh vực: khoa học máy tính, trí tuệ nhân tạo, ngôn ngữ học, toán học và thống kê. NLP hỗ trợ xây dựng các ứng dụng như nhận dạng và tổng hợp giọng nói, dịch máy, nhận dạng ký tự quang học (OCR), phân tích tình cảm, hệ thống đối thoại, v.v. [3]. Mục tiêu của NLP là xây dựng các hệ thống có khả năng tương tác với con người thông qua ngôn ngữ tự nhiên một cách tự nhiên và hiệu quả.

Một trong những thách thức lớn nhất của NLP là làm thế nào để máy tính "hiểu" được ý nghĩa của từ ngữ. Các phương pháp ban đầu như Bag-of-Words hay TF-IDF chỉ xem văn bản như một tập hợp các từ khóa rời rạc, bỏ qua ngữ cảnh và ý nghĩa. Cuộc cách mạng thực sự đến từ mô hình Word embedding, nơi biểu diễn mỗi từ hoặc câu dưới dạng một vector (một dãy số) trong không gian nhiều chiều. Các từ/câu có ngữ nghĩa tương tự sẽ có vị trí gần nhau trong không gian này. Đây chính là nền tảng cho phép máy tính thực hiện các phép "so sánh ý nghĩa", hay cơ chế vector similarity.

Vector similarity (độ tương đồng vector), một khái niệm trong toán học và khoa học dữ liệu, dùng để đo lường mức độ gần nhau giữa hai vector trong không gian đa chiều. Vector similarity không phụ thuộc vào độ dài của vector mà tập trung vào góc giữa chúng, giúp xử lý dữ liệu cao chiều hiệu quả mà không bị ảnh hưởng bởi độ lớn. Các ứng dụng phổ biến bao gồm tìm kiếm ngữ nghĩa (semantic search), phân loại văn bản, hệ thống khuyến nghị, hệ thống trả lời email tự động...

Trong đề tài này, để đánh giá độ tương đồng của hai vector, cosine similarity (độ tương đồng cosine) được sử dụng như một metric chính để đo lường độ tương đồng giữa các vector embeddings của email. Đây là một dạng vector similarity phổ biến nhất trong NLP, được tính dựa trên cosine của góc giữa hai vector trong không gian vector. Với hai vector thuộc tính A và B, độ tương đồng cosine được biểu diễn bằng tích vô hướng và độ lớn như sau:

$$\text{cosine similarity} = \frac{A \cdot B}{|A| \cdot |B|} \quad (1.1)$$

Trong đó:

- $A \cdot B$: Tích vô hướng của hai vector A và B.
- $|A|$ và $|B|$: Độ lớn Euclidean của từng vector A, B.

Theo lý thuyết, giá trị cosine similarity nằm trong khoảng $[-1, 1]$. Tuy nhiên, vì đề tài sử dụng thư viện sentence-transformer, nên hệ thống chỉ nhận các giá trị từ 0 đến 1, càng về 0 nghĩa là hai vector càng đối lập về mặt ngữ nghĩa, ngược lại càng về 1, ngữ nghĩa của chúng càng gần nhau. Trong thực tế, đặc biệt với embeddings

trong NLP, vector thường được chuẩn hóa (normalized) về độ dài bằng 1, dẫn đến cosine similarity chỉ còn là tích vô hướng giữa hai vector, nên cosine similarity chỉ phụ thuộc vào hướng, không bị ảnh hưởng bởi độ dài vector, rất phù hợp cho văn bản có độ dài khác nhau, đồng thời tốc độ tính toán cũng nhanh hơn.

Ưu điểm của độ đo tương đồng cosine nằm ở việc tính toán hiệu quả ở không gian nhiều chiều (high-dimensional space), cũng như không nhạy cảm với độ dài vector, lý tưởng cho email ngắn/dài khác nhau, và tích hợp dễ với LLM và RAG. Tuy nhiên nhược điểm của độ đo này là không xem xét thứ tự từ, nên thường kết hợp với embedding chất lượng cao.

1.1.3. Mô hình ngôn ngữ lớn (Large Language Model – LLM)

Các mô hình ngôn ngữ lớn là bước tiến hóa vượt bậc, với hiệu suất vượt trội trong nhiều tác vụ xử lý ngôn ngữ tự nhiên nhờ khả năng tạo ra ngôn ngữ giống con người và tiềm năng cách mạng hóa khoa học và công nghệ.

Mô hình ngôn ngữ lớn (LLM) là một loại thuật toán trí tuệ nhân tạo sử dụng các kỹ thuật học sâu và bộ dữ liệu cực lớn để hiểu ngữ nghĩa sâu của văn bản, tóm tắt, tạo và dự đoán nội dung mới một cách mạch lạc, tự nhiên và phù hợp với ngữ cảnh, cũng như thực hiện nhiều tác vụ NLP chỉ bằng prompt. Một số LLM tiêu biểu được nhiều người sử dụng hiện nay có thể kể đến GPT (OpenAI), Claude (Anthropic), LLaMA (Meta) hay Gemini (Google)...

Trong đề tài này, LLM đóng vai trò là hạt nhân suy luận và tạo sinh (reasoning and generation core). Sau khi nhận được ngữ cảnh từ các email đã gửi, LLM sẽ vận dụng khả năng hiểu ngôn ngữ sâu sắc của mình và phân tích ý định người gửi để tạo ra một email trả lời hoàn chỉnh, mô phỏng lại văn phong của người dùng.

1.1.4. Kiến trúc Tác tử (Agentic Architecture)

Các chương trình LLM thuần có những hạn chế đáng kể như thiếu bộ nhớ dài hạn, không thể tự động tương tác với các công cụ bên ngoài, gặp khó khăn trong việc theo đuổi mục tiêu... làm cản trở hiệu quả làm việc của chúng trong các trường hợp đòi hỏi suy luận liên tục hoặc quy trình làm việc nhiều bước [4]. Để khắc phục những hạn chế này, các LLM sẽ cần hoạt động như những tác tử (agents).

Trong lĩnh vực Trí tuệ nhân tạo, một Tác tử (Agent) không chỉ đơn thuần là một mô hình học máy, mà là một hệ thống tự trị có khả năng nhận thức môi trường, suy luận để đưa ra quyết định và hành động để đạt được mục tiêu cụ thể. Đối với đề tài này, việc xây dựng một trợ lý email không chỉ dừng lại ở việc gọi một mô hình ngôn ngữ, mà là thiết kế một Tác tử AI hoàn chỉnh. Kiến trúc của một hệ thống tác tử LLM gồm bốn thành phần chính:

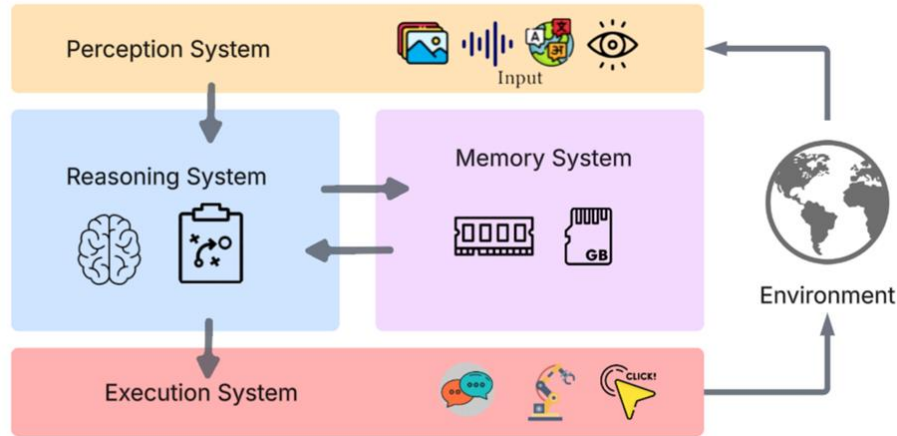
Hệ thống Nhận thức (Perception System): Là khả năng của Tác tử thu thập thông tin và xử lý từ môi trường bên ngoài, chẳng hạn như hình ảnh, âm thanh, hoặc bất kỳ hình thức thông tin nào khác. Nhiệm vụ của nó là biến đổi thông tin này thành các biểu diễn có ý nghĩa mà LLM có thể hiểu và sử dụng [4]. Trong bối cảnh của dự án, đây chính là cơ chế sử dụng Gmail API để đọc và phân tích cấu trúc của các email mới đến.

Hệ thống Suy luận (Reasoning System): Đây là “bộ não” của Tác tử. Nó không chỉ bao gồm Mô hình Ngôn ngữ Lớn (LLM) để sinh văn bản, mà còn bao gồm một kế hoạch trong đó chứa toàn bộ luồng logic điều phối các quyết định, và nó cũng chịu trách nhiệm điều chỉnh kế hoạch này dựa trên phản hồi từ môi trường và đánh giá các hành động của chính nó để sửa lỗi hoặc cải thiện hiệu quả thực hiện [4]. Hệ thống sử dụng các kỹ thuật như Prompt Engineering để chỉ dẫn cho LLM, đảm bảo câu trả lời được tạo ra phù hợp với ngữ cảnh và yêu cầu.

Hệ thống Bộ nhớ (Memory System): Để vượt qua hạn chế về trí nhớ ngắn hạn của các LLM truyền thống, tác tử cần một bộ nhớ dài hạn. Điều này bao gồm mọi thứ từ kinh nghiệm trong quá khứ đến các tài liệu liên quan và dữ liệu có cấu trúc được lưu trữ trong cơ sở dữ liệu quan hệ [4]. Trong đề tài này, “bộ nhớ” chính là kho tri thức được xây dựng từ các email đã gửi của người dùng. Kỹ thuật RAG được áp dụng để cho phép Tác tử “nhớ lại” và truy xuất các thông tin liên quan từ bộ nhớ này nhằm nâng cao độ chính xác của các phản hồi, cụ thể hơn là cá nhân hóa câu trả lời.

Hệ thống Thực thi (Execution System): Là khả năng của Tác tử biến các quyết định thành hành động cụ thể. Mô-đun này đảm bảo rằng các hướng dẫn của tác nhân được thực hiện trong thế giới thực hoặc mô phỏng, hoàn thành quy trình tương tác

bằng cách thực hiện những gì đã được quyết định trước đó [4]. Ở đây, đó là việc sử dụng Gmail API tạo và lưu một email nháp vào tài khoản của người dùng, hoàn thành mục tiêu mà không cần can thiệp thủ công.



Hình 1.1. Các thành phần chính của kiến trúc tác tử LLM

Nguồn: *Fundamentals of Building Autonomous LLM Agents (2025)*

1.1.5. Kỹ thuật Sinh Tăng cường Truy xuất (Retrieval-Augmented Generation - RAG)

Nếu LLM là bộ não, thì RAG chính là cơ chế cung cấp “trí nhớ dài hạn” và kiến thức chuyên biệt cho bộ não đó. Các LLM thông thường chỉ có kiến thức tổng quát từ dữ liệu huấn luyện và không biết gì về dữ liệu cá nhân, riêng tư của người dùng (như các email đã gửi). RAG là một kỹ thuật nâng cao LLM bằng cách sử dụng kiến thức bên ngoài để cải thiện độ chính xác của phản hồi. Quy trình của RAG gồm ba bước như sau:

Truy xuất (Retrieve): Trong RAG, việc truy xuất được thực hiện bằng cách tính toán độ tương đồng giữa các vector được nhúng (embedding) của các email đã gửi và email cần trả lời. Điều này cho phép LLM truy cập thông tin cập nhật và chính xác, cải thiện các phản hồi.

Tăng cường (Augment): Sau khi các thông tin liên quan được truy xuất, nó sẽ được thêm vào ngữ cảnh trong câu lệnh (prompt) cùng với nội dung email cần trả lời. Sự tăng cường đầu vào này sẽ cho phép LLM tạo ra các phản hồi dựa trên các tính chất mang tính cá nhân, giúp phản hồi chính xác cho từng trường hợp sử dụng

cụ thể và giảm khả năng xảy ra “ảo giác” (hallucinations).

Sinh (Generate): Câu lệnh đã được tăng cường ngữ cảnh này sẽ được gửi đến LLM. Giờ đây, LLM có đầy đủ thông tin để từ đó tạo ra một câu trả lời chất lượng.

1.2. Khảo sát các hệ thống đã có

Trong bối cảnh số lượng email gửi-nhận ngày càng tăng, nhu cầu phản hồi nhanh chóng và chính xác đang trở thành thách thức đối với cá nhân và doanh nghiệp. Việc phân tích các nghiên cứu đã có, chỉ ra hạn chế và định hướng cần tập trung là cần thiết nhằm xác định đóng góp khoa học và giá trị ứng dụng của đề tài.

Các hướng nghiên cứu chính hiện nay có thể được phân chia thành bốn mảng. Thứ nhất, nền tảng của việc gợi ý email trả lời ngắn tự động được thực hiện bởi một nhóm thuộc Google với tên gọi “Smart Reply”, trong đó nhóm tác giả đề xuất mô hình end-to-end dựa trên LSTM để mã hóa nội dung email và sinh các câu trả lời ngắn. Hệ thống đã được triển khai thực tế trên Gmail và phục vụ hàng trăm triệu tin nhắn mỗi ngày [5], chứng minh tính khả thi của tiếp cận này. Thứ hai, với sự nổi lên của các mô hình ngôn ngữ lớn (LLM), nhiều nghiên cứu đã thiên về mở rộng khả năng của hệ thống trả lời email theo hướng tạo nội dung phức tạp hơn, có ngữ cảnh, tính cá nhân hóa và hỗ trợ môi trường cộng tác. Một ví dụ điển hình là hệ thống LLM-based Smart Reply (LSR), trong đó nhóm tác giả xem xét vai trò của LLM trong việc hỗ trợ làm việc nhóm (group collaboration) và giúp người dùng soạn thảo tin nhắn [6].

Thứ ba, một hướng nghiên cứu quan trọng nhằm nâng cao chất lượng gợi ý trả lời email là tối ưu hoá quá trình retrieval thông qua huấn luyện end-to-end. Công trình “End-to-End Autoregressive Retrieval via Bootstrapping for Smart Reply Systems” chỉ ra rằng việc huấn luyện đồng thời toàn bộ mô hình truy xuất (retrieval) dưới dạng autoregressive text-to-text retrieval giúp cải thiện đáng kể độ phù hợp (relevance) và độ đa dạng (diversity) của tập câu trả lời gợi ý. Khác với pipeline truyền thống (retrieval $\rightarrow$ shortlist $\rightarrow$ diversification/ranking), mô hình này xây dựng reply-set bằng phương pháp bootstrapping và học trực tiếp mối quan hệ giữa message và reply-set trong một mô hình duy nhất. Kết quả thực nghiệm cho thấy mức cải thiện độ phù

hợp từ 5,1% đến 17,9% và độ đa dạng từ 0,5% đến 63,1% so với các phương pháp hiện tại [7].

Thứ tư, một số nghiên cứu gần đây không chỉ dừng ở việc thiết kế mô hình sinh phản hồi, mà còn chú trọng xây dựng tập dữ liệu phù hợp, phương pháp đánh giá và cả thử nghiệm người dùng/ thực nghiệm thực tế. Ví dụ, Towle & Zhou (2023) đề xuất phương pháp mô phỏng hành vi người dùng để tạo “sets of replies” nhằm cải thiện độ đa dạng và khả năng ít nhất một phản hồi đúng mong muốn trong hệ thống Smart Reply [8]. Ngoài ra, từ phía thực tế ứng dụng, hệ thống Smart Reply của Gmail (Google Research) cũng được phát triển theo cách đảm bảo hiệu năng, tốc độ và khả năng áp dụng rộng, đặc biệt là việc tạo ra một mô hình đủ nhỏ và hiệu quả để chạy trực tiếp trên thiết bị di động, chú trọng yếu tố triển khai thực tế [9].

Mặc dù các hướng nghiên cứu trên đã đạt được những thành tựu đáng kể, song vẫn tồn tại những khoảng trống công nghệ rõ rệt khi đối chiếu với nhu cầu thực tiễn. Thứ nhất, về sự đánh đổi giữa độ phức tạp ngữ nghĩa và hiệu năng, các phương pháp tiếp cận truyền thống như Smart Reply hay phiên bản tối ưu hóa cho di động tập trung tối đa vào tốc độ và khả năng triển khai trên diện rộng, nhưng kiến trúc LSTM và giới hạn tính toán khiến chúng thường chỉ dừng lại ở việc gợi ý các câu trả lời ngắn, thiếu chiều sâu. Ngược lại, các nghiên cứu ứng dụng LLM như LSR tuy giải quyết được vấn đề tạo sinh nội dung phức tạp cho cộng tác nhóm, nhưng việc phụ thuộc vào các API thương mại (như ChatGPT) thường dẫn đến độ trễ cao và phụ thuộc vào kết nối mạng, chưa đạt được trải nghiệm thời gian thực.

Thứ hai, về chất lượng cá nhân hóa, các nỗ lực gần đây trong việc cải thiện mô hình truy xuất hay mô phỏng hành vi người dùng đã thành công trong việc tăng độ đa dạng (diversity) và độ phù hợp (relevance) của tập câu trả lời. Tuy nhiên, các nghiên cứu này chủ yếu tập trung vào việc chọn đúng ý định (intent) hơn là sao chép văn phong (writing style) đặc trưng của người gửi. Cơ chế 'bootstrapping' giúp tìm ra câu trả lời đúng, nhưng chưa đảm bảo câu trả lời đó nghe giống như chính người dùng viết ra.

Thứ ba, về vấn đề ảo giác và ngữ cảnh, mặc dù đã đưa LLM vào quy trình, nhưng nguy cơ “ảo giác” (hallucination) vẫn hiện hữu nếu thiếu một cơ chế ràng buộc dữ liệu (grounding) chặt chẽ từ lịch sử email cá nhân, đặc biệt trong các chuỗi hội thoại dài mà các mô hình LSTM thường gặp khó khăn trong việc ghi nhớ.

Từ những hạn chế trên, đề tài “Ứng dụng LLM để phát triển ứng dụng trả lời email tự động” cần tập trung vào một số hướng chính để đảm bảo tính mới và khả năng ứng dụng. Cụ thể, đề tài đề xuất giải pháp sử dụng mô hình ngôn ngữ lớn (Llama-3.3-70B) thông qua Groq LPU để giải quyết bài toán hiệu năng, cũng như kết hợp retrieval và generation theo pipeline RAG là cần thiết nhằm đảm bảo câu trả lời dựa trên dữ liệu thật, giảm sai lệch thông tin, nhằm tạo ra các phản hồi vừa nhanh, vừa sâu sắc, vừa mang đậm dấu ấn cá nhân. Bên cạnh đó, đề tài cũng hỗ trợ cơ chế human-in-the-loop cho phép người dùng chỉnh sửa kết quả (email trả lời) và mô hình học từ điều chỉnh đó, thay vì tự gửi đi như các hệ thống trả lời tự động hiện tại.

1.3. Công nghệ sử dụng

Các yếu tố quyết định đến hiệu suất, khả năng mở rộng và bảo trì của hệ thống phụ thuộc vào việc lựa chọn công nghệ phù hợp. Dựa trên các yêu cầu đặc thù của một ứng dụng tích hợp trí tuệ nhân tạo, một tech stack thích hợp đã được lựa chọn, bao gồm frontend, backend, một hệ thống cơ sở dữ liệu phù hợp để quản lý các dữ liệu người dùng, đặc biệt là dữ liệu vector, và các công nghệ liên quan cũng như hỗ trợ cho LLM. Chi tiết các công nghệ sử dụng đang liệt kê dưới đây:

1.3.1. Frontend



Hình 1.2. Logo Next.js

Nguồn: Internet

Next.js là framework React chính thức do Vercel phát triển, được sử dụng làm nền tảng chính cho giao diện người dùng trong dự án này, bao gồm trang đăng nhập, trang trả lời email, trang soạn email và trang cài đặt. Next.js vượt trội hơn so với React thông thường bởi các tính năng nâng cao như server-side rendering (SSR) và

static site generation (SSG). Những điều này giúp ứng dụng web tải nhanh hơn, tăng SEO và cải thiện trải nghiệm người dùng.



Hình 1.3. Logo Typescript

Nguồn: Internet

TypeScript được sử dụng thay cho JavaScript thuần túy nhằm đảm bảo an toàn kiểu dữ liệu (type safety), mang lại cơ chế type-checking và giảm thiểu lỗi runtime, tăng năng suất phát triển, làm cho việc cộng tác trong dự án trở nên dễ dàng hơn. TypeScript đặc biệt hữu ích với các dự án AI/web có nhiều logic phức tạp, giúp code dễ đọc và dễ bảo trì hơn.



Hình 1.4. Logo TailwindCSS

Nguồn: Internet

Tailwind CSS là framework CSS được dùng để xây dựng thiết kế theo phương pháp utility-first, giúp tạo style nhanh, hỗ trợ xây dựng giao diện responsive, đảm bảo tính nhất quán thiết kế và tối ưu kích thước CSS thông qua kỹ thuật tree-shaking. Nhờ đó, frontend nhẹ, hiệu năng cao và dễ tùy chỉnh.

1.3.2. Backend



Hình 1.5. Logo Python

Nguồn: Internet

Python được chọn làm ngôn ngữ chính cho backend, một ngôn ngữ phổ biến trong xử lý AI, NLP và xây dựng hệ thống server. Đây là ngôn ngữ thống trị trong lĩnh vực Trí tuệ Nhân tạo và Khoa học Dữ liệu, vì toàn bộ hệ sinh thái AI, như các thư viện cốt lõi và các thư viện xử lý vector đều được phát triển và hỗ trợ tốt nhất trên Python, cho phép tích hợp liền mạch và khai thác tối đa sức mạnh của chúng.

**Hình 1.6. Logo FastAPI & Uvicorn**

Nguồn: Internet

FastAPI được chọn làm web framework chính vì những ưu điểm vượt trội về hiệu năng và tốc độ phát triển. Được xây dựng trên nền tảng Starlette và Pydantic, FastAPI hỗ trợ lập trình bất đồng bộ (async/await) một cách tự nhiên, cực kỳ lý tưởng cho các tác vụ chờ đợi I/O như gọi đến API của Google, giúp tối ưu hóa hiệu suất của hệ thống. Hơn nữa, FastAPI có khả năng tự động tạo tài liệu API tương tác (Swagger UI), giúp đơn giản hóa quá trình kiểm thử và tích hợp với frontend. FastAPI kết hợp cùng Uvicorn - server tốc độ cao, cho phép backend vận hành ổn định, xử lý đồng thời nhiều request và hỗ trợ cho các tác vụ như cập nhật trạng thái xử lý email.

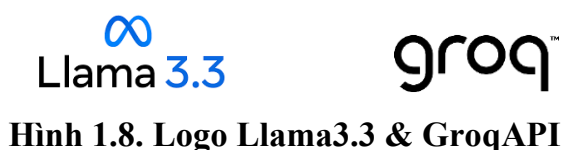
1.3.3. Cơ sở dữ liệu và lưu trữ vector

**Hình 1.7. Logo Supabase & PostgreSQL**

Nguồn: Internet

Dữ liệu của hệ thống được lưu trữ bằng PostgreSQL, triển khai dưới dạng managed database thông qua hạ tầng Supabase. Supabase dùng PostgreSQL làm database nền tảng. PostgreSQL tuân thủ ACID, hỗ trợ JSON, full-text search tiếng Việt và có khả năng mở rộng tốt cho những ứng dụng dữ liệu lớn. Điểm nổi bật của hệ thống là sử dụng pgvector, extension cho phép lưu trữ và truy vấn vector embeddings, phục vụ cho quá trình tìm email tương tự hoặc gợi ý nội dung trả lời. Supabase cung cấp thêm dịch vụ Authentication, API tự động và Realtime, giúp đơn giản hoá phần backend mà vẫn đảm bảo bảo mật thông qua Row Level Security.

1.3.4. Mô hình ngôn ngữ lớn & Công nghệ hỗ trợ



Hình 1.8. Logo Llama3.3 & GroqAPI

Nguồn: Internet

Để giải quyết bài toán độ trễ (latency) trong các hệ thống tương tác thời gian thực, đề tài lựa chọn sử dụng nền tảng Groq Cloud thay vì triển khai trên GPU truyền thống. Công nghệ cốt lõi của Groq là vi xử lý LPU (Language Processing Unit), một kiến trúc phần cứng được thiết kế chuyên biệt cho tác vụ suy luận của các mô hình ngôn ngữ lớn (LLM Inference). Khác với GPU sử dụng kiến trúc song song tối ưu cho huấn luyện, LPU vận hành theo kiến trúc dòng dữ liệu đơn hướng (single-core streaming architecture), qua đó giảm thiểu các nút thắt cổ chai liên quan đến băng thông bộ nhớ. Trên nền tảng này, dự án lựa chọn mô hình Llama 3.3 70B Versatile và triển khai thông qua Groq Cloud API nhằm đạt được sự cân bằng giữa năng lực suy luận mạnh mẽ của mô hình 70 tỷ tham số, vốn khó có thể vận hành hiệu quả trên phần cứng cá nhân, và tốc độ xử lý gần như tức thì nhờ kiến trúc LPU. Bên cạnh đó, việc sử dụng Groq API vẫn đảm bảo yêu cầu về tính riêng tư do dữ liệu người dùng không được sử dụng cho mục đích huấn luyện, đồng thời tối ưu hóa độ trễ để đáp ứng các yêu cầu của ứng dụng tương tác thời gian thực.

Bảng 1.1. So sánh giữa mô hình Llama 3.3 70B với một số mô hình hỗ trợ Text-to-text và multilingual của Groq

Mô hình	Tổng tham số	Tham số hoạt động (Active)	Độ dài ngữ cảnh/ Input
GPT OSS 120B	~117B	~5.1B	128k tokens
GPT OSS 20B	~21B	~3.6B	128k tokens
Kimi K2	~1 trillion (1T)	~32B	-
Llama 4 Scout	~109B	~17B	~10 M tokens
Llama 3.3 70B	~70B	~70B	~128k tokens
Whisper Large v3	~1.5B	~1.5B	Ghi âm/ audio

Nguồn: Internet

Một vài lý do cho thấy mô hình Llama 3.3 70B Versatile là lựa chọn tối ưu so với các mô hình khác. Thứ nhất, về sức mạnh suy luận và mật độ tri thức vượt trội, Llama 3.3 70B mang lại hiệu năng tương đương với các mô hình đồng như GPT-4 nhưng với chi phí thấp hơn nhiều. Việc sở hữu 70 tỷ tham số giúp mô hình xử lý tốt các tác vụ "nặng" như phân tích luồng email dài (thread), duy trì văn phong (tone) nhất quán và không bị nhầm lẫn giữa các thực thể trong văn bản – điều mà các mô hình nhỏ như 8B hay 20B thường gặp khó khăn.

Thứ hai, mô hình này giải quyết triệt để thách thức về độ trễ. Thông thường, mô hình càng lớn thì tốc độ càng chậm. Tuy nhiên, thông qua hạ tầng Groq, Llama 3.3 70B đạt được tốc độ xử lý cực cao, tạo ra trải nghiệm người dùng liền mạch và chuyên nghiệp. Thứ ba, Llama 3.3 hỗ trợ đa ngôn ngữ. Qua thử nghiệm, mô hình nắm bắt tốt ngữ pháp, văn phong trang trọng trong tiếng Việt, hạn chế tối đa hiện tượng dịch nguyên văn từ tiếng Anh, giúp email phản hồi tự nhiên và gần gũi.

Thứ tư, về tính bảo mật và quyền riêng tư dữ liệu, khác với các API thương mại thông thường, Groq cam kết chính sách bảo mật nghiêm ngặt cho người dùng doanh nghiệp/nhà phát triển. Theo chính sách hiện tại, dữ liệu truyền qua API của Groq không được sử dụng để huấn luyện lại mô hình (Zero-retention policy), đảm bảo các thông tin nhạy cảm trong email khách hàng luôn được giữ kín. Thứ năm, về sự ổn định và khả năng tích hợp, Llama 3.3 70B Versatile tuân thủ cực tốt các yêu cầu về định dạng đầu ra (như JSON mode), điều này rất quan trọng khi kết hợp với các framework như LangChain hay LangGraph để xây dựng các Agent tự động. So với các mô hình mới như Kimi K2 hay Llama 4 Scout, Llama 3.3 là một lựa chọn ổn định cho môi trường vận hành thực tế.

Cuối cùng, việc lựa chọn mô hình cho đề án này không dựa trên cuộc đua về tổng số tham số (Parameters), mà tập trung vào mức độ phù hợp với bài toán email và ngưỡng tiếp cận về chi phí. Một mô hình quá lớn (như 120B) sẽ gây lãng phí tài nguyên và độ trễ cao, trong khi mô hình quá nhỏ (dưới 20B) lại không đủ khả năng xử lý các ngữ cảnh email phức tạp. Vì thế Llama 3.3 70B rất phù hợp về hiệu suất. Hơn nữa, Llama 3.3 70B thông qua Groq hiện được coi là mô hình “miễn phí

thực sự” dành cho cộng đồng phát triển. Groq cung cấp hạn mức miễn phí với tốc độ xử lý LPU vượt trội, cho phép sinh viên và các nhà nghiên cứu vận hành một mô hình cấp độ doanh nghiệp mà không cần đầu tư vào hệ thống phần cứng đắt tiền hay trả phí API đắt đỏ như các mô hình đóng khác.

Mặc dù là mô hình mạnh mẽ trong phân khúc tầm trung, Llama 3.3 70B vẫn có những giới hạn nhất định. Do vẫn dựa trên kiến trúc Transformer, mô hình đôi khi vẫn có thể gặp hiện tượng “ảo giác” nếu dữ liệu đầu vào từ RAG bị nhiễu hoặc không đầy đủ. Ngoài ra, việc phụ thuộc vào API đám mây (Groq) đòi hỏi một kết nối internet ổn định, mặc dù chi phí vận hành thấp hơn nhiều so với việc tự duy trì một dàn máy chủ GPU A100/H100 để chạy mô hình 70B tại chỗ. Tuy nhiên, với các ưu điểm về tốc độ và chất lượng phản hồi, đây hiện là giải pháp cân bằng nhất cho bài toán tự động hóa email thông minh.



Hình 1.9. Logo HuggingFace

Nguồn: Internet

Để tạo vector embeddings phục vụ truy vấn ngữ nghĩa (semantic search), hệ thống sử dụng mô hình sentence-transformers/all-MiniLM-L6-v2 từ HuggingFace Transformers. Đây là mô hình nhẹ, tốc độ cao, đa ngôn ngữ, phù hợp cho các bài toán email classification, clustering và tìm kiếm ngữ nghĩa. Trong đề tài này, HuggingFace được sử dụng để vector hóa các email đã gửi, là cơ sở ngữ cảnh cho LLM tạo các email trả lời mang tính cá nhân hóa nhiều hơn.



Hình 1.10. Logo LangChain & LangGraph

Nguồn: Internet

LangChain đóng vai trò là lớp trung gian linh hoạt, kết nối các module AI trong hệ thống mà không cần tái cấu trúc mã nguồn khi thay đổi công nghệ. LangChain cung cấp các công cụ thiết yếu như RecursiveCharacterTextSplitter để

chia nhỏ nội dung email nhằm bảo toàn ngữ cảnh, và PromptTemplate để cấu trúc hóa câu lệnh động. Nhờ kiến trúc trừu tượng hóa cao, LangChain đảm bảo tính mở rộng và sự nhất quán trong việc quản lý dữ liệu đầu vào cho LLM.

LangGraph - phần mở rộng của LangChain - được sử dụng để xây dựng kiến trúc máy trạng thái (state machine) cho quy trình xử lý email tự động. Hệ thống định nghĩa một luồng công việc gồm 4 node tuần tự: Fetch Email → Retrieve Context (RAG) → Generate Reply → Create Draft. Thông qua StateGraph, trạng thái hệ thống (GraphState chứa user_id, context, draft...) được duy trì và chuyển giao giữa các node, cho phép kiểm soát chặt chẽ logic nghiệp vụ và xử lý lỗi tại từng bước. So với các mô hình Agent tự quyết (như ReACT), cách tiếp cận này giúp loại bỏ hallucination, tối ưu chi phí tài nguyên và đảm bảo quy trình vận hành ổn định, minh bạch từ đầu đến cuối.

1.3.5. Xác thực người dùng



Hình 1.11. Logo Gmail

Nguồn: Internet

Để truy cập và xử lý email thực, hệ thống tích hợp Gmail API & xác thực người dùng thông qua giao thức OAuth 2.0 (Google sử dụng OAuth 2.0 (và OIDC) để hỗ trợ đăng nhập bằng tài khoản Google), cho phép ứng dụng đọc email, gửi email và theo dõi hộp thư mà không cần truy cập mật khẩu người dùng. Các phiên đăng nhập và phân quyền được quản lý bằng JSON Web Token (JWT), đảm bảo giao tiếp giữa frontend và backend an toàn.

1.3.6. Công cụ phát triển



Hình 1.12. Logo Git & GitHub

Hệ thống sử dụng Git và GitHub làm công cụ quản lý phiên bản, hỗ trợ cộng tác nhóm, quản lý branch và triển khai CI/CD khi cần thiết. Các workflow Git giúp theo dõi lịch sử phát triển, rollback và tích hợp dễ dàng với môi trường triển khai container hoặc cloud.



Hình 1.13. Logo Visual Studio Code

Nguồn: Internet

Visual Studio Code (VS Code) là trình soạn thảo mã nguồn (code editor) nhẹ, rất phổ biến, hỗ trợ nhiều extension cho nhiều ngôn ngữ, thư viện khác nhau (TypeScript, React, Tailwind CSS, Python...). VS Code được sử dụng làm môi trường phát triển cho cả frontend & backend, với nhiều khả năng hữu ích, mạnh mẽ như debugging, tích hợp Git, cũng như phối hợp hiệu quả với các extensions, góp phần hỗ trợ tối đa cho việc phát triển dự án.

CHƯƠNG 2. PHÂN TÍCH YÊU CẦU VÀ THIẾT KẾ HỆ THỐNG

2.1. Phân tích yêu cầu

2.1.1. Yêu cầu chức năng

Yêu cầu chức năng (Functional Requirements - FR) mô tả các hành vi, chức năng mà hệ thống phải thực hiện, bao gồm:

FR1: Quản lý xác thực và tài khoản người dùng

- Người dùng phải có khả năng đăng ký tài khoản mới thông qua cơ chế xác thực an toàn của Google (Google OAuth2).
- Người dùng phải có khả năng đăng nhập vào hệ thống bằng tài khoản Google đã liên kết, hệ thống tự động kiểm tra và chuyển hướng người dùng đã đăng nhập về trang workspace.
- Hệ thống phải yêu cầu và nhận được sự cho phép của người dùng để truy cập các quyền cần thiết trên Gmail bao gồm: đọc email, tạo và quản lý nháp, gửi email, và truy cập thông tin profile người dùng.
- Người dùng phải có khả năng đăng xuất khỏi hệ thống một cách an toàn, chấm dứt phiên làm việc hiện tại.
- Người dùng chưa đăng nhập khi truy cập các trang yêu cầu xác thực (như workspace) phải được tự động chuyển hướng về trang đăng nhập.
- Hệ thống phải tạo và quản lý JWT token với thời hạn 7 ngày để xác thực các API requests từ frontend.
- Hệ thống phải lưu trữ Google Refresh Token của người dùng để tự động làm mới Access Token khi hết hạn mà không yêu cầu người dùng đăng nhập lại.

FR2: Quản lý bộ nhớ và Cá nhân hóa

- Sau khi đăng nhập, người dùng phải có khả năng kích hoạt thủ công tác vụ đồng bộ để quét và phân tích các email từ thư mục "Đã gửi" của người dùng.
- Tác vụ đồng bộ phải chạy dưới dạng background task không chặn giao diện người dùng, với khả năng hiển thị tiến trình thông qua UI.

- Nội dung các email đã gửi phải được chuyển đổi thành dạng vector và lưu trữ trong cơ sở dữ liệu có khả năng lưu ở dạng vector, trong đó liên kết với thông tin của người dùng.
- Vector embeddings phải được lưu trữ trong cơ sở dữ liệu PostgreSQL (qua Supabase) với pgvector extension, kèm theo metadata.
- Hệ thống phải kiểm tra và bỏ qua các email đã được vector hóa trước đó để tránh trùng lặp dữ liệu.

FR3: Chức năng gợi ý trả lời email

- Hệ thống phải có khả năng hiển thị danh sách các email mới nhận trong hộp thư đến của người dùng.
- Khi người dùng chọn một email, hệ thống phải tự động hiển thị nội dung chi tiết email (subject, sender, body) và kiểm tra xem đã có draft suggestion hay chưa.
- Người dùng phải có khả năng chọn nhiều email cùng lúc (tối đa 5) để tạo gợi ý AI hàng loạt.
- Luồng xử lý AI bao gồm các bước: (1) đọc nội dung email được chọn từ Gmail, (2) tìm kiếm trong cơ sở dữ liệu vector để lấy ra top 3 email chunks có ngữ cảnh liên quan nhất, (3) tạo prompt template kết hợp context và nội dung email gốc, sau đó gọi LLM để sinh ra nội dung trả lời có subject và body, (4) tạo Gmail draft với thread references và lưu metadata vào Supabase.
- Nội dung trả lời do AI tạo ra phải được hiển thị cho người dùng trong một giao diện soạn thảo.
- Hệ thống phải hiển thị loading state rõ ràng trong quá trình AI đang xử lý.

FR4: Tương tác với email gợi ý

- Người dùng phải có khả năng chỉnh sửa (edit) nội dung của email trả lời đã được hệ thống gợi ý.
- Hệ thống phải có khả năng tạo một email nháp (Draft) trong tài khoản Gmail của người dùng với nội dung do AI gợi ý, thay vì gửi trực tiếp.

- Sau khi xem xét và chỉnh sửa (nếu có), người dùng phải có khả năng thực hiện hành động "Gửi" email trực tiếp từ giao diện của hệ thống.
- Email gợi ý phải được tự động tạo thành Draft trong Gmail với thread references để Gmail nhận diện đây là email trả lời trong conversation.
- Draft metadata phải được lưu trữ song song trong Supabase để hỗ trợ các tình huống draft bị xóa khỏi Gmail.
- Người dùng phải có khả năng xóa và tạo lại gợi ý nếu không hài lòng với kết quả, hệ thống sẽ xóa draft cũ và chạy lại toàn bộ workflow AI.
- Nếu draft đã bị xóa khỏi Gmail nhưng còn trong Supabase, hệ thống phải có khả năng gửi email trực tiếp từ nội dung Supabase thay vì báo lỗi.

2.1.2. Yêu cầu phi chức năng

Yêu cầu phi chức năng (Non-Functional Requirements) xác định các tiêu chí chất lượng, ràng buộc và thuộc tính vận hành của hệ thống, bao gồm:

NFR1: Hiệu năng (Performance)

- Thời gian phản hồi của hệ thống để tạo và hiển thị một email gợi ý không vượt quá 10 giây.
- Thời gian tải trang ban đầu của ứng dụng không được vượt quá 10 giây.
- Các tương tác giao diện người dùng cơ bản (như chọn email, chỉnh sửa văn bản) phải có thời gian phản hồi dưới 10 giây.

NFR2: Bảo mật (Security)

- Tất cả các thông tin nhạy cảm như token truy cập Google phải được lưu trữ một cách an toàn phía máy chủ và được mã hóa khi truyền tải.
- Hệ thống phải sử dụng cơ chế xác thực JWT (JSON Web Token) để bảo vệ các API endpoint, đảm bảo chỉ người dùng đã đăng nhập mới có thể truy cập tài nguyên của mình.
- Dữ liệu của người dùng này phải được phân tách hoàn toàn và không thể bị truy cập bởi người dùng khác.

NFR3: Tính khả dụng (Availability)

- Hệ thống phải đảm bảo độ sẵn sàng cao, hoạt động ổn định và giảm thiểu thời gian chết (downtime).
- Hệ thống phải có cơ chế ghi log (logging) chi tiết để theo dõi các lỗi phát sinh trong quá trình gọi API đến các dịch vụ bên ngoài và dễ dàng gỡ lỗi.
- Mỗi node trong LangGraph workflow phải có error handling riêng, cho phép một số nodes thất bại (như RAG) mà không crash toàn bộ workflow.

NFR4: Giao diện thân thiện và Trải nghiệm người dùng (Usability & User Experience)

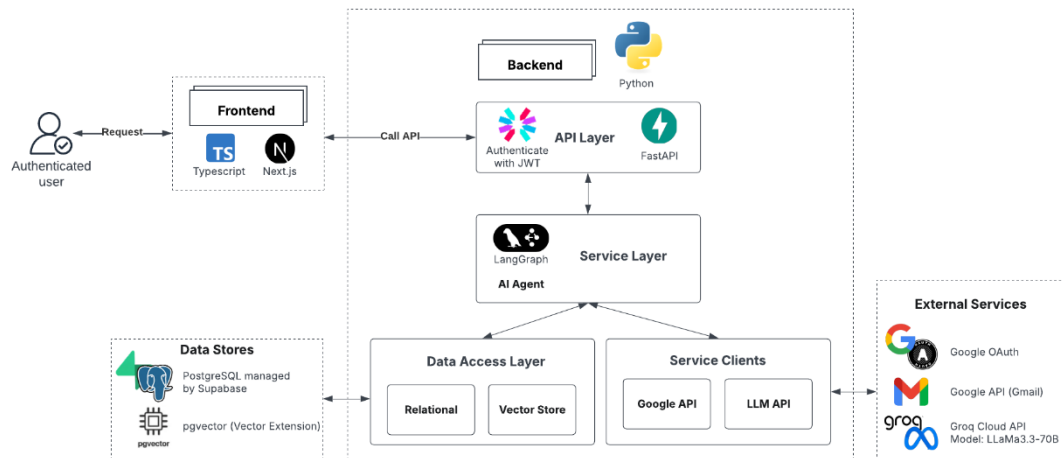
- Giao diện người dùng phải được thiết kế trực quan, rõ ràng, dễ điều hướng.
- Người dùng mới phải có thể sử dụng các chức năng cốt lõi mà không cần sự hướng dẫn chi tiết.
- Hệ thống phải cung cấp các phản hồi trực quan cho người dùng trong quá trình xử lý tác vụ nền hoặc các tác vụ mất thời gian (ví dụ: hiển thị spinner loading khi đang tạo gợi ý).
- Hệ thống phải hiển thị trạng thái email rõ ràng (đã có gợi ý).
- Người dùng chọn quá 5 emails phải nhận được cảnh báo rõ ràng.
- Load More button phải nằm trong vùng scrollable của email list, không bị ẩn ngoài viewport.

NFR5: Khả năng tương thích (Compatibility)

- Ứng dụng phải hoạt động ổn định và hiển thị tốt trên các phiên bản mới nhất của các trình duyệt web.
- Giao diện phải có thiết kế đáp ứng (responsive), nghĩa là tương thích tốt trên cả máy tính và thiết bị di động.

2.2. Thiết kế hệ thống

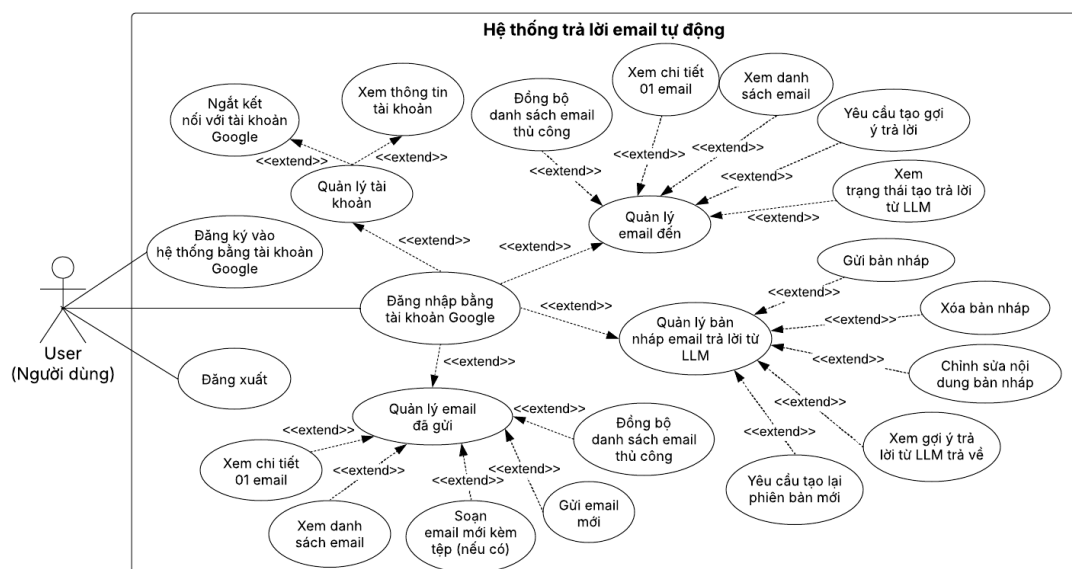
2.2.1. Kiến trúc tổng thể



Hình 2.1. Sơ đồ kiến trúc tổng thể

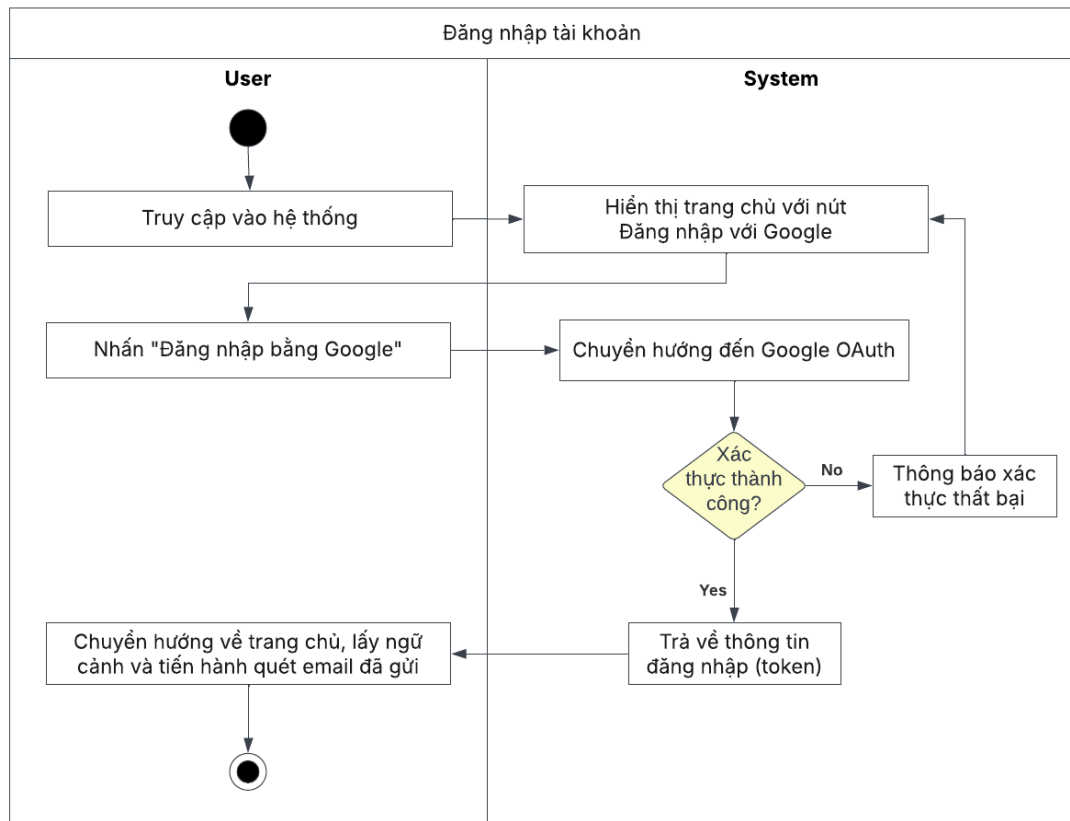
Hệ thống được thiết kế theo mô hình layered architecture: Service Layer đóng vai trò trung tâm điều phối nghiệp vụ và AI Agent, trong khi Data Access Layer và Service Clients được tách riêng nhằm giảm phụ thuộc vào hạ tầng và mô hình LLM. Mô hình LLaMA-3.3-70B được sử dụng thông qua Groq API và được quản lý tập trung qua lớp giao diện LLM Client.

2.2.2. Biểu đồ use case của hệ thống

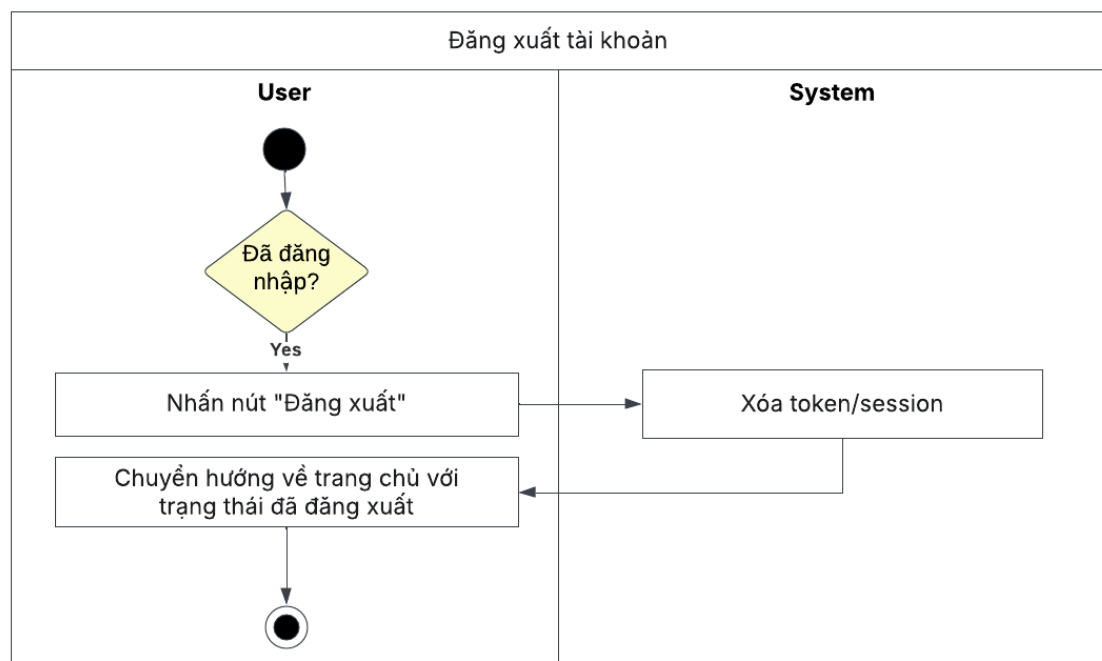


Hình 2.2. Sơ đồ use case toàn hệ thống

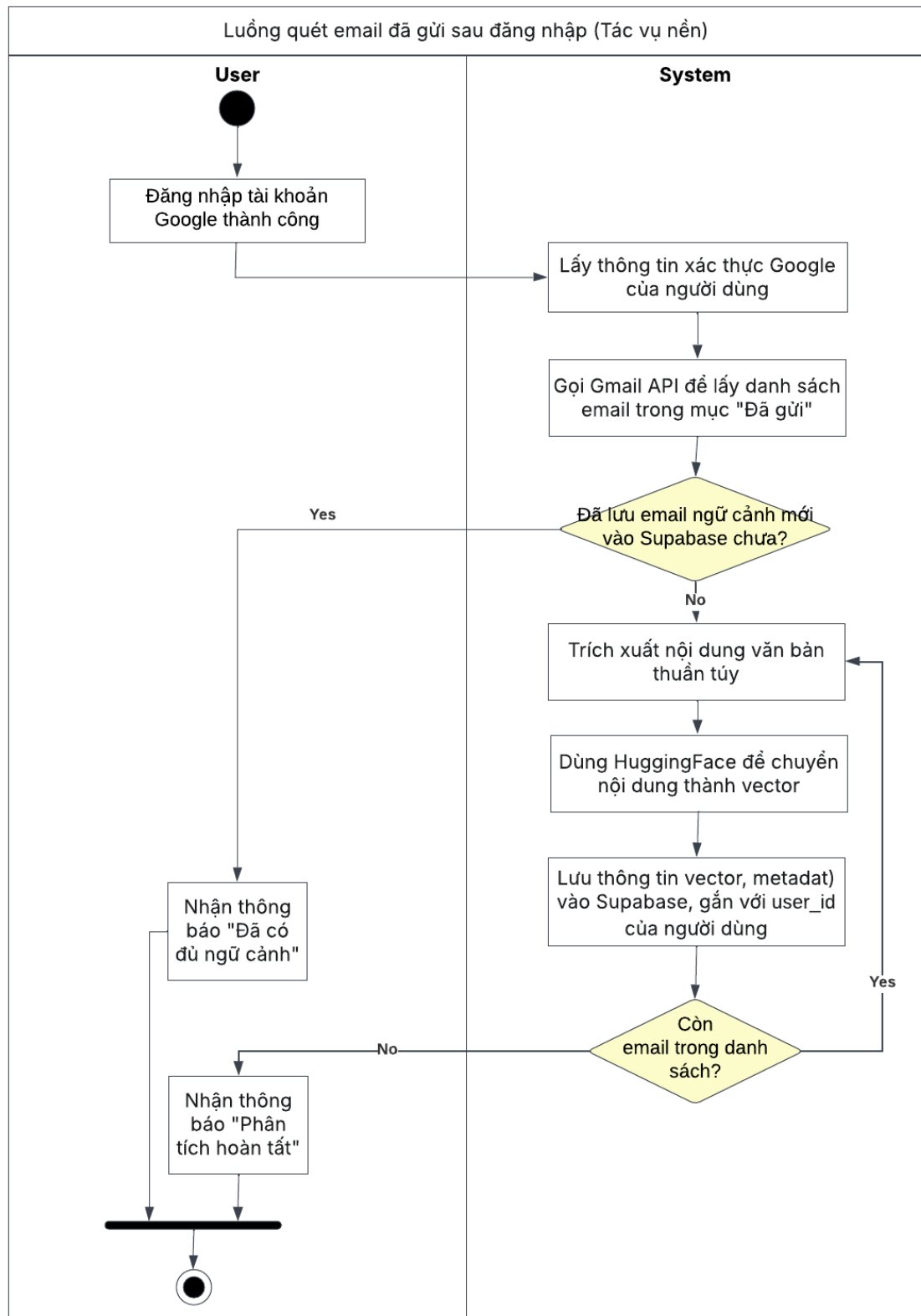
2.2.3. Biểu đồ hoạt động (Activity diagram)



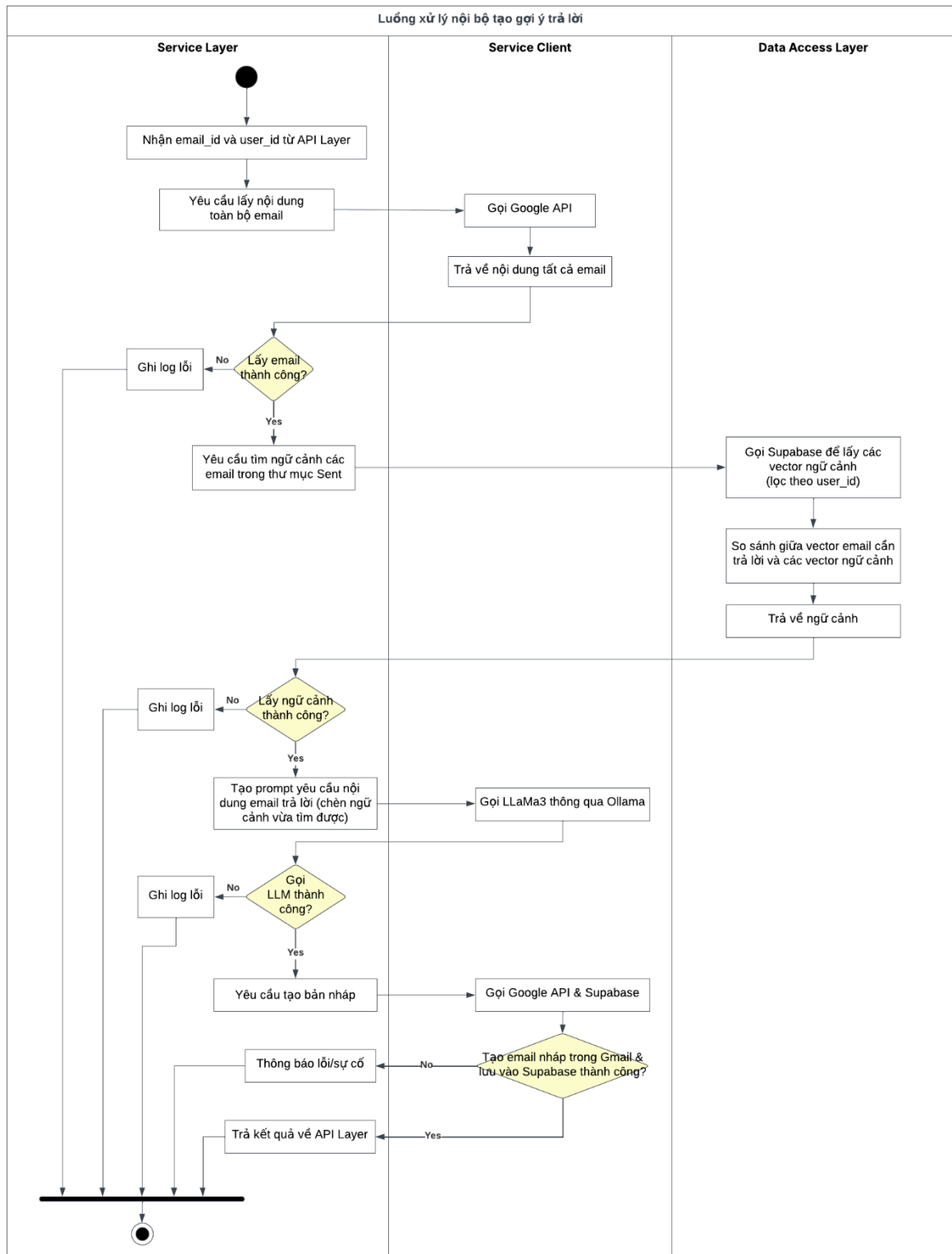
Hình 2.3. Biểu đồ hoạt động Đăng nhập tài khoản



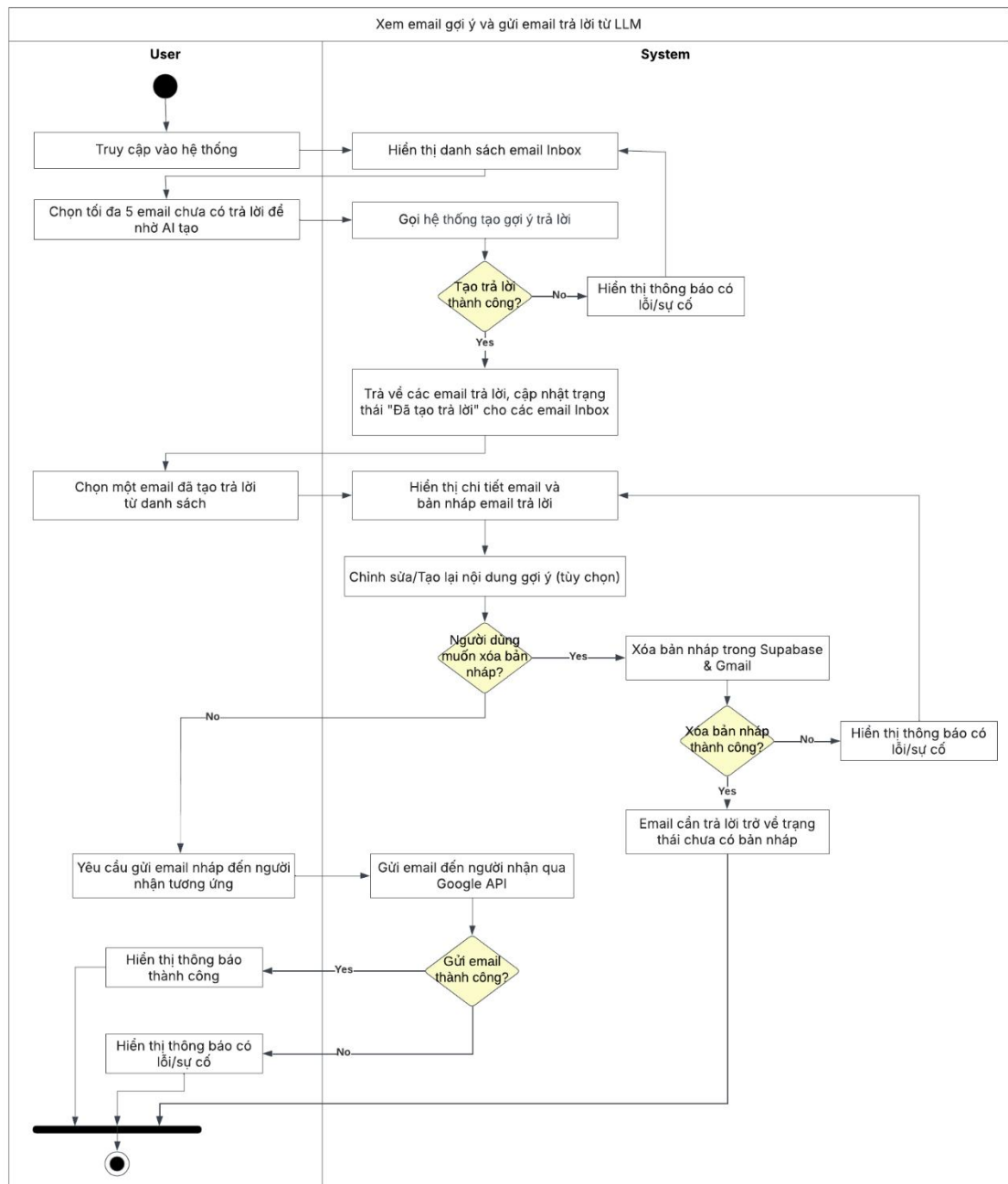
Hình 2.4. Biểu đồ hoạt động Đăng xuất tài khoản



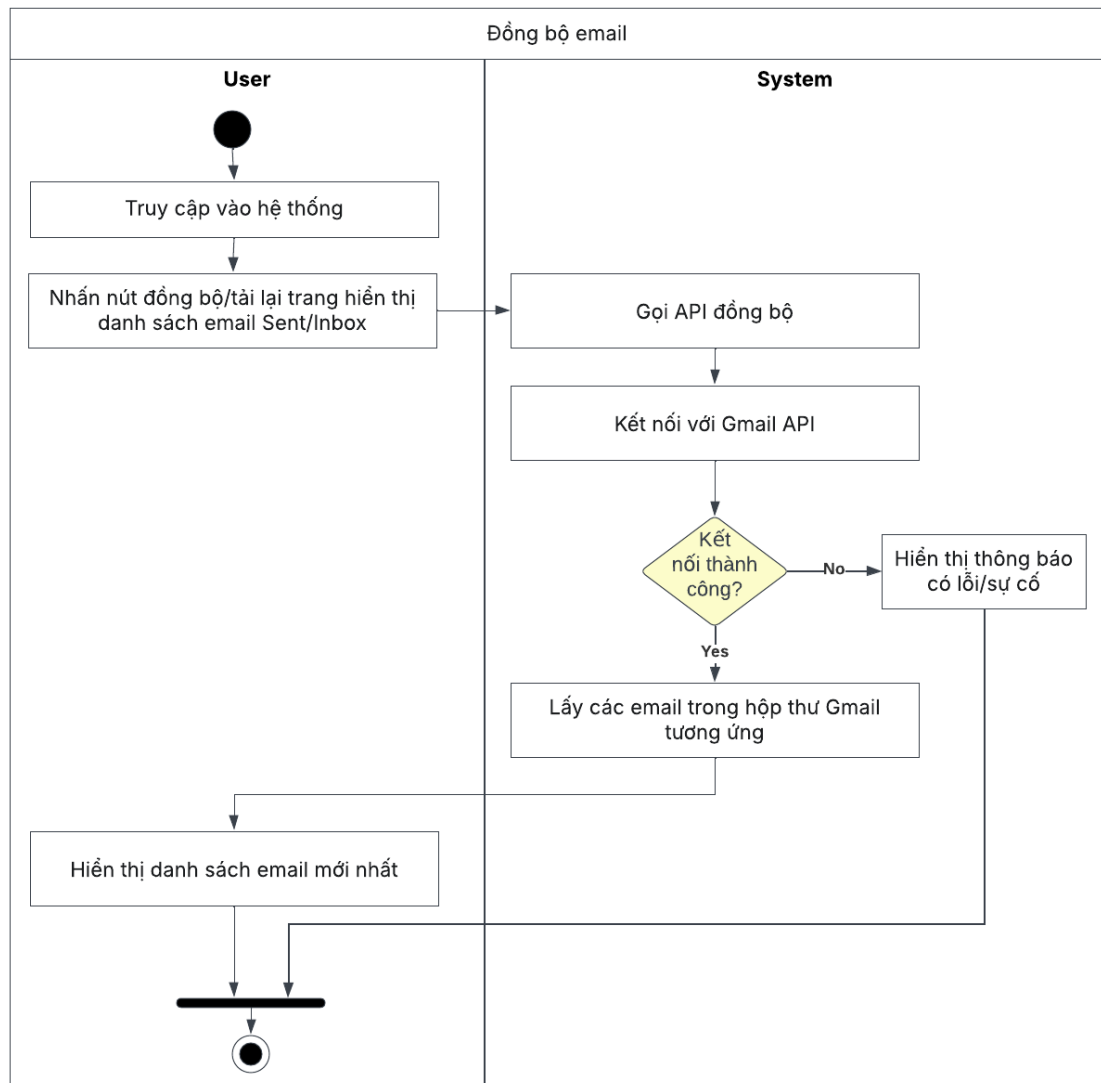
Hình 2.5. Biểu đồ hoạt động Quét email đã gửi

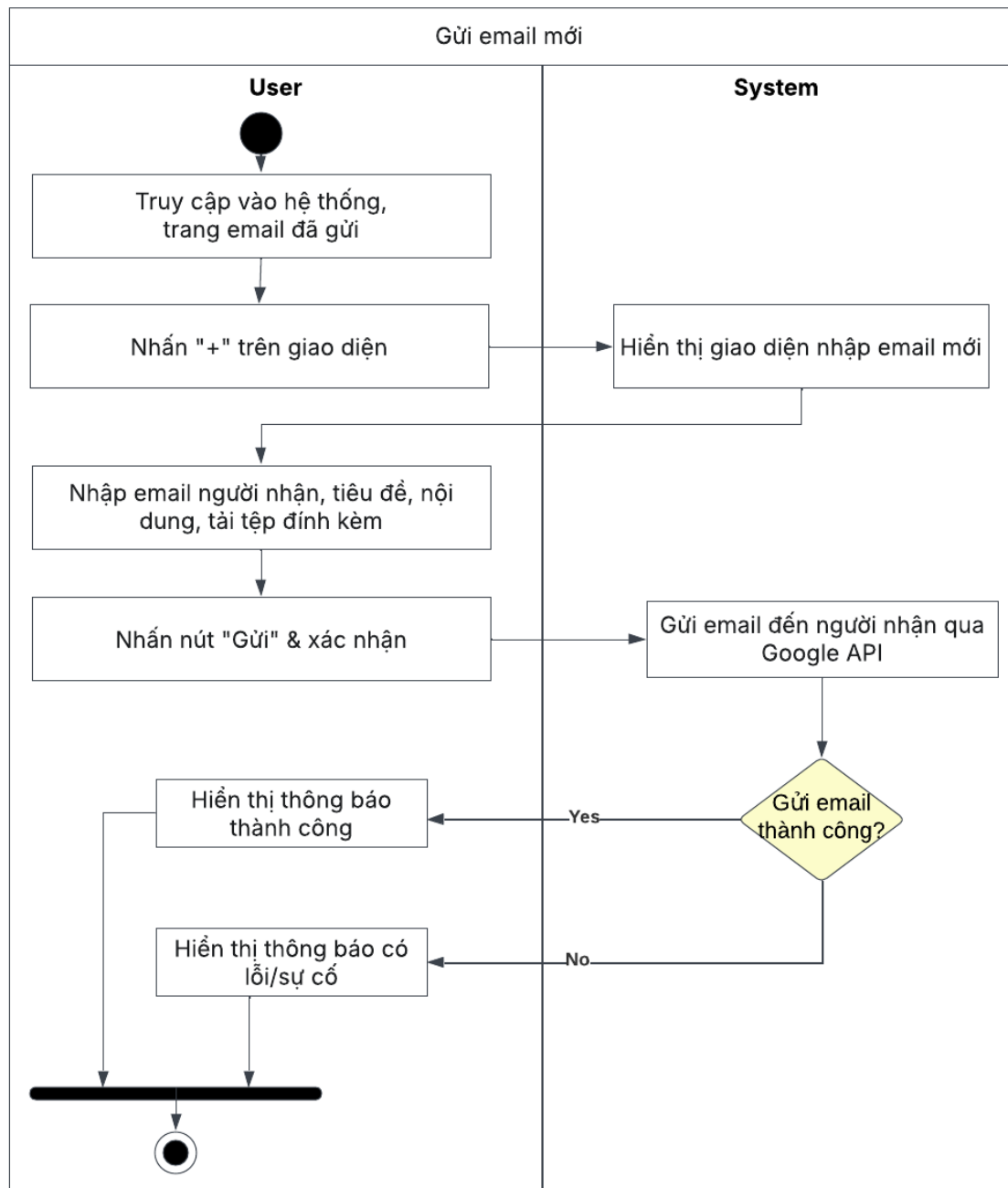


Hình 2.6. Biểu đồ hoạt động Tạo gợi ý trả lời

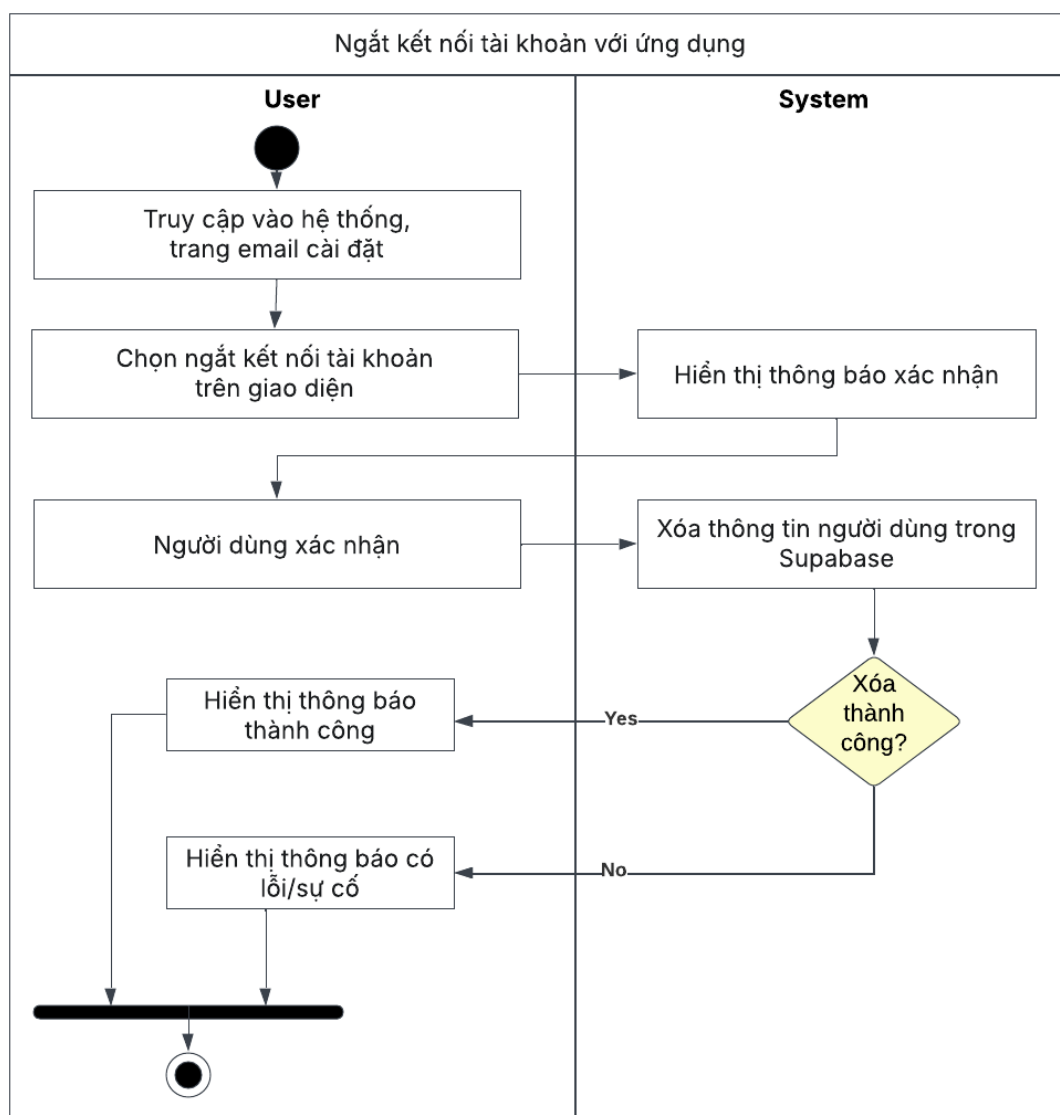


Hình 2.7. Biểu đồ hoạt động Xem và gửi email trả lời



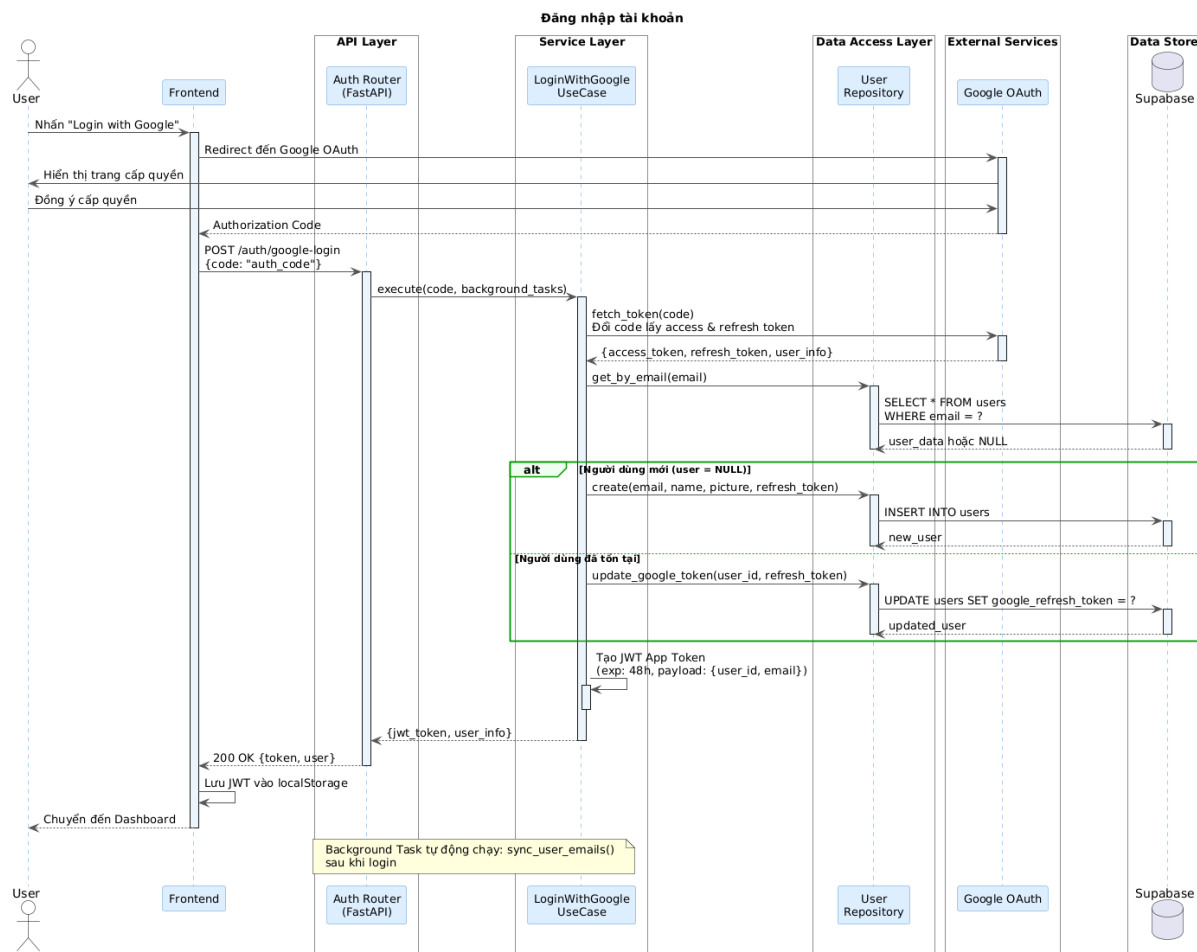


Hình 2.9. Biểu đồ hoạt động Gửi email mới

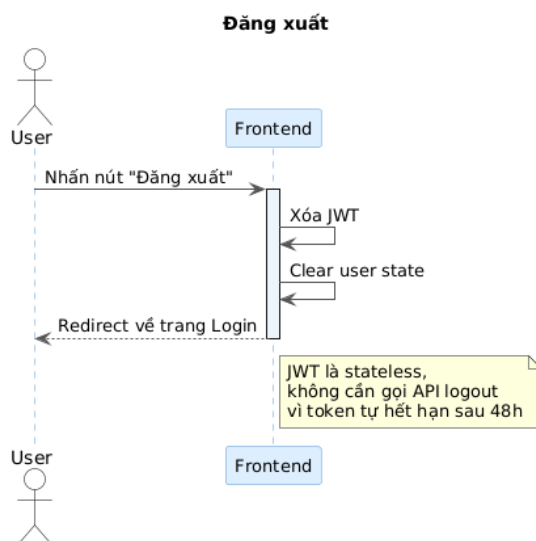


Hình 2.10. Biểu đồ hoạt động Ngắt kết nối tài khoản

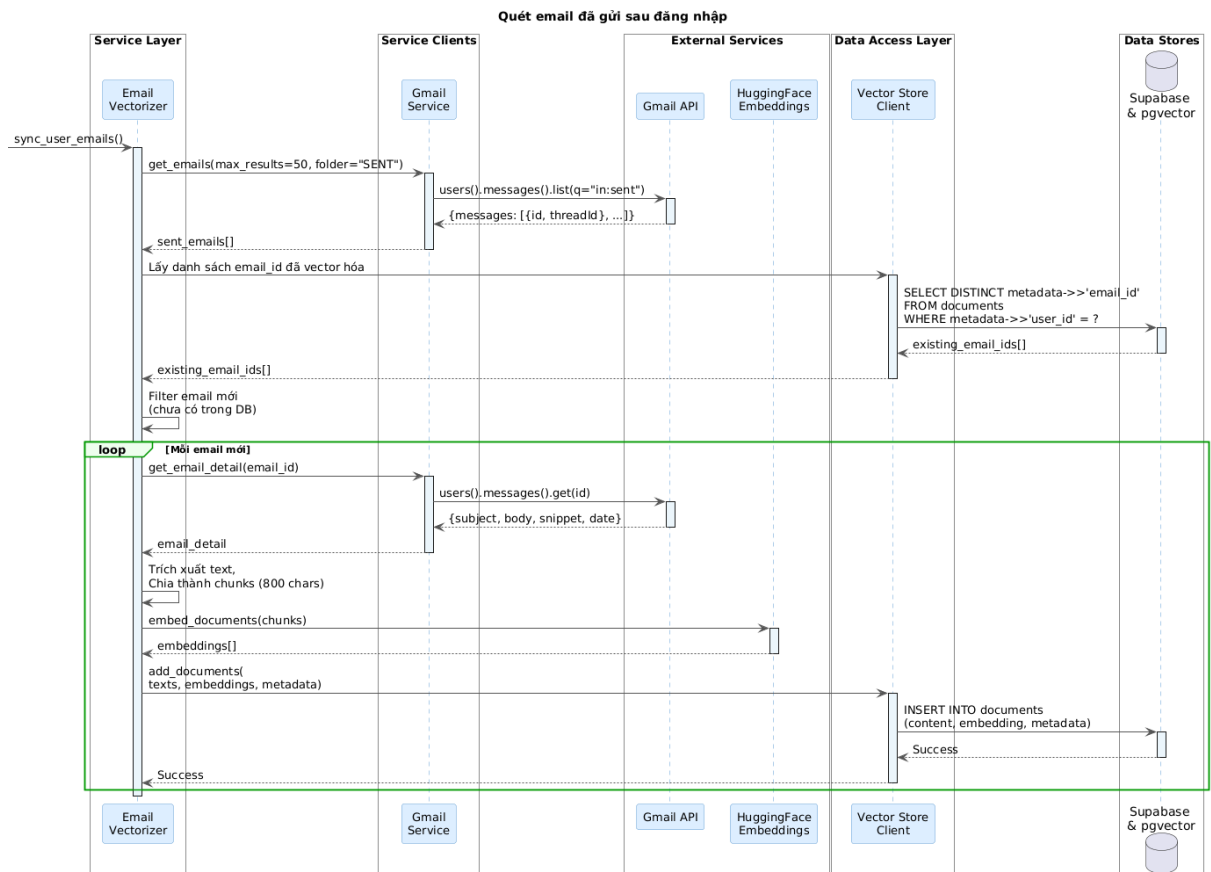
2.2.4. Biểu đồ tuần tự (Sequence diagram)



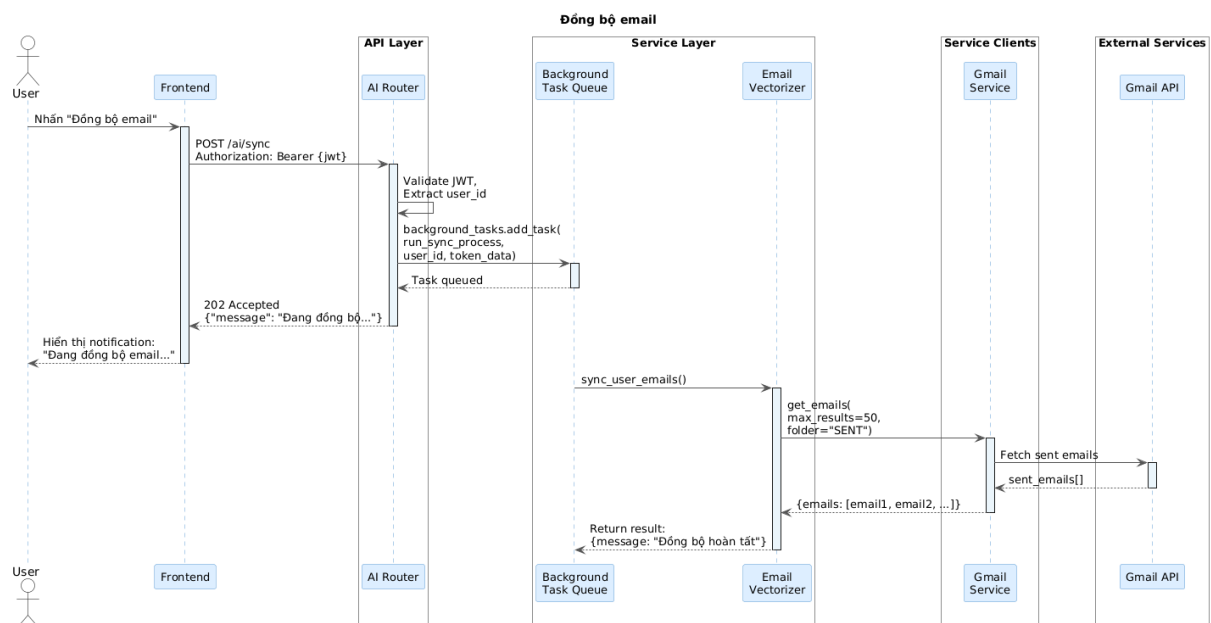
Hình 2.11. Biểu đồ tuần tự Đăng nhập tài khoản



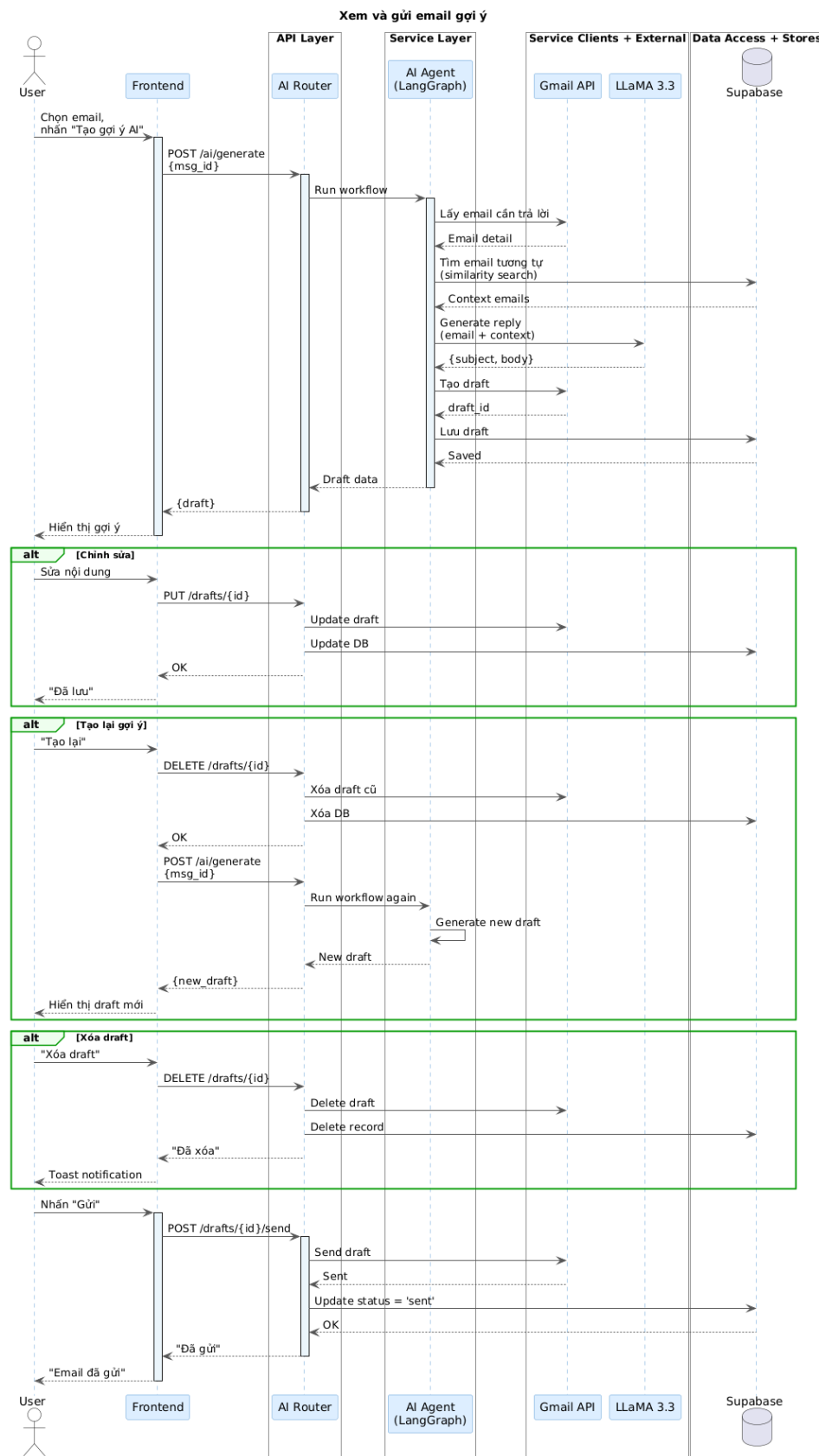
Hình 2.12. Biểu đồ tuần tự Đăng xuất tài khoản



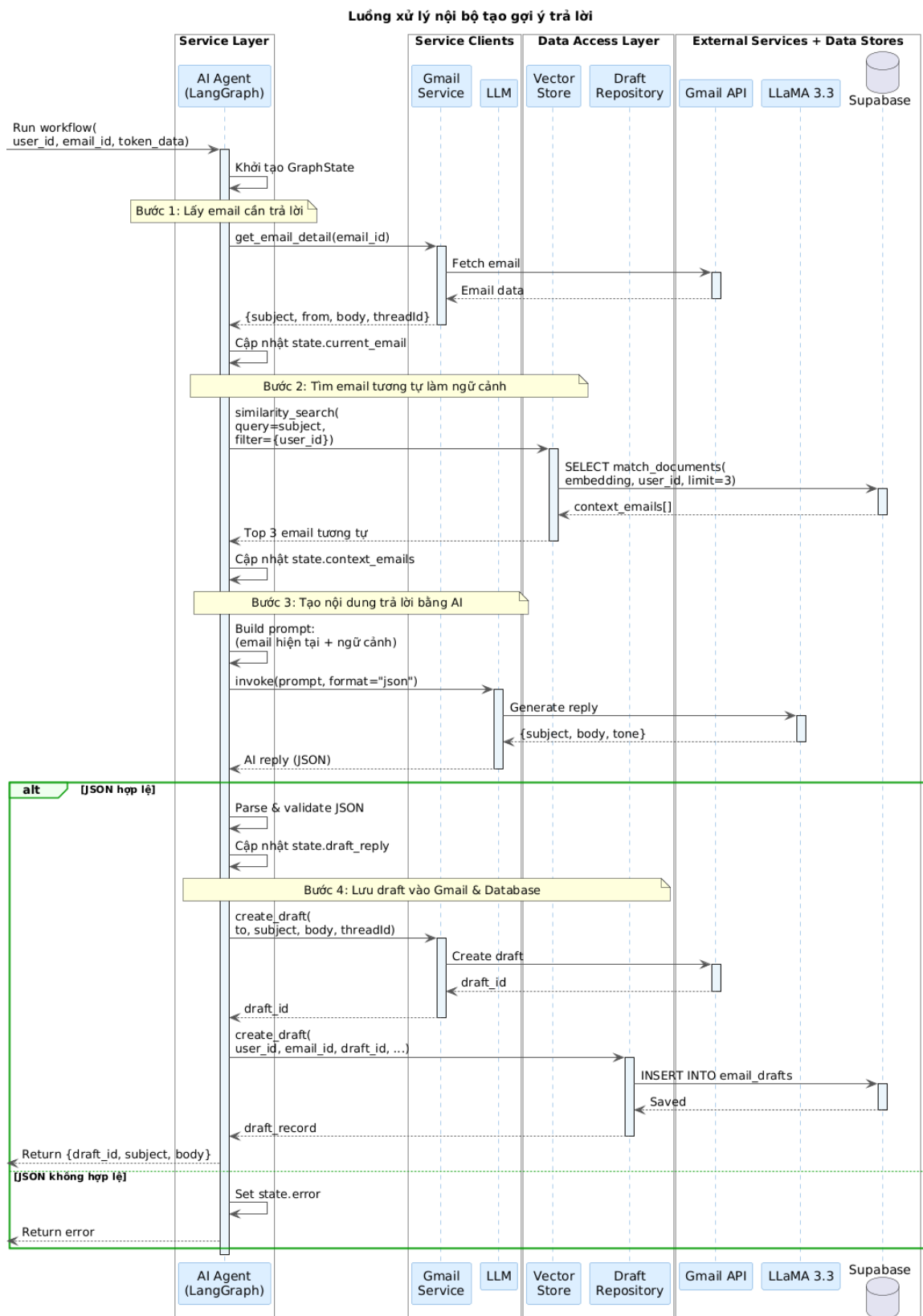
Hình 2.13. Biểu đồ tuần tự Quét email đã gửi



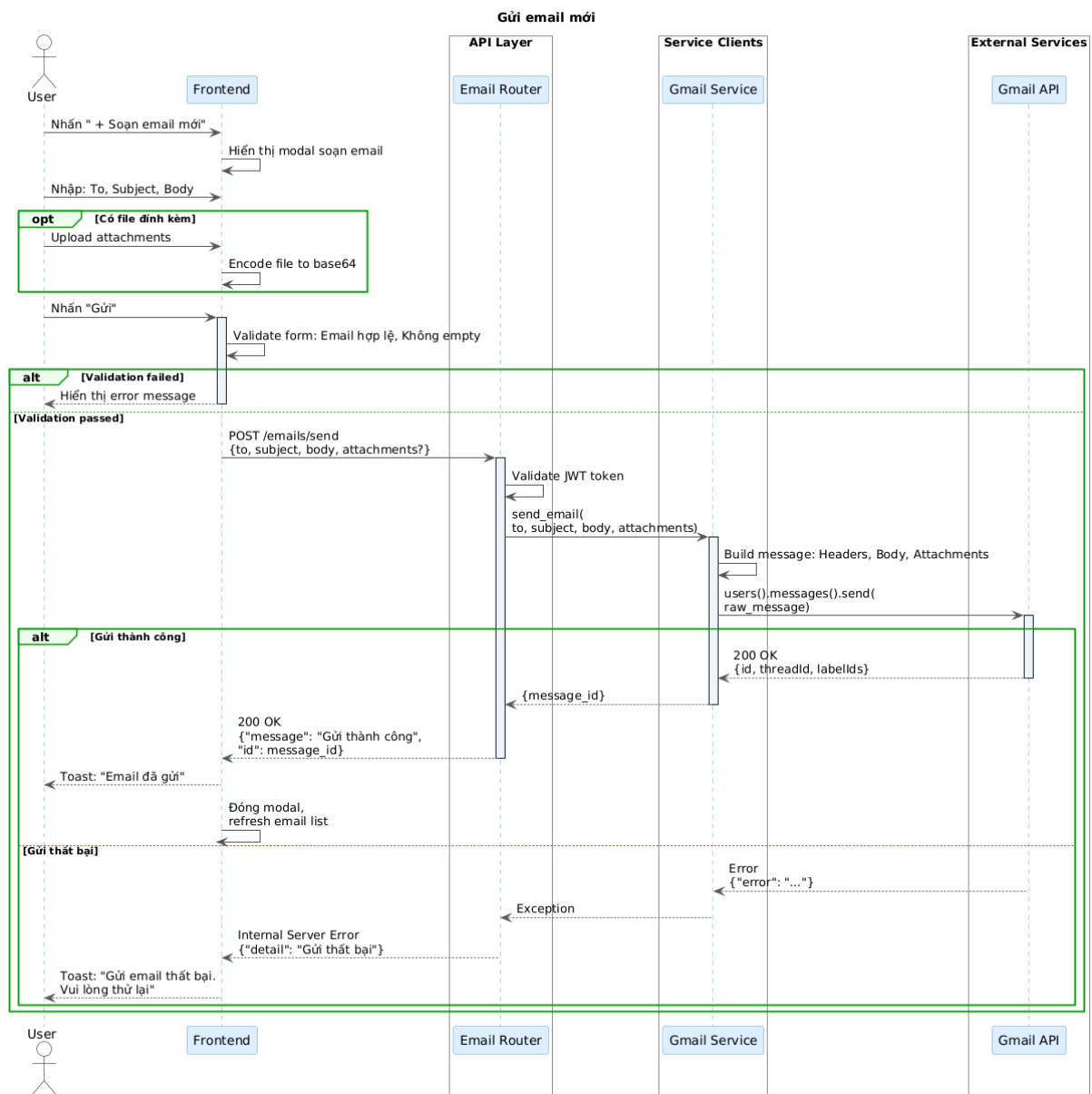
Hình 2.14. Biểu đồ tuần tự Đồng bộ email



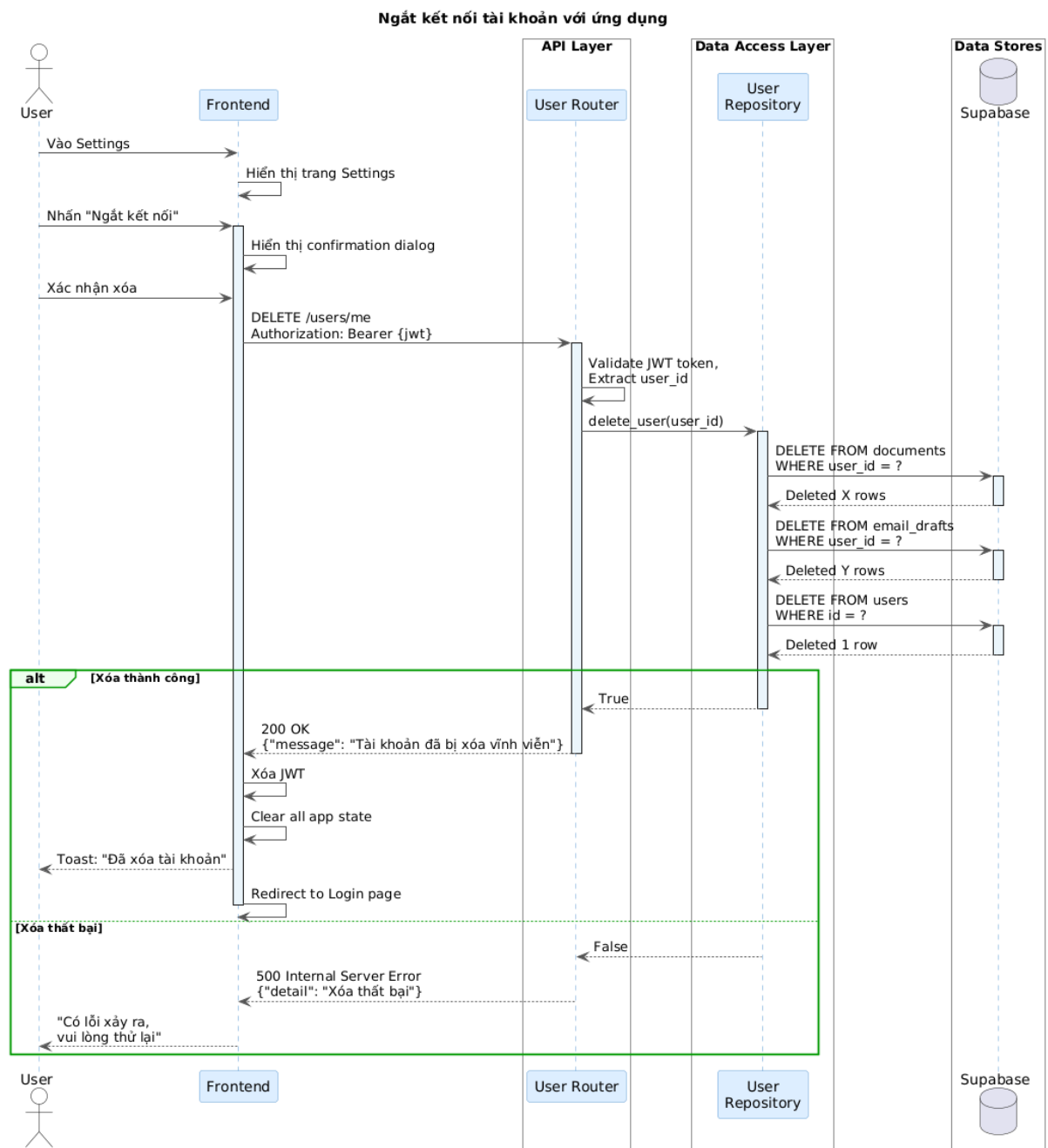
Hình 2.15. Biểu đồ tuần tự Xem và gửi email trả lời



Hình 2.16. Biểu đồ tuần tự Tạo gợi ý trả lời



Hình 2.17. Biểu đồ tuần tự Gửi email mới



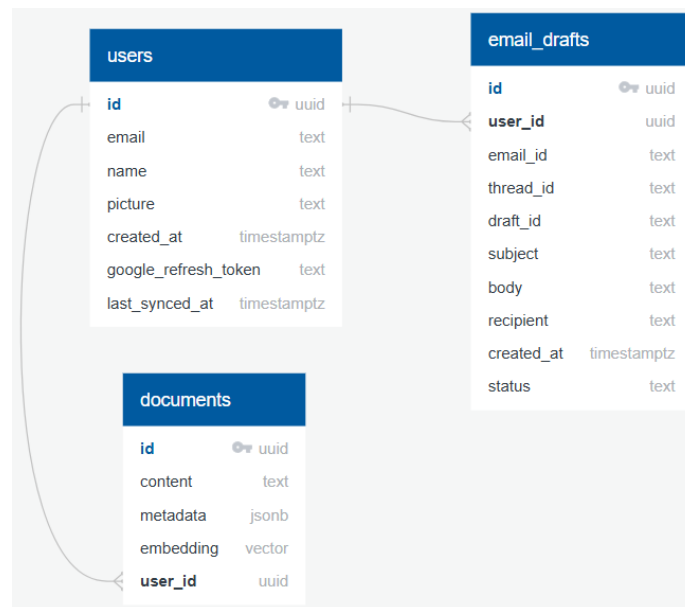
Hình 2.18. Biểu đồ hoạt động Ngắt kết nối tài khoản

2.2.5. Thiết kế cơ sở dữ liệu

Thiết kế cơ sở dữ liệu là quá trình xác định cấu trúc lưu trữ dữ liệu một cách hiệu quả và có tổ chức. Cơ sở dữ liệu Postgres sẽ bao gồm 3 thực thể chính: Users (Người dùng), Documents (Dữ liệu huấn luyện/Vector store), và Email Drafts (Bản nháp email).

Hệ thống sử dụng Supabase - nền tảng Backend-as-a-Service dựa trên PostgreSQL - làm hệ quản trị cơ sở dữ liệu chính. Việc lựa chọn PostgreSQL cho phép tận dụng tiện ích mở rộng pgvector để lưu trữ và truy vấn vector, phục vụ trực tiếp cho tính năng RAG (Retrieval-Augmented Generation) của AI.

Dưới đây đặc tả chi tiết cấu trúc của từng bảng trong cơ sở dữ liệu:



Hình 2.19. Cơ sở dữ liệu

Bảng 2.1. Bảng users (Người dùng)

STT	Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
1	id	uuid	PRIMARY KEY	Mã định danh duy nhất của người dùng.
2	email	text	NOT NULL	Địa chỉ email của người dùng.
3	name	text	NULL	Tên hiển thị người dùng.
4	picture	text	NULL	Đường dẫn ảnh đại diện (Avatar).
5	created_at	timestampz	NULL, DEFAULT now()	Thời gian tạo tài khoản.
6	google_refresh_token	text	NULL	Token làm mới để duy trì phiên làm việc với Google API.
7	last_synced_at	timestampz	NULL	Thời điểm cuối cùng hệ thống đồng bộ dữ liệu.

Bảng 2.2. Bảng documents (Kho dữ liệu Vector)

STT	Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
1	id	uuid	PRIMARY KEY	Khóa chính của tài liệu.
2	content	text	NULL	Nội dung văn bản gốc của email (Subject + Body).
3	metadata	jsonb	NULL	Dữ liệu cấu trúc dạng JSON chứa thông tin phụ (ngày gửi, tiêu đề...).

4	embedding	vector	NULL	Vector đặc trưng số hóa nội dung email (384 chiều).
5	user_id	uuid	FRIMARY KEY (users.id)	Khóa ngoại tham chiếu đến bảng users, xác định chủ sở hữu dữ liệu.

Bảng 2.3. Bảng email_drafts (Bản nháp Email)

STT	Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
1	id	uuid	PRIMARY KEY	Định danh bản nháp.
2	user_id	uuid	FRIMARY KEY (users.id)	Khóa ngoại tham chiếu đến bảng users.
3	email_id	text	NOT NULL	ID của email gốc cần trả lời.
4	thread_id	text	NOT NULL	ID luồng hội thoại (Thread ID) của Gmail.
5	draft_id	text	NOT NULL	ID bản nháp trên hệ thống Gmail API.
6	subject	text	NOT NULL	Tiêu đề email nháp.
7	body	text	NOT NULL	Nội dung email nháp.
8	recipient	text	NOT NULL	Người nhận email nháp khi gửi bản nháp đi.
9	status	text	NOT NULL	Trạng thái mail
10	created_at	timestampz	NULL, DEFAULT now()	Thời gian tạo bản nháp.
11	status	text	NULL	Trạng thái gửi bản nháp

CHƯƠNG 3. CÀI ĐẶT, TRIỂN KHAI & KIỂM THỬ

3.1. Môi trường phát triển

3.1.1. Phần cứng

Để đảm bảo quá trình phát triển diễn ra thuận lợi, hệ thống phần cứng được trang bị tối ưu cho hiệu năng, đặc biệt khi xử lý AI và cơ sở dữ liệu vector. Cụ thể, máy tính phát triển sử dụng bộ vi xử lý Intel Core i5 hoặc tương đương, cung cấp hiệu suất xử lý ổn định cho các tác vụ lập trình, biên dịch và chạy đa nhiệm. Bộ nhớ RAM yêu cầu tối thiểu 8-16 GB để vận hành mượt mà. Ổ cứng SSD tối thiểu 512 GB để tăng tốc độ khởi động ứng dụng Next.js, lưu trữ các bản ghi cơ sở dữ liệu và tối ưu hóa thời gian truy xuất vector embeddings từ pgvector.

3.1.2. Phần mềm

Yếu tố phần mềm đóng vai trò then chốt trong quá trình xây dựng và triển khai hệ thống, giúp đội ngũ phát triển hiện thực hóa các yêu cầu của dự án thành một sản phẩm cụ thể. Vì thế, việc lựa chọn công nghệ, ngôn ngữ lập trình và công cụ hỗ trợ được cân nhắc kỹ lưỡng nhằm đảm bảo tính hiệu quả, khả năng mở rộng và dễ dàng bảo trì sau này. Trước hết, hệ điều hành được sử dụng là Windows 11 hoặc macOS, đây là những môi trường phổ biến và hỗ trợ đa dạng công cụ lập trình cần thiết.

Về backend, dự án sử dụng Python phiên bản 3.11 với framework FastAPI 0.104. Đây là một sự kết hợp mạnh mẽ, cho phép xây dựng các API hiệu suất cao với khả năng xử lý bất đồng bộ, đồng thời tận dụng được hệ sinh thái AI phong phú của Python. Uvicorn 0.24 được sử dụng làm máy chủ ASGI để chạy ứng dụng FastAPI với hiệu năng cao. Về AI, hệ thống sử dụng thư viện chính thức groq để kết nối và truyền nhận dữ liệu với mô hình Llama 3.3 70B Versatile thông qua đăng ký tài khoản Groq, lấy API Key, sau đó cài đặt thư viện client. LangChain 0.3.x được sử dụng làm framework tích hợp LLM, trong khi LangGraph 0.2.x điều phối luồng logic của Tác tử AI. Để tạo vector embeddings cho email, dự án sử dụng mô hình sentence-transformers/all-MiniLM-L6-v2 từ HuggingFace, tạo ra vector 384 chiều cho mỗi đoạn văn bản.

Ở phía frontend, hệ thống kết hợp React 19.2 thông qua framework Next.js 16.0.3 và TypeScript 5. Next.js cung cấp một cấu trúc frontend vững chắc, hỗ trợ các tính năng tối ưu hóa hiệu năng như Server-Side Rendering và API Routes, trong khi TypeScript đảm bảo tính an toàn và chặt chẽ cho mã nguồn thông qua hệ thống kiểm tra kiểu tĩnh. TailwindCSS 4 được sử dụng để xây dựng giao diện người dùng. Cơ sở dữ liệu được lựa chọn là PostgreSQL phiên bản 14 trở lên thông qua Supabase, tích hợp extension pgvector 0.5.1 để lưu trữ và tìm kiếm vector embeddings. Để tích hợp với Gmail, dự án sử dụng Gmail API v1 kết hợp OAuth 2.0 cho xác thực người dùng. JSON Web Tokens được sử dụng để quản lý phiên làm việc giữa frontend và backend.

Trong quá trình phát triển, nhóm sử dụng công cụ IDE Visual Studio Code với hệ sinh thái plugin phong phú, bao gồm Python extension, Pylance (language server), TypeScript/React extensions, được dùng để phát triển cả frontend và backend. Để đảm bảo quá trình phát triển diễn ra suôn sẻ, Git và GitHub được sử dụng như công cụ quản lý mã nguồn, cho phép kiểm soát phiên bản và cộng tác hiệu quả trên nhánh frontend của repository.

3.1.3. Kết nối Internet

Yêu cầu kết nối Internet ổn định và băng thông tốt. Do mô hình nằm trên Cloud của Groq, tốc độ phản hồi của hệ thống phụ thuộc vào độ trễ mạng từ máy trạm đến máy chủ Groq.

3.2. Cài đặt hệ thống

3.2.1. Cấu hình backend

Hệ thống Backend được phát triển dựa trên nền tảng Python 3.11 kết hợp với framework FastAPI, đóng vai trò trung tâm trong việc xử lý các nghiệp vụ cốt lõi, điều phối luồng hoạt động của AI và cung cấp giao thức REST API. Về cơ sở dữ liệu, hệ thống sử dụng PostgreSQL triển khai qua Supabase làm nơi lưu trữ chính. Để đảm bảo tính bảo mật & nhất quán, các thông tin kết nối quan trọng như SUPABASE_URL và SUPABASE_SERVICE_KEY được cấu hình trong .env và được khởi tạo kết nối ở supabase_client.py thông qua thư viện supabase-py.

Kiến trúc API được tổ chức bài bản theo dạng module bên trong thư mục api, giúp code dễ dàng quản lý và mở rộng. Cụ thể, các router được phân chia với các nhiệm vụ cụ thể: `auth_router.py` xử lý xác thực OAuth 2.0 với Google; `email_router.py` chịu trách nhiệm đồng bộ và quản lý email; `ai_router.py` đảm nhận việc tạo phản hồi hoặc gợi ý từ AI; và `user_router.py` quản lý thông tin người dùng. Tại `main.py`, FastAPI Router sẽ tích hợp toàn bộ các module này, đồng thời tự động tạo Swagger UI để hỗ trợ quá trình phát triển & kiểm thử.

Điểm đáng chú ý của hệ thống nằm ở việc tích hợp AI/ML. Backend sử dụng thư viện `groq` để kết nối và truyền nhận dữ liệu với mô hình Llama 3.3 70B Versatile. Quy trình xử lý logic (reasoning) được định nghĩa bằng LangGraph tại `reasoning.py` dưới dạng một máy trạng thái (state machine) gồm các bước tuần tự: `fetch_email_node` (lấy email từ Gmail API) → `retrieve_context_node` (tìm kiếm ngữ cảnh trên vector database) → `generate_reply_node` (tạo email trả lời) → `create_draft_node` (tạo bản nháp trong Gmail và lưu vào DB). Bên cạnh đó, hệ thống sử dụng mô hình `sentence-transformers/all-MiniLM-L6-v2` ở `vectorizer.py` để vector hóa các email đã gửi, phục vụ cho kỹ thuật RAG.

Về cơ chế xác thực và phân quyền, hệ thống áp dụng chuẩn OAuth 2.0 thông qua Google. Sau khi người dùng đăng nhập thành công, backend sẽ sinh ra mã JWT để quản lý phiên làm việc, bắt buộc phải được gửi kèm trong header `Authorization` cho mọi yêu cầu gửi lên server. Tại backend, dependency `get_current_user()` trong `deps.py` sẽ xác minh JWT và trích xuất `user_id`. Ngoài ra, Google Access Token và Refresh Token cũng được lưu trữ an toàn trong bảng `users` để phục vụ các thao tác gọi Gmail API về sau.

Cuối cùng, để đảm bảo an toàn hệ thống, toàn bộ thông tin nhạy cảm (như các khóa Supabase, JWT Secret, Google Client ID/Secret) đều được lưu trong `.env`. Cơ chế CORS được thiết lập chi tiết tại `main.py`, cho phép frontend truy cập tài nguyên hợp lệ. Đồng thời, chính sách Row Level Security (RLS) của Supabase cũng được kích hoạt để đảm bảo nguyên tắc người dùng chỉ có thể truy cập và thao tác trên dữ liệu của chính họ.

Nhờ cấu hình này, backend đáp ứng được các yêu cầu về tính ổn định, bảo mật, nhất quán cũng như khả năng mở rộng, đồng thời tạo nền tảng vững chắc cho frontend và toàn bộ hệ thống.

3.2.2. Cấu hình frontend

Phần Frontend của hệ thống được xây dựng dựa trên nền tảng Next.js phiên bản 16.0.3 kết hợp với thư viện React 19.2.0, đóng vai trò cung cấp giao diện người dùng tương tác để quản lý email, xem các gợi ý từ AI và chỉnh sửa bản thảo. Về nền tảng kỹ thuật, hệ thống sử dụng App Router để hỗ trợ linh hoạt giữa Server-Side Rendering và Client-Side Rendering, đồng thời sử dụng các React hooks như useState, useEffect và useRouter để xử lý logic. Về mặt giao diện, dự án tích hợp Tailwind CSS 4 theo hướng tiếp cận utility-first, với các thiết lập theme và style cho nội dung email được định nghĩa toàn cục tại globals.css.

Lớp giao tiếp với Backend (API Client Layer) được đặt tại api.ts, chịu trách nhiệm định nghĩa các hàm tương tác chính. Hệ thống kết nối tới Backend thông qua biến môi trường NEXT_PUBLIC_API_URL, trong đó mỗi request gửi đi đều đính kèm JWT token trong header Authorization. Đối với việc quản lý trạng thái và bảo mật, JWT token cùng thông tin người dùng được lưu trữ trong sessionStorage và truy xuất qua các hàm như getAuthToken(), setAuthToken() và getUserInfo(). Component AuthGuard tại đảm nhiệm vai trò bảo vệ các routes, tự động điều hướng người dùng chưa đăng nhập về trang login. Các trạng thái giao diện cục bộ (như danh sách email, bản thảo, trạng thái loading) được quản lý trực tiếp bằng React hooks.

Cuối cùng, quy trình xác thực được thiết kế bắt đầu từ trang landing, nơi người dùng được chuyển hướng đến Google OAuth với các quyền truy cập gồm readonly, send và modify. Sau khi người dùng xác thực, mã authorization code sẽ được nhận tại route /auth/callback và được chuyển đổi thành JWT token thông qua Backend để hoàn tất phiên đăng nhập.

3.2.3. Cài đặt cơ sở dữ liệu

Trong dự án này, hệ quản trị cơ sở dữ liệu PostgreSQL phiên bản 14 trở lên thông qua dịch vụ Supabase được lựa chọn bởi những ưu điểm nổi bật như khả năng xử lý dữ liệu quan hệ phức tạp, hỗ trợ tốt các chuẩn SQL, cùng với độ ổn định cao trong môi trường triển khai thực tế. Supabase cung cấp các tính năng mở rộng như Row Level Security (RLS), RESTful API tự động, phù hợp với nhu cầu bảo mật của ứng dụng email. Điểm nổi bật là việc tích hợp pgvector 0.5.1 extension cho phép lưu trữ và tìm kiếm vector embeddings trực tiếp trong PostgreSQL, giúp đơn giản hóa kiến trúc vì không cần duy trì vector database riêng biệt. Pgvector hỗ trợ toán tử cosine distance để tính độ tương đồng giữa các vector embeddings 384 chiều từ mô hình sentence-transformers/all-MiniLM-L6-v2.

Để kết nối giữa tầng ứng dụng và cơ sở dữ liệu, Supabase Python Client được tích hợp như một lớp trung gian tại `supabase_client.py`, cung cấp interface theo phong cách builder pattern để thực hiện các thao tác CRUD một cách trực quan. Nó không chỉ đơn giản hóa việc tương tác với PostgreSQL mà còn tự động áp dụng Row Level Security policies, đảm bảo tính độc lập về việc truy cập được dữ liệu của mỗi user. Điều này giúp giảm thiểu rủi ro bảo mật mà không cần viết logic phân quyền phức tạp trong code.

Cấu trúc dữ liệu của hệ thống được thiết kế gồm ba bảng chính: Bảng `users` lưu trữ thông tin người dùng và `google_refresh_token` để duy trì kết nối Gmail API, bảng `documents` đóng vai trò vector store cho RAG, và bảng `email_drafts` quản lý các draft reply do AI tạo ra. Mỗi bảng đều có ràng buộc khóa chính, khóa ngoại và NOT NULL nhằm bảo đảm tính toàn vẹn dữ liệu. Bên cạnh đó, hệ thống triển khai PostgreSQL Function `match_documents()` để thực hiện RAG retrieval, nhận vào query embedding và filter theo `user_id`, sau đó sử dụng cosine distance để tìm documents tương đồng nhất.

Ngoài ra, cơ sở dữ liệu còn hỗ trợ migrations thông qua Supabase CLI, cho phép quản lý các thay đổi schema dưới dạng SQL migration files có version. Tính năng này giúp nhóm phát triển dễ dàng triển khai và đồng bộ cấu trúc database giữa

các môi trường mà không gây mất mát dữ liệu. Supabase cũng tự động thực hiện daily backup với khả năng Point-in-Time Recovery, đảm bảo dữ liệu có thể khôi phục an toàn khi cần thiết.

Nhờ kiến trúc này, hệ thống đạt được sự cân bằng giữa tính đơn giản, hiệu suất cao và bảo mật chặt chẽ, đồng thời dễ dàng mở rộng khi nhu cầu phát triển thêm trong tương lai.

3.3. Triển khai chức năng

Xây dựng hệ thống phản hồi email tự động là một bài toán tích hợp toàn diện, đòi hỏi sự kiến trúc hóa tỉ mỉ để đồng bộ hóa các luồng dữ liệu giữa giao diện, trí tuệ nhân tạo (AI Reasoning), cơ sở dữ liệu và các dịch vụ bên thứ ba như Gmail API. Mỗi tính năng được phát triển đều phải giải quyết bài toán cân bằng giữa hiệu năng xử lý, an toàn thông tin và tính khả dụng, nhằm mang lại một giải pháp công nghệ tối ưu.

3.3.1. Đăng nhập với Google

Đây là chức năng nền tảng, bảo đảm xác thực và ủy quyền cho hệ thống truy cập Gmail API. Người dùng đăng nhập qua Google OAuth 2.0, cấp quyền đọc, gửi và chỉnh sửa email. Chức năng xác thực được triển khai theo các bước sau:

OAuth Flow: Khi người dùng nhấn nút đăng nhập, Frontend redirect user đến Google OAuth URL. Sau khi đồng ý các quyền truy cập, Google redirect về /auth/callback kèm authorization code.

Token Exchange: Frontend gửi code lên backend qua API POST /auth/google-login. Backend sử dụng google_auth_oauthlib để đổi code lấy Access Token và Refresh Token từ Google.

Lưu trữ: Backend lấy thông tin user từ Google UserInfo API, upsert (insert & update) vào bảng users kèm Refresh Token. Hệ thống tạo JWT chứa user_id và trả về frontend.

Phân quyền: Client lưu JWT vào sessionStorage và gửi kèm header Authorization: Bearer <token> cho mọi request. Middleware backend verify JWT để đảm bảo chỉ user đã đăng nhập mới truy cập tài nguyên.

Background Sync: Sau login, backend tự động trigger background task đồng bộ 50 email đã gửi vào vector database phục vụ RAG.

3.3.2. Đồng bộ và quản lý email

Chức năng này đóng vai trò là "khung xương sống" của hệ thống, cho phép người dùng xem danh sách email inbox, chi tiết từng email và đồng bộ dữ liệu mới từ Gmail. Các thao tác bao gồm lấy danh sách, xem chi tiết, phân trang và lọc theo folder (INBOX, SENT). Chức năng này được triển khai thông qua REST API và React components, cụ thể như sau:

Backend: Endpoint GET /emails verify JWT, khởi tạo GmailService với Google token, gọi Gmail API users().messages().list() để lấy danh sách email IDs và fetch chi tiết từng email. Dữ liệu được transform thành format chuẩn và trả về frontend kèm next_page_token để phân trang.

Frontend: Component workspace gọi fetchEmails() khi mount, hiển thị danh sách qua EmailList component, cũng như hệ thống hỗ trợ load more khi scroll và đồng bộ email khi user click biểu tượng trên giao diện.

Đánh dấu Draft: Sau khi load emails, frontend gọi API lấy drafts từ Supabase, map với email để đánh dấu email nào đã có AI reply sẵn.

Dữ liệu email được liên kết chặt chẽ với drafts và sent status, đảm bảo mọi thay đổi (tạo draft, gửi email) đều được cập nhật đồng bộ qua việc re-fetch, tạo điều kiện cho việc giám sát toàn diện và nâng cao tính minh bạch.

3.3.3. Tạo AI Reply với LangGraph

Đây là chức năng cốt lõi của ứng dụng, cho phép tạo gợi ý trả lời thông minh dựa trên nội dung email và văn phong cá nhân. Reply được generate qua luồng LangGraph gồm 4 bước, tự động lưu thành Gmail draft, đồng thời hỗ trợ regenerate khi user không hài lòng. Cụ thể như sau:

Backend: Endpoint POST /ai/generate khởi tạo EmailReasoningSystem, tạo LangGraph workflow với 4 nodes với quy trình như sau:

- fetch_email_node: Lấy chi tiết email gốc từ Gmail API.

- `retrieve_context_node`: Tạo embedding cho email bằng sentence-transformers, gọi PostgreSQL function `match_documents()` tìm top 3 email đã gửi tương đồng nhất.
- `generate_reply_node`: Xây dựng prompt kèm context văn phong, nội dung email gốc và instruction, sau đó gọi Groq Llama 3.3 70B Versatile để generate reply. Prompt tự động phát hiện ngôn ngữ và chèn lại lời người gửi.
- `create_draft_node`: Tạo Gmail draft và lưu vào Supabase để theo dõi.

Frontend: User chọn email và click "Generate AI Reply". `AiSuggestionPanel` gọi `generateAiReply()` hiển thị loading, sau đó nhận response và load draft content để người dùng xem trước và chỉnh sửa/xóa/tạo lại nếu cần.

Regenerate: User có thể regenerate bằng cách xóa draft cũ khỏi Gmail và Supabase và trigger lại workflow để tạo reply mới.

3.3.4. Chỉnh sửa và gửi Draft

Người dùng có thể chỉnh sửa nội dung AI reply trước khi gửi, góp phần mang tính cá nhân hóa cao cho ứng dụng, cho phép user kiểm soát hoàn toàn nội dung trước khi gửi đi, tăng tính human-in-the-loop. Cụ thể:

Edit Draft (Chỉnh sửa bản nháp): User click vào draft content để chỉnh sửa. Sau chỉnh sửa, người dùng nhấn gửi, nếu nội dung khác với original, frontend gọi API `PUT /emails/drafts/{draft_id}` để update draft trên Gmail.

Send Reply (Gửi bản nháp): User click "Send", frontend update draft (nếu có sửa) sau đó gọi API `POST /emails/drafts/{draft_id}/send`. Backend gọi Gmail API `users().drafts().send()`, draft trở thành sent message. Frontend đánh dấu email là `replySent` và update UI.

Bảo mật được chú trọng thông qua JWT verification cho mọi API call, đảm bảo user chỉ chỉnh sửa và gửi draft của chính họ. Row Level Security trên Supabase cũng kiểm tra `user_id` khớp với authenticated user.

3.3.5. Soạn email mới

Chức năng này nằm ở trang Compose, cho phép user soạn email hoàn toàn mới thông qua form nhập To, Subject (optional), Body và upload file đính kèm (optional). Khi người dùng nhấn Gửi, frontend gọi POST /emails/send với FormData chứa email content và files (nếu có), khi đó backend sử dụng Gmail API users().messages().send() để gửi email. Bên cạnh đó, trang này cũng hiển thị danh sách email đã gửi để user theo dõi lịch sử.

3.3.6. Quản lý thông tin cá nhân

Chức năng này nằm ở trang Settings, hiển thị thông tin user profile (name, email, picture) qua API GET /users/me. Hơn nữa, user có thể xem lịch sử sync email, ngắt kết nối cũng như xóa toàn bộ dữ liệu tài khoản (documents, drafts, user record) khỏi hệ thống. Khi người dùng Logout, hệ thống sẽ xóa JWT khỏi sessionStorage.

3.3.7. Vector Synchronization - Học văn phong người dùng

Chức năng này giúp hệ thống học văn phong từ email đã gửi của user, cho phép đồng bộ email cũ vào vector database, tạo embeddings để phục vụ RAG retrieval.

Sau khi user login, frontend gọi POST /ai/sync, backend trigger background task chạy vectorizer.py, lấy 50 email SENT gần nhất từ Gmail, chia thành chunks (800 chars/chunk), tạo embeddings và insert vào bảng documents với user_id filter. Bên cạnh đó, vectorizer kiểm tra email đã tồn tại trong database hay chưa & chỉ sync email mới để tránh trùng lặp.

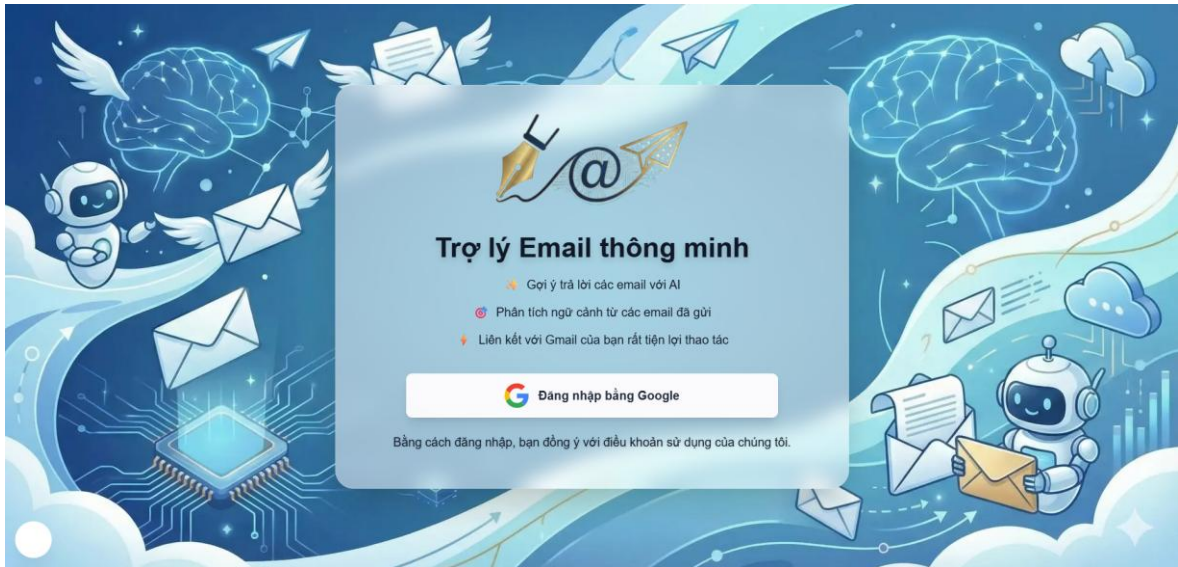
Thông tin vector không chỉ hữu ích cho AI trong việc tạo reply phù hợp văn phong, mà còn là nguồn dữ liệu quan trọng để cải thiện độ chính xác của gợi ý. Nhờ đó AI ngày càng hiểu rõ cách user viết thư, tạo ra reply tự nhiên và cá nhân hóa hơn.

3.3.8. Dashboard Workspace

Dashboard được xem là trung tâm điều khiển của hệ thống, tại đây người dùng có thể nhanh chóng nắm bắt danh sách email, xem AI suggestions và quản lý drafts. Giao diện được chia thành 3 phần: Email List (trái), Email Content (giữa) & AI Suggestion Panel (phải). Bằng việc sử dụng React hooks và real-time state

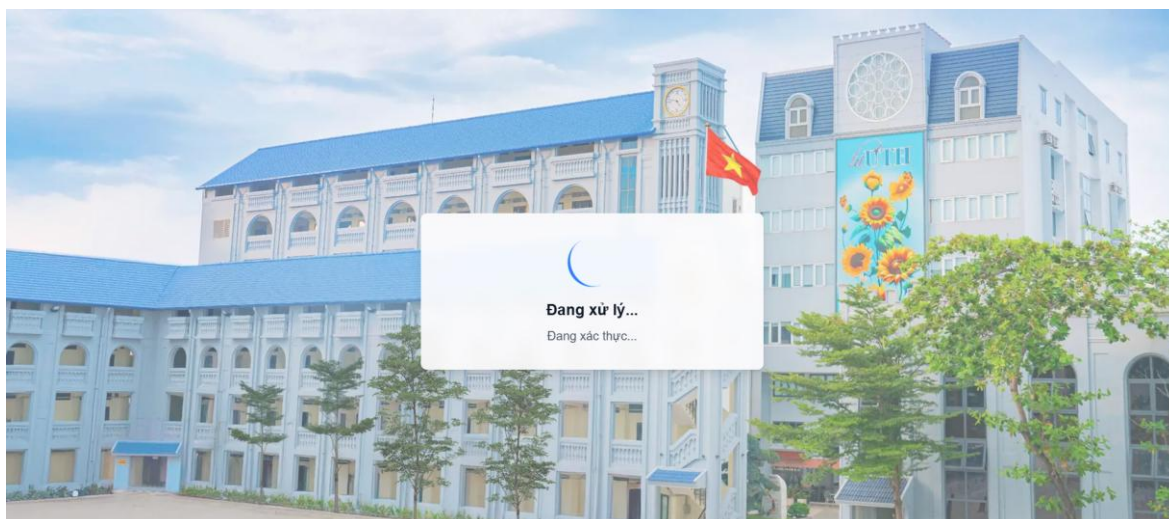
management, trang này không chỉ giúp user theo dõi email mới, mà còn hỗ trợ multitasking với draft qua checkbox selection. Component workspace/page.tsx quản lý toàn bộ state (emails, selectedEmail, loading, syncing), đồng bộ giữa các panel thông qua props và callbacks. Đây là công cụ quan trọng trong việc tối ưu workflow xử lý email hàng ngày, giúp user tiết kiệm thời gian và tăng hiệu suất làm việc.

3.4. Minh họa giao diện

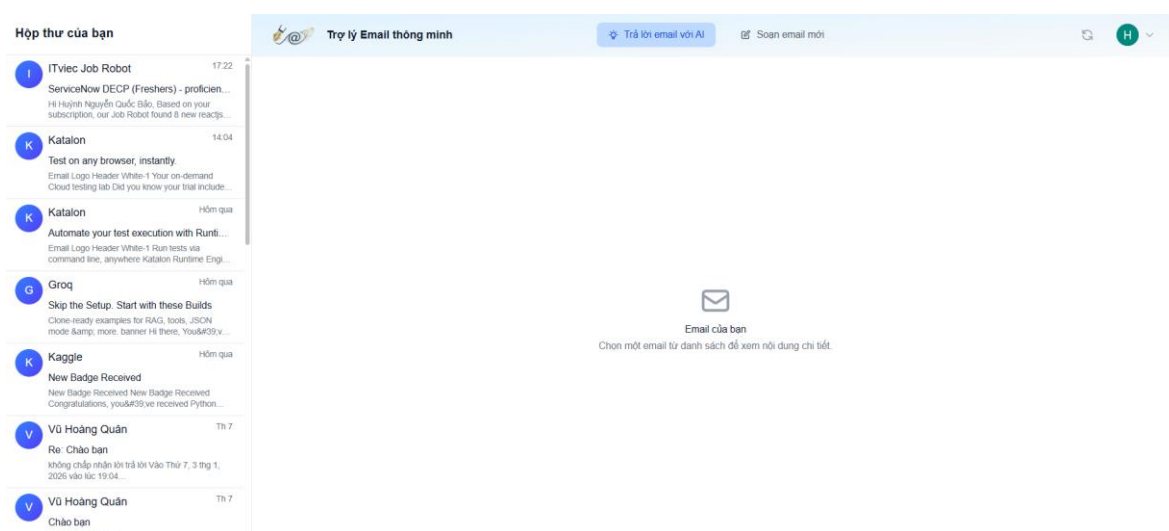


Hình 3.1. Giao diện trang Đăng nhập

Khi vào hệ thống, người dùng sẽ nhìn thấy trang mở đầu với nút “Đăng nhập bằng Google” nằm ở vị trí trung tâm, đồng nghĩa với việc họ cần phải đăng nhập bằng tài khoản Google để sử dụng ứng dụng này. Song song đó, họ cũng cần cấp các quyền cần thiết để hệ thống có thể truy cập vào Gmail của họ và thực hiện các tác vụ liên quan. Sau đó hệ thống sẽ quét các email đã gửi của họ và vector hóa chúng để làm ngữ cảnh phục vụ cho “bộ nhớ” của LLM.

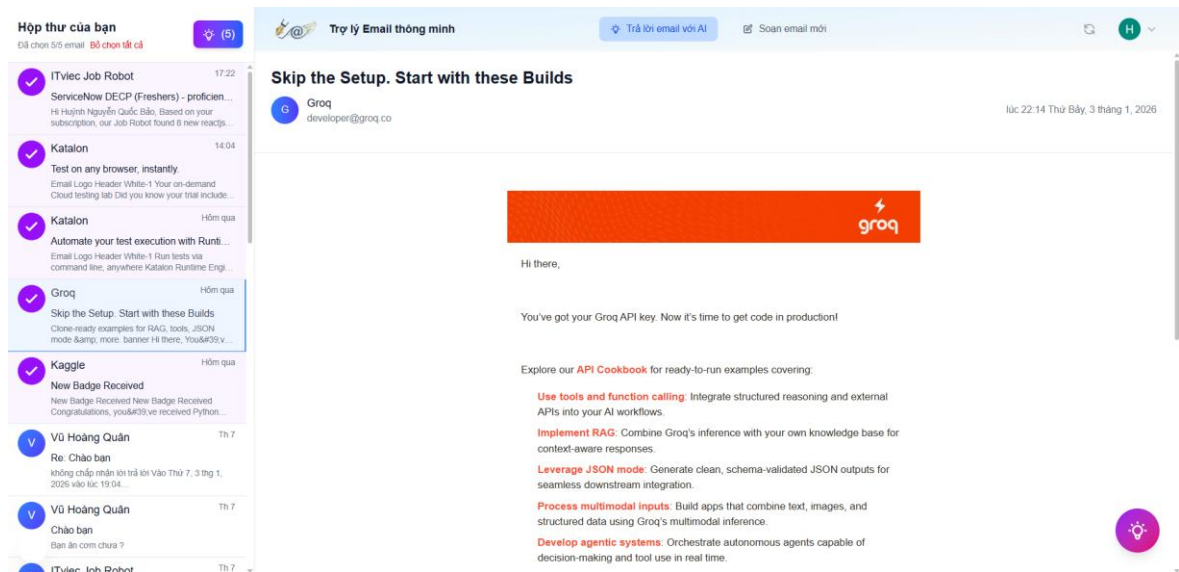


Hình 3.2. Giao diện trang xác thực, quét email đã gửi & lấy ngữ cảnh



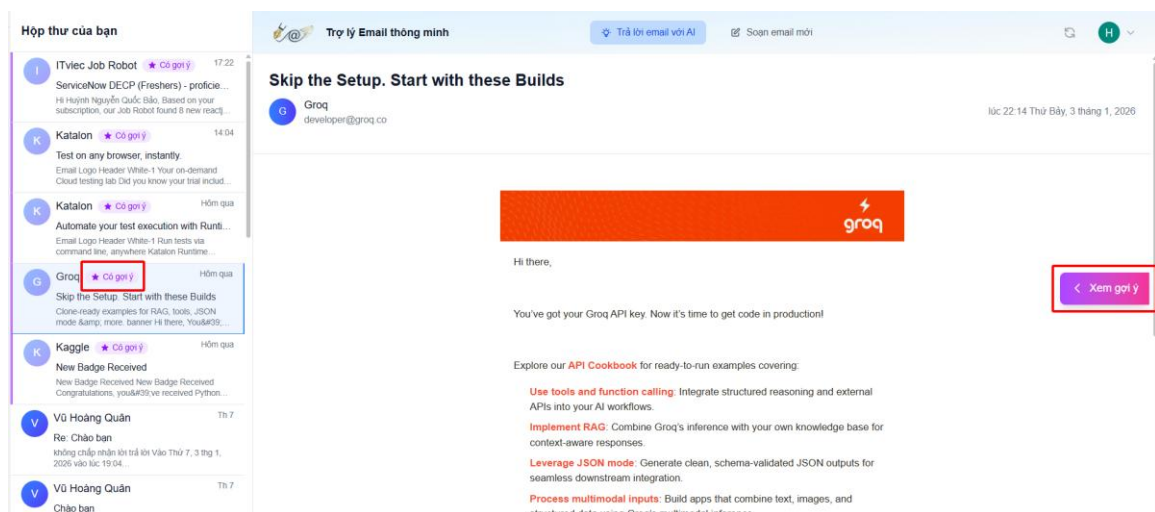
Hình 3.3. Giao diện trang trả lời email với AI

Giao diện trang trả lời email được thiết kế trực quan với các phần riêng biệt: Phần bên trái hiển thị danh sách các email trong hộp thư đến (Inbox) của Gmail, cuối danh sách có phần tải thêm email chưa hiển thị hết, phần còn lại sẽ hiển thị nội dung của từng email khi người dùng nhấp chọn một email từ danh sách, và phần cuối cùng sẽ hiển thị kết quả trả lời email mà mô hình AI trả về nếu có.

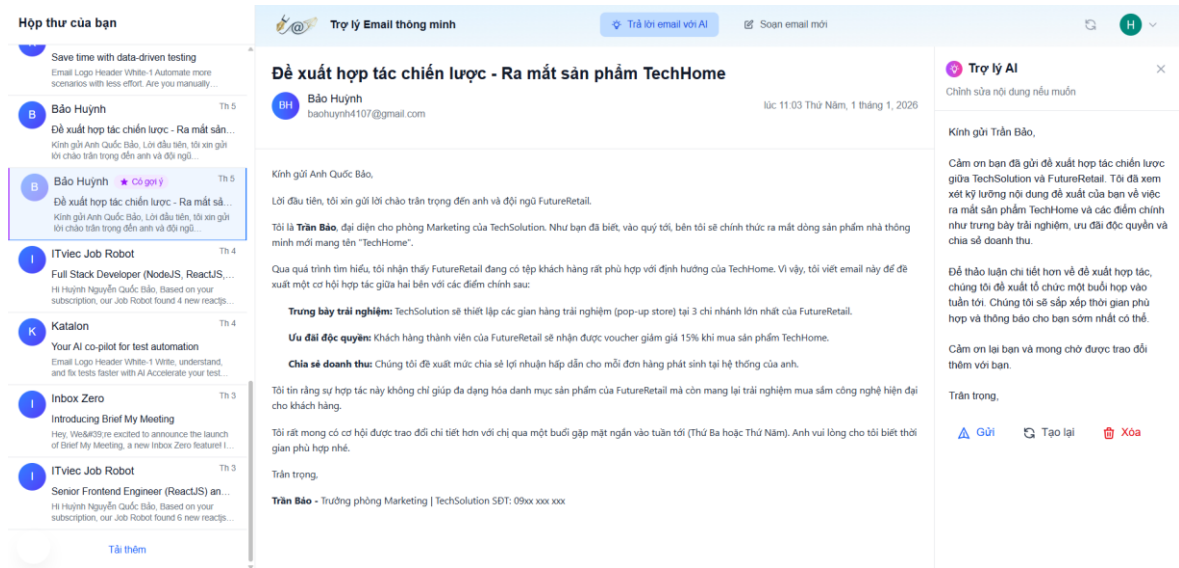


Hình 3.4. Giao diện xem chi tiết email và chọn email để trả lời với AI

Giao diện được thiết kế dễ nhìn, hỗ trợ hiển thị trực quan, chính xác các email từ người gửi, cũng như bố trí phù hợp để người dùng dễ dàng thao tác. Khi người dùng muốn trả lời email với sự hỗ trợ của mô hình AI, người dùng sẽ tick chọn các ô ở các email tương ứng ở danh sách, và chỉ chọn tối đa 5 email để đảm bảo hiệu suất trả lời của mô hình, sau đó nhấn nút tạo trả lời ở trên cùng. Bên cạnh đó, người dùng cũng có thể đồng bộ thủ công các email trong hộp thư đến ở Gmail bằng cách nhấn biểu tượng đồng bộ ở Header, nằm cạnh ảnh đại diện của họ. Ngoài ra, họ cũng có thể chọn tạo câu trả lời ngay tại email đang xem bằng cách nhấn biểu tượng ở góc phải dưới cùng.

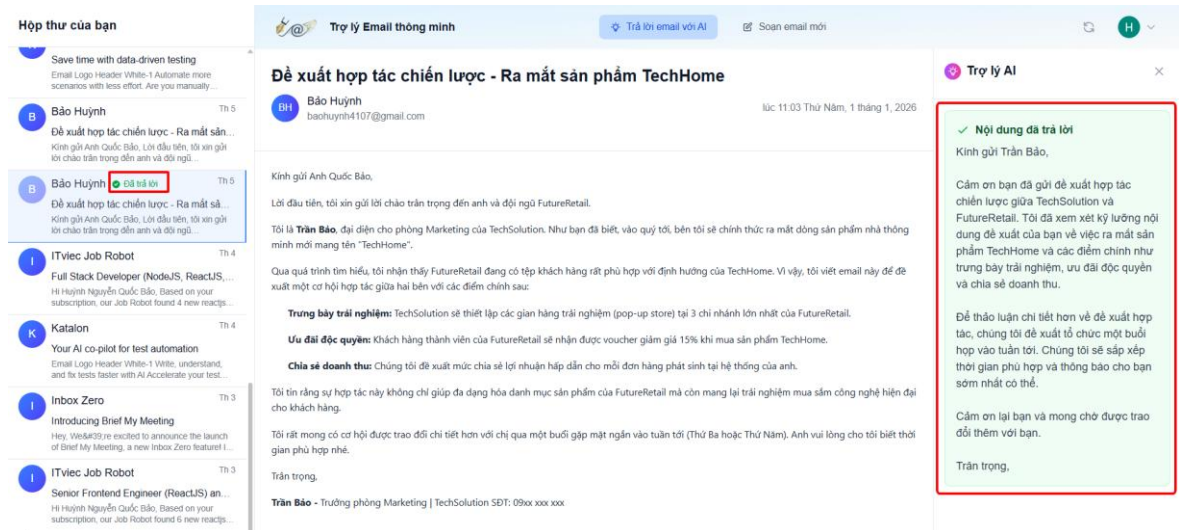


Hình 3.5. Giao diện AI hoàn tất tạo trả lời - Ảnh 1



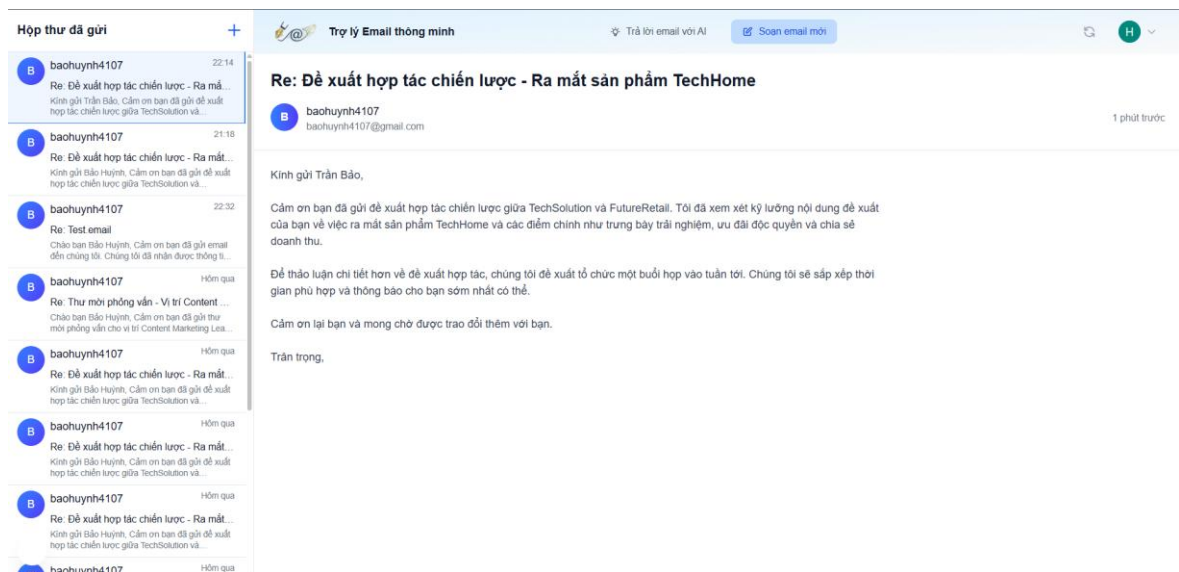
Hình 3.6. Giao diện AI hoàn tất tạo trả lời - Ảnh 2

Khi có kết quả từ mô hình, giao diện hiển thị các biểu tượng đánh dấu nổi bật trên giao diện danh sách. Hệ thống sẽ lưu lại các bản nháp này để tránh việc mất nội dung, cũng như tránh được việc mô hình phải tạo lại câu trả lời trên chính email đã thực hiện trước đó, điều khá gây bất tiện cho người sử dụng. Bên cạnh đó khi click chọn một email trong số đó, phần bên phải sẽ hiển thị nội dung trả lời mà mô hình AI đã tạo, cùng với các nút thao tác với bản nháp như gửi, chỉnh sửa, tạo lại & xóa, góp phần nâng cao sự cộng tác & giám sát của con người vào quy trình tự động của hệ thống, tăng tính chính xác cho các tác vụ của họ, tránh thực hiện các hành động không được kiểm soát. Nếu không xem nội dung bản nháp nữa thì người dùng tắt nó đi, phần bên phải sẽ thu lại và hiển thị banner để người dùng có thể bấm vào xem lại.



Hình 3.7. Giao diện đã gửi email trả lời

Sau khi người dùng nhấn gửi bản nháp, hệ thống tự động cập nhật trạng thái của email là “Đã gửi trả lời”, đồng thời email được gửi đi cũng xuất hiện ở Gmail giúp gia tăng tính tiện lợi & đồng bộ.



Hình 3.8. Giao diện trang soạn email & xem chi tiết email đã gửi

Giao diện soạn email được thiết kế gần giống phần tạo câu trả lời với mô hình AI, giúp người dùng dễ sử dụng và dễ thao tác hơn. Giao diện gồm 2 phần, phần bên trái cũng là danh sách các email, nhưng là các email đã gửi (Sent) trong Gmail của người dùng, phần còn lại dùng để hiển thị chi tiết email mà họ đã gửi, hoặc giao diện soạn email mới nếu người dùng nhấn biểu tượng “+” trên giao diện.

Soạn email mới

Người nhận *

recipient@example.com

Tiêu đề

Nhập tiêu đề email

Nội dung *

Nhập nội dung email...

Đính kèm tệp (Muốn gửi nhiều tệp thì chọn nhiều tệp từ máy trong một lần)

Chọn tệp

Gửi email

Hình 3.9. Giao diện soạn email mới

Người nhận *

hiuth2004@gmail.com

Tiêu đề

Chương II Xử lý ảnh

Nội dung *

Chào Vũ Quân,
Tôi viết mail này để gửi cho bạn phần soạn của tôi Chương II. Các thuật toán xử lý ảnh (Phần 1). Hy vọng tài liệu này
hữu ích với bạn.
Trân trọng

Đính kèm tệp (Muốn gửi nhiều tệp thì chọn nhiều tệp từ máy trong một lần)

Chọn tệp

Chap2_ThuatToanXLA_ver1.pdf (4376.7 KB)

X

Gửi email

Hộp thư đã gửi

+

hiuth2004

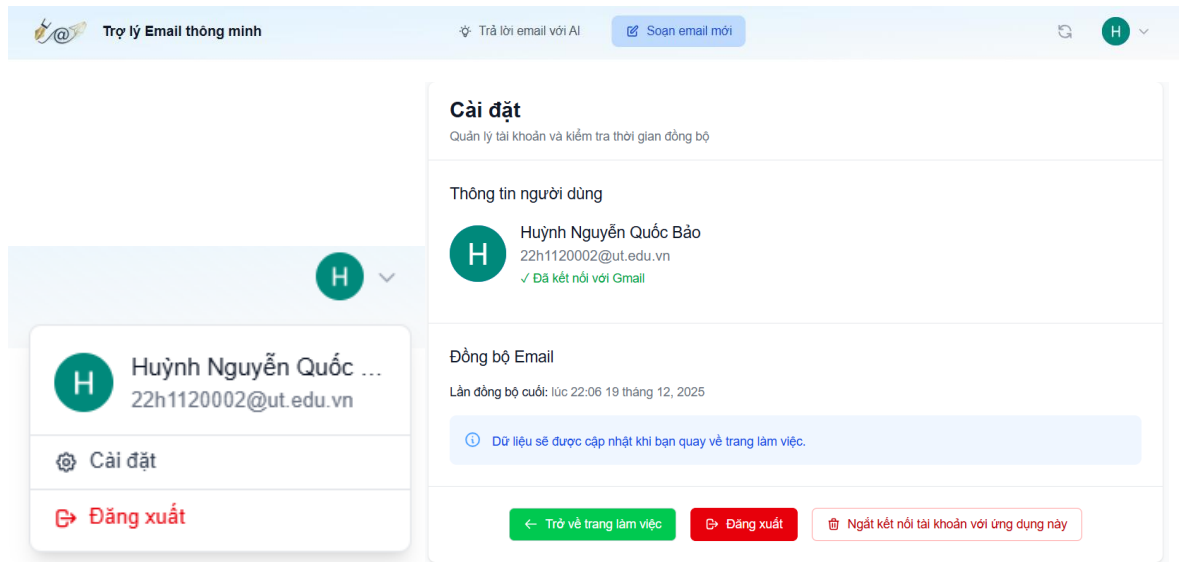
22:15

Chương II Xử lý ảnh

Chào Vũ Quân, Tôi viết mail này để gửi cho bạn phần soạn của tôi Chương II. Các thuật toán xử lý ảnh (Phần 1). Hy...

Hình 3.10. Giao diện soạn email mẫu & danh sách cập nhật sau gửi

Giao diện soạn email được thiết kế đơn giản hơn so với Gmail, nhưng vẫn đảm bảo đầy đủ cấu trúc của một email thông thường, gồm email người nhận, tiêu đề email, nội dung email, cũng như các tệp đính kèm cần gửi. Sau khi xác nhận gửi, giao diện tự động cập nhật email vừa gửi lên danh sách trên giao diện.



Hình 3.11. Giao diện Header, menu và trang cài đặt

Trên phần Header của giao diện có bố trí 2 nút “Trả lời email với Trợ lý AI” & “Soạn email mới” để dễ dàng chuyển đổi qua lại giữa 2 trang làm việc, cũng như một phần hiển thị ảnh đại diện tài khoản Google của người dùng, tại đó họ có thể vào trang cài đặt cũng như đăng xuất khỏi ứng dụng. Tại giao diện cài đặt, người dùng sẽ nhìn thấy thông tin tài khoản của họ, thời gian đồng bộ các email, giúp họ kiểm soát trạng thái của ứng dụng thuận tiện hơn. Song song đó, phần cài đặt cũng đặt các nút thao tác với ứng dụng & tài khoản được bố trí dễ nhìn, màu sắc dễ phân biệt như Trở về trang làm việc, Đăng xuất & Ngắt kết nối tài khoản với ứng dụng này.

3.5. Kiểm thử

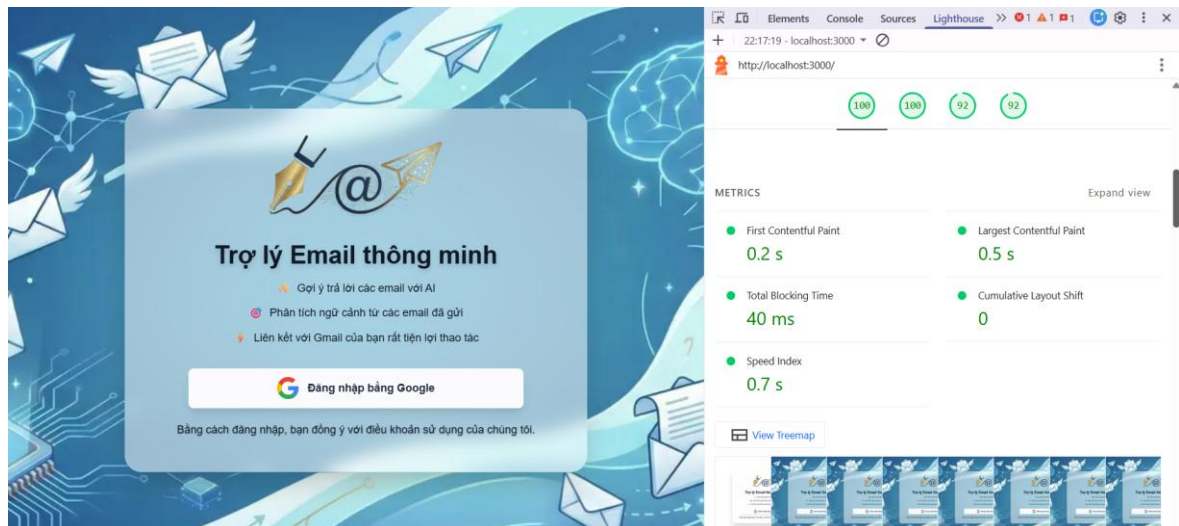
3.5.1. Kiểm thử hiệu năng

Google Lighthouse là một công cụ kiểm thử tự động, mã nguồn mở được Google phát triển nhằm mục đích đánh giá và chuẩn hóa chất lượng trang web. Công cụ này hoạt động linh hoạt trên nhiều môi trường: tích hợp sẵn trong Chrome DevTools, chạy dưới dạng tiện ích mở rộng (Extension) hoặc vận hành qua giao diện

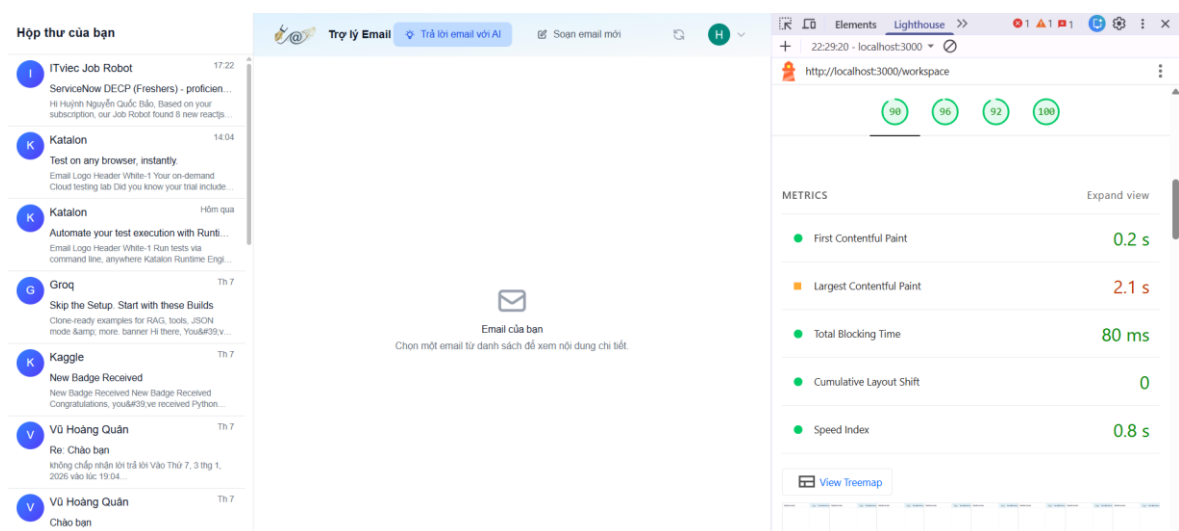
dòng lệnh (CLI/Node.js). Lighthouse thực hiện quy trình kiểm tra (audit) toàn diện dựa trên 5 tiêu chí cốt lõi: Hiệu năng (Performance), Khả năng truy cập (Accessibility), Tuân thủ SEO, Các phương pháp tối ưu (Best Practices) và Ứng dụng Web lữ tiên (PWA). Kết quả đầu ra là một báo cáo chi tiết, cung cấp điểm số định lượng cho từng hạng mục và các chỉ số Core Web Vitals, đồng thời đề xuất các giải pháp kỹ thuật cụ thể giúp lập trình viên tối ưu hóa trải nghiệm người dùng và thứ hạng tìm kiếm. Nhóm thực hiện sử dụng công cụ Lighthouse này cho phân kiểm thử hiệu năng của hệ thống.

Khi đánh giá hiệu năng (Performance), công cụ Lighthouse dựa trên các chỉ số sau:

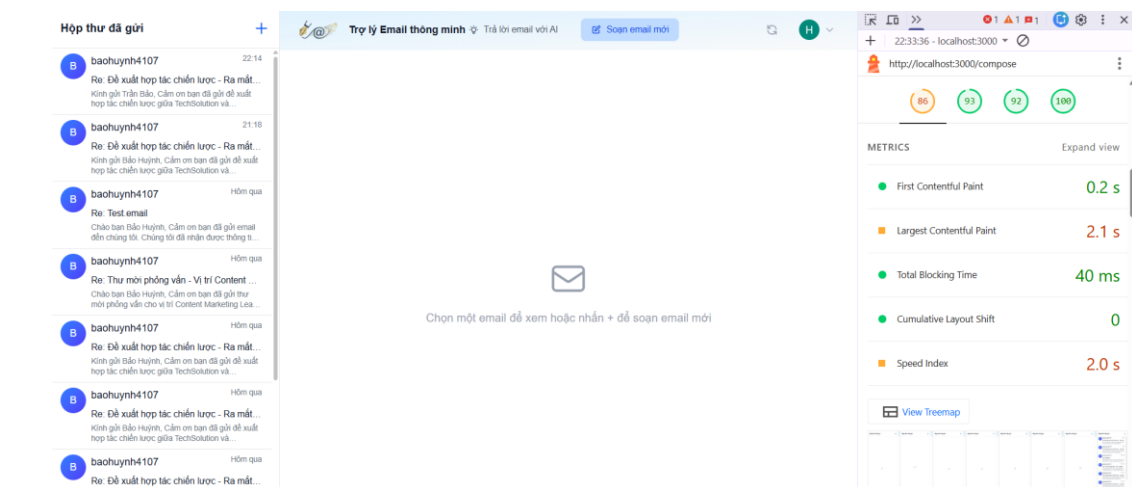
- *First Contentful Paint (FCP)*: Thời điểm trình duyệt hiển thị nội dung đầu tiên của DOM lên màn hình. Đây là dấu hiệu đầu tiên cho người dùng thấy trang đang được tải.
- *Largest Contentful Paint (LCP)*: Thời gian cần thiết để hiển thị phần tử nội dung lớn nhất trong khung nhìn (thường là ảnh banner hoặc khối văn bản chính). Đây là chỉ số quan trọng nhất đánh giá tốc độ tải cảm nhận.
- *Total Blocking Time (TBT)*: Tổng thời gian luồng chính (main thread) bị chặn bởi các tác vụ dài (> 50ms) trong khoảng thời gian từ FCP đến khi trang sẵn sàng tương tác (TTI), gây ra độ trễ khi người dùng cố gắng thao tác.
- *Cumulative Layout Shift (CLS)*: Điểm số đo lường sự ổn định thị giác. Chỉ số này đánh giá mức độ dịch chuyển bất ngờ của các phần tử giao diện trong quá trình tải trang.
- *Speed Index*: Chỉ số đo tốc độ hiển thị trực quan của nội dung trong quá trình tải trang. Speed Index càng thấp, người dùng càng sớm nhìn thấy trọn vẹn nội dung của trang web.



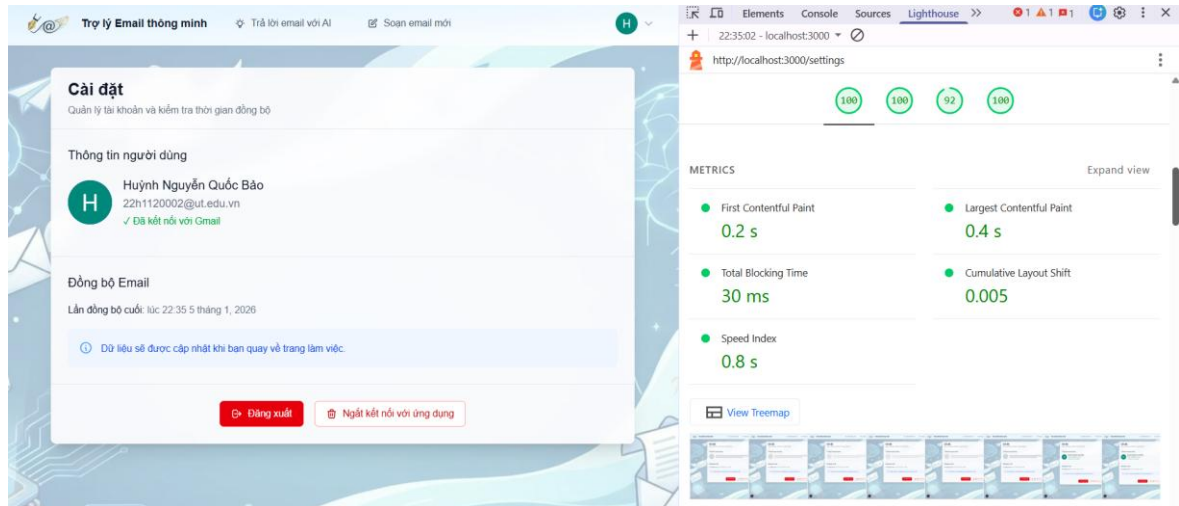
Hình 3.12. Kiểm thử hiệu năng trang mở đầu



Hình 3.13. Kiểm thử hiệu năng trang trả lời email với AI



Hình 3.14. Kiểm thử hiệu năng trang soạn email mới



Hình 3.15. Kiểm thử hiệu năng trang cài đặt

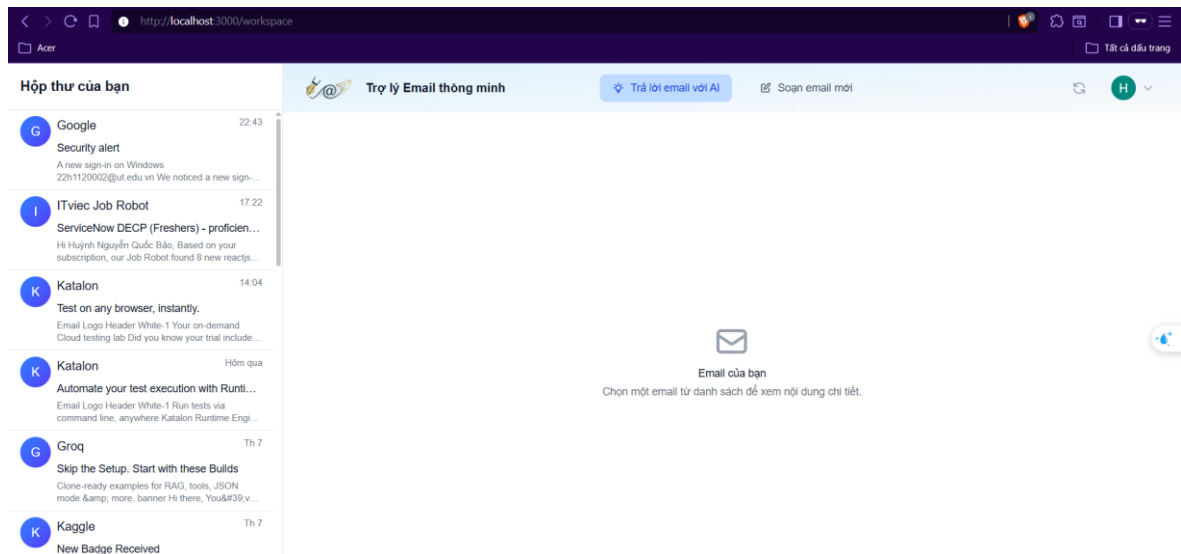
Đánh giá các kết quả:

- Trang chủ (mở đầu) đạt điểm tuyệt đối về hiệu năng với thời gian hiển thị nội dung đầu tiên (FCP) chỉ mất 0,2s, cho thấy trang tải gần như tức thì. Chỉ số thay đổi bố cục (CLS) bằng 0, chứng tỏ giao diện cực kỳ ổn định, không có hiện tượng giật lag khi tải. Thời gian TBT rất thấp (40ms), đảm bảo trải nghiệm mượt mà ngay từ đầu.
- Trang soạn email mới mặc dù không đạt 100 điểm như kỳ vọng, nhưng chỉ số tương tác (TBT) vẫn rất tốt ở mức 40ms, đảm bảo người dùng có thể gõ phím và tương tác ngay lập tức. Điểm số bị kéo xuống chủ yếu do LCP đạt 2,1s và Speed Index là 2,0s, phản ánh việc tải trình soạn thảo văn bản hoặc các thư viện đi kèm đang chiếm dụng tài nguyên tải ban đầu & cần tối ưu thêm.
- Trang cài đặt hiển thị thông tin người dùng và trạng thái đồng bộ nhưng tốc độ tải vẫn rất nhanh (LCP 0,4s). Chỉ số Speed Index là 0,8s, nằm trong ngưỡng rất tốt (màu xanh), cho thấy việc tối ưu hóa tải trang tĩnh và thông tin người dùng đã được thực hiện rất tốt.
- Trang workspace cho thấy điểm hiệu năng đạt 90/100. Chỉ số FCP (0,2s) và Speed Index (0,8s) cho thấy nội dung xuất hiện gần như tức thì. Tuy nhiên, chỉ số LCP đạt 2,1s cho thấy việc render danh sách email hoặc các

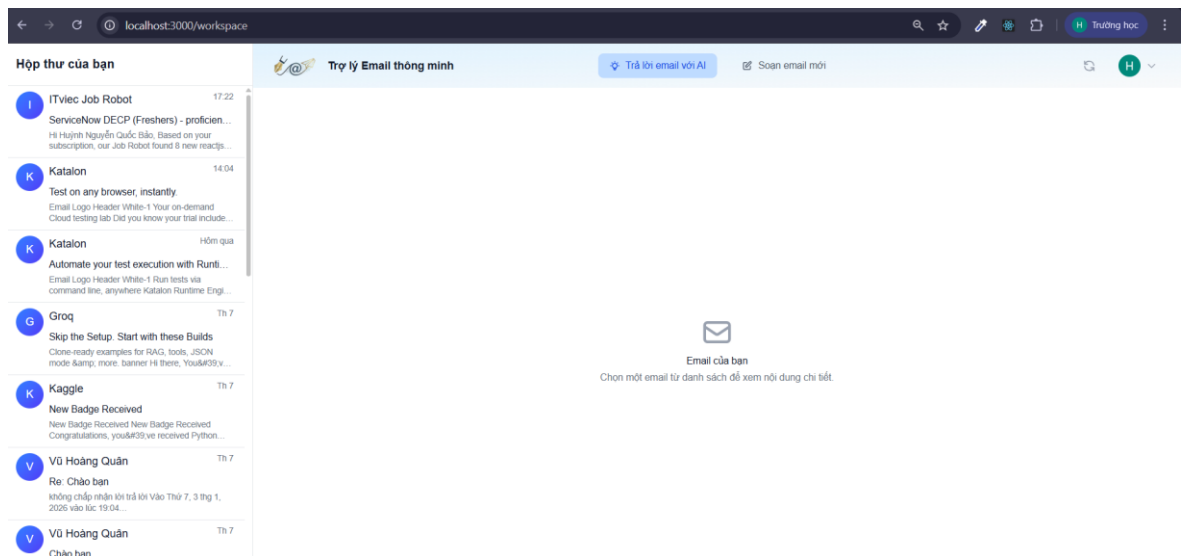
thành phần chính đang mất một chút thời gian. Đáng chú ý, chỉ số TBT là 80ms chứng tỏ các tác vụ JavaScript chặn luồng chính được xử lý gọn gàng.

3.5.2. Kiểm thử trên nhiều trình duyệt

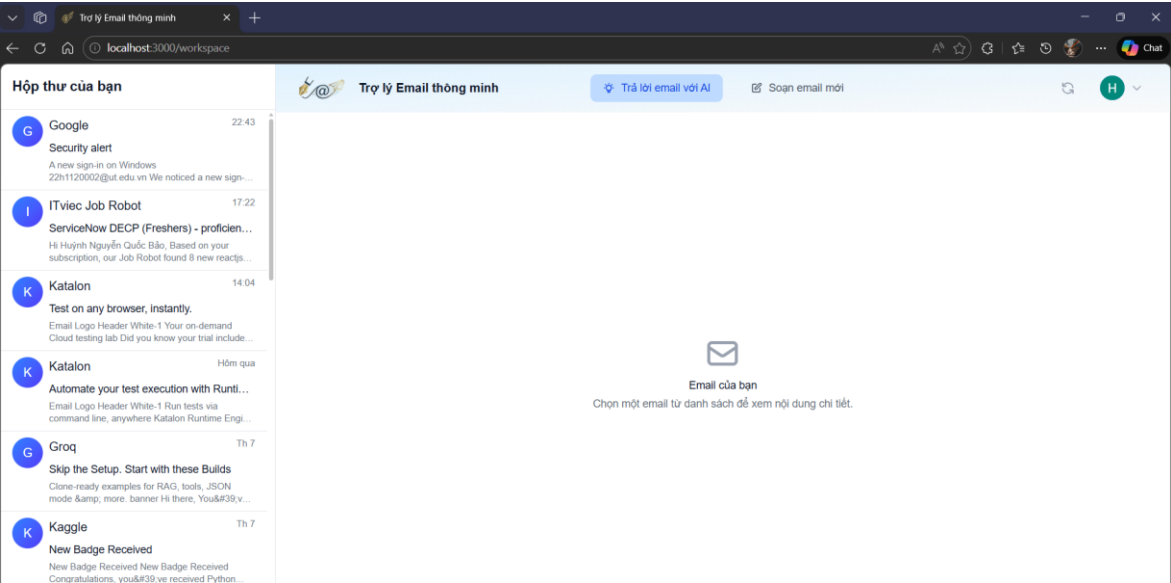
Ứng dụng có thể chạy & hiển thị tốt trên nhiều trình duyệt như Chrome, Microsoft Edge, Brave...



Hình 3.16. Giao diện trên trình duyệt Brave



Hình 3.17. Giao diện trên trình duyệt Chrome



Hình 3.18. Giao diện trên trình duyệt Microsoft Edge

3.5.3. Kiểm thử chức năng

Kiểm thử các chức năng giúp đảm bảo các chức năng hoạt động mượt mà, trơn tru, đúng luồng, cũng như đảm bảo các thành phần giao diện hiển thị đúng trạng thái và hợp lý trong quá trình sử dụng.

Bảng 3.1. Kiểm thử chức năng

Mã TC	Tên test case	Các bước thực hiện	Dữ liệu đầu vào	Kết quả mong đợi	Kết quả thực tế	Trạng thái
TC 01	Đăng nhập - Nhập email sai định dạng	1. Mở trang Email Composer. 2. Nhập email không đúng định dạng vào ô "To". 3. Nhấn nút "Gửi".	Email: "invalid.email" hoặc "user@"	- Hiện thị thông báo lỗi "Địa chỉ email không hợp lệ". - Toast đỏ xuất hiện. - Email không được gửi.	Toast error hiển thị đúng thông báo "Địa chỉ email không hợp lệ". Email không được gửi.	Pass

Mã TC	Tên test case	Các bước thực hiện	Dữ liệu đầu vào	Kết quả mong đợi	Kết quả thực tế	Trạng thái
TC 02	Composer - Validate trường người nhận trống	1. Mở trang Email Composer. 2. Để trống ô "To". 3. Nhập nội dung email. 4. Nhấn nút "Gửi".	To: "" (rỗng) Body: "Test content"	- Hiện thị thông báo "Vui lòng nhập địa chỉ email người nhận". - Toast warning xuất hiện. - Email không được gửi.	Toast warning hiển thị đúng. Validation hoạt động chính xác.	Pass
TC 03	Composer - Validate nội dung email trống	1. Mở trang Email Composer. 2. Nhập địa chỉ email hợp lệ. 3. Để trống nội dung email. 4. Nhấn nút "Gửi".	To: "test@example.com" Body: "" (rỗng)	- Hiện thị thông báo "Vui lòng nhập nội dung email". - Toast warning xuất hiện. - Email không được gửi.	Validation body hoạt động đúng. Toast warning xuất hiện.	Pass

Mã TC	Tên test case	Các bước thực hiện	Dữ liệu đầu vào	Kết quả mong đợi	Kết quả thực tế	Trạng thái
TC 04	Nút Generate - Hiển thị trạng thái loading	1. Mở AI Suggestion Panel. 2. Nhấn nút "Tạo lại gợi ý". 3. Xác nhận trong hộp thoại.	Email có draftId hợp lệ	- Nút bị disable ngay lập tức. - Hiện thị loading state. - Nội dung cũ bị xóa. - Chờ đến khi có nội dung mới.	Loading state hiển thị đúng. Nút disable trong quá trình xử lý.	Pass
TC 05	Confirm Dialog - Xác nhận gửi email	1. Mở AI Suggestion Panel. 2. Có nội dung AI suggestion. 3. Nhấn nút "Gửi". 4. Kiểm tra hộp thoại confirm.	Draft đã tạo sẵn	- Hiện thị confirm dialog với tiêu đề "Xác nhận gửi email". - Có 2 nút: "Gửi" và "Hủy". - Type: info (màu xanh).	Dialog confirm hiển thị chính xác với type info màu xanh.	Pass
TC 06	Confirm Dialog - Xác nhận xóa draft	1. Mở AI Suggestion Panel. 2. Nhấn nút "Xóa bản nháp". 3. Kiểm tra hộp thoại confirm.	Draft đã tạo sẵn	- Hiện thị confirm dialog. - Có cảnh báo "Lưu ý: Nếu xóa, bản nháp trên Gmail cũng sẽ bị mất!". - Type: danger (màu đỏ).	Dialog danger màu đỏ hiển thị với cảnh báo đầy đủ.	Pass

Mã TC	Tên test case	Các bước thực hiện	Dữ liệu đầu vào	Kết quả mong đợi	Kết quả thực tế	Trạng thái
TC 07	Confirm Dialog - Xác nhận tạo lại gợi ý	1. Mở AI Suggestion Panel. 2. Nhấn nút "Tạo lại gợi ý". 3. Kiểm tra hộp thoại confirm.	Draft đã tồn tại	- Hiện thị confirm dialog "Tạo lại gợi ý". - Có cảnh báo "Bản nháp hiện tại sẽ bị xóa". - Type: warning (màu vàng).	Dialog warning màu vàng hiển thị đúng với thông báo cảnh báo.	Pass
TC 08	Toast Notification - Hiện thị thông báo thành công	1. Thực hiện hành động gửi email thành công. 2. Quan sát toast notification.	Email hợp lệ	- Toast màu xanh xuất hiện. - Hiện thị "Email đã được gửi thành công!". - Tự động đóng sau vài giây.	Toast success màu xanh xuất hiện và tự động đóng sau 3 giây.	Pass
TC 09	Email Composer - Lưu draft vào local Storage	1. Mở Email Composer. 2. Nhập email, subject, body. 3. Reload trang. 4. Kiểm tra dữ liệu vẫn còn.	To: "test@example.com" Subject: "Test" Body: "Content"	- Dữ liệu được lưu tự động trong localStorage - Sau reload, form vẫn giữ nguyên nội dung.	Local Storage lưu trữ đúng. Sau reload dữ liệu được restore hoàn toàn.	Pass

Mã TC	Tên test case	Các bước thực hiện	Dữ liệu đầu vào	Kết quả mong đợi	Kết quả thực tế	Trạng thái
TC 10	EmailList - Hiển thị thời gian email	1. Mở EmailList. 2. Kiểm tra định dạng thời gian của các email	Emails có timestamp khác nhau	- Email hôm nay: hiển thị giờ (ví dụ: "14:30"). - Email hôm qua: "Hôm qua". - Email trong tuần: "Thứ hai", "Thứ ba". - Email cũ hơn: "Thg 1 5".	Tất cả định dạng thời gian hiển thị chính xác theo quy tắc.	Pass
TC 11	AuthGuard - Chuyển hướng khi chưa đăng nhập	1. Xóa token trong session Storage. 2. Truy cập /workspace hoặc /settings.	Token: null	- Tự động chuyển hướng về trang đăng nhập (/). - Không cho phép truy cập trang bảo vệ.	AuthGuard hoạt động đúng. Redirect về trang login ngay lập tức.	Pass
TC 12	AuthGuard - Chuyển hướng khi đã đăng nhập	1. Đăng nhập thành công (có token). 2. Truy cập trang login (/).	Token hợp lệ trong session Storage	- Tự động chuyển hướng sang /workspace. - Không hiển thị trang login.	Redirect tự động sang /workspace . Logic AuthGuard chính xác.	Pass

Mã TC	Tên test case	Các bước thực hiện	Dữ liệu đầu vào	Kết quả mong đợi	Kết quả thực tế	Trạng thái
TC 13	Email Composer - Xóa file đính kèm	1. Chọn file đính kèm. 2. Nhấn nút xóa file. 3. Kiểm tra danh sách files.	File: "doc.pdf"	- File bị xóa khỏi danh sách. - UI cập nhật ngay lập tức. - Không gửi file đã xóa.	File bị xóa thành công. UI cập nhật.	Pass
TC 14	AI Suggestion Panel - Chỉnh sửa nội dung AI	1. Load AI suggestion. 2. Click vào contentEditable div. 3. Thêm/sửa nội dung. 4. Kiểm tra edited Content state.	Draft content sẵn có	- Cho phép chỉnh sửa tự do. - edited Content được cập nhật. - original Draft Content không đổi.	ContentEditable hoạt động tốt. State management chính xác.	Pass
TC 15	AI Suggestion Panel - Hiện thị trạng thái "Đã gửi"	1. Gửi email thành công. 2. Kiểm tra UI của panel.	Email với reply Sent: true	- Hiện thị badge "Đã gửi". - Disable các nút action. - Nội dung vẫn hiển thị nhưng readonly.	Badge "Đã gửi" hiển thị. Tất cả nút action bị disable đúng.	Pass

CHƯƠNG 4. KẾT QUẢ ĐẠT ĐƯỢC VÀ HƯỚNG PHÁT TRIỂN

4.1. Kết quả đạt được

Sau quá trình nghiên cứu lý thuyết về các mô hình ngôn ngữ lớn (LLM) và thực hiện xây dựng ứng dụng, đề tài “Xây dựng ứng dụng trợ lý trả lời Email tự động sử dụng LLM” đã hoàn thành các mục tiêu đề ra. Hệ thống không chỉ dừng lại ở mức độ demo mà đã phần nào đó đã vận hành như một sản phẩm hoàn chỉnh. Dưới đây là những phân tích chi tiết về các kết quả đạt được của sản phẩm:

4.4.1. Các chức năng đã hoàn thành

Qua quá trình thiết kế và phát triển, hệ thống đã đạt được những thành quả quan trọng, thể hiện qua việc hoàn thiện và vận hành ổn định các chức năng cốt lõi.

Về xác thực & quản lý người dùng (*Authentication & User management*), hệ thống đã xây dựng thành công cơ chế đăng nhập hiện đại thông qua việc tích hợp giao thức OAuth 2.0 với Google. Giải pháp này không chỉ đơn giản hóa quy trình đăng nhập xuống còn một cú nhấp chuột mà còn loại bỏ hoàn toàn rủi ro bảo mật liên quan đến việc lưu trữ mật khẩu người dùng. Để đảm bảo trải nghiệm liền mạch, hệ thống sử dụng JWT kết hợp với cơ chế lưu trữ an toàn (*Secure Storage*), giúp duy trì phiên làm việc ổn định và người dùng không cần đăng nhập lại liên tục khi tải lại trang. Bên cạnh đó, tại giao diện "Cài đặt", người dùng có thể quản lý hồ sơ cá nhân (xem thông tin định danh, Avatar), thực hiện đăng xuất an toàn hoặc yêu cầu xóa vĩnh viễn tài khoản khỏi hệ thống (bao gồm cả việc xóa triệt để dữ liệu vector đã đồng bộ).

Về đồng bộ và quản lý email (*Email sync & management*), đối với khả năng xử lý dữ liệu, ứng dụng đã thiết lập thành công kết nối API hai chiều trực tiếp với Gmail, cho phép đồng bộ dữ liệu theo thời gian thực. Giao diện danh sách email được thiết kế trực quan, hiển thị đầy đủ các siêu dữ liệu (Metadata) cần thiết như người gửi, tiêu đề, thời gian. Đáng chú ý, hệ thống xử lý hiệu quả phần hiển thị đoạn trích (snippet), giúp người dùng nắm bắt nội dung nhanh mà không cần mở chi tiết thư. Module hiển thị cũng chứng tỏ khả năng tương thích cao khi render chính xác cả định dạng Text thuần túy lẫn các email HTML phức tạp, giữ nguyên bố cục gốc của thư. Ngoài ra, giao diện chọn đa mục cũng đã được hoàn thiện, cho phép người dùng chọn

nhiều email cùng lúc thông qua các checkbox, tạo tiền đề vững chắc cho các tính năng thao tác nhóm trong tương lai.

Về trí tuệ nhân tạo (*AI Core & Generative module*), đây là phân hệ trung tâm của đồ án, đánh dấu việc ứng dụng thành công mô hình ngôn ngữ lớn Llama 3.3 70B Versatile (70 tỷ tham số) thông qua Groq Cloud Infrastructure, kết hợp với framework LangGraph để orchestration và LangChain components để xử lý prompt, mang lại khả năng hiểu sâu ngữ cảnh đa ngôn ngữ như tiếng Việt và tiếng Anh. Quy trình xử lý được tự động hóa theo mô hình mô hình RAG Pipeline, từ trích xuất nội dung email, vector hóa dữ liệu, đến việc gửi prompt kèm ngữ cảnh tới LLM để sinh phản hồi. Điểm ưu việt của hệ thống là khả năng cá nhân hóa câu trả lời thông qua việc AI không phản hồi một cách máy móc mà dựa vào nội dung thư đến để lựa chọn văn phong phù hợp. Cuối cùng, ứng dụng này tuân thủ nguyên tắc "Human-in-the-loop", cung cấp giao diện soạn thảo (Editor) cho phép người dùng kiểm duyệt và tinh chỉnh nội dung do AI tạo ra trước khi gửi đi, đảm bảo tối đa tính chính xác và an toàn thông tin.

4.4.2. So sánh với mục tiêu ban đầu

Đề tài “Trợ lý Email Thông minh” đã được triển khai thành công với kiến trúc hệ thống tách biệt rõ ràng giữa Backend và Frontend. Nhóm nghiên cứu cũng đã phân nào trau dồi cho mình các hiểu biết liên quan đến LLM, RAG cũng như thực hiện đề tài này bằng sự nỗ lực hết mình, vì đây là một luồng kiến thức hoàn toàn mới, và nhóm chưa có kinh nghiệm tiếp xúc với những khía cạnh này trước đây. Ngoài ra, hệ thống không chỉ đạt được hầu hết các mục tiêu nghiên cứu mà còn hoàn thiện thêm nhiều tính năng nâng cao, đáp ứng tốt nhu cầu sử dụng thực tế của nhiều đối tượng người dùng.

Thứ nhất, về tự động hóa và cá nhân hóa, hệ thống đã hoàn thành xuất sắc mục tiêu xây dựng ứng dụng tự động gợi ý bản nháp thông qua việc triển khai workflow LangGraph với 4 node xử lý tuần tự từ lấy dữ liệu, tìm kiếm ngữ cảnh, sinh nội dung đến tạo draft. Về mặt giao diện, component AiSuggestionPanel cho phép hiển thị gợi ý theo thời gian thực và hỗ trợ người dùng chỉnh sửa trước khi gửi. Đồng

thời, mục tiêu cá nhân hóa đã đạt được đầy đủ thông qua cơ chế đồng bộ và học hỏi từ các email đã gửi. Hệ thống sử dụng class EmailVectorizer với HuggingFace Embeddings để chia nhỏ và lưu trữ dữ liệu vào Supabase Vector Store, sau đó áp dụng kỹ thuật RAG để truy xuất 3 email tương đồng nhất làm ngữ cảnh, giúp LLM học được văn phong đặc trưng của người dùng, tăng tính nhất quán.

Thứ hai, về kiến trúc công nghệ và kỹ thuật RAG, kiến trúc Tác tử LLM (LLM Agent) đã được ứng dụng thành công thông qua LangGraph, cho phép quản lý workflow có trạng thái và xử lý lỗi tập trung tại mỗi bước logic. Mô hình Multi-Agent được thiết lập với các tác tử chuyên biệt như Fetcher, Retriever, Generator và Composer. Song song đó, kỹ thuật RAG hoạt động hiệu quả với quy trình khép kín: truy xuất dữ liệu dựa trên user_id (Retrieval), tăng cường ngữ cảnh vào prompt (Augmentation), và sinh phản hồi thông qua mô hình Llama 3.3 (Generation). Các biện pháp chống nhái (anti-parroting) và phát hiện ngôn ngữ tự động cũng được tích hợp để đảm bảo chất lượng nội dung sinh ra.

Thứ ba, về xây dựng ứng dụng hoàn chỉnh và tối ưu hiệu suất, hệ thống đã đạt mức độ hoàn thiện sẵn sàng cho triển khai với đầy đủ các phân hệ:

- *Backend*: bao gồm các điểm nổi bật như xác thực OAuth 2.0, quản lý email tiện lợi (list emails với pagination, get chi tiết email, gửi email với file đính kèm, hỗ trợ thread conversation,...), các tính năng AI nâng cao như đồng bộ email để học, tạo draft reply...
- *Frontend*: gồm các mặt đáng chú ý như Google OAuth callback handling, giao diện thiết kế theo phong cách dashboard 3 cột chuyên nghiệp, tối ưu hóa UX, giảm thiểu số lần click chuột, cũng như áp dụng Tech Stack hiện đại như Next.js, Typescript, TailwindCSS...
- *Database* (Supabase PostgreSQL) với các bảng chính: users, documents, email_drafts.
- Và các thiết lập cần thiết như Environment variables được quản lý qua .env, CORS được cấu hình cho Frontend, Logging chi tiết ở mọi layer & Error handling toàn diện.

Về hiệu suất, hệ thống giúp giảm thiểu thời gian soạn thảo đáng kể, từ mức 3-5 phút xuống còn khoảng 30-60 giây để LLM tạo nội dung Draft (và thêm 30-60 giây cho người dùng review/chỉnh sửa, tiết kiệm khoảng 70-80% thời gian cho người dùng. Tính chuyên nghiệp trong giao tiếp cũng được đảm bảo nhờ khả năng giữ văn phong nhất quán, định dạng email chuẩn HTML...

Bên cạnh các mục tiêu cốt lõi, hệ thống đã phát triển thêm 5 tính năng quan trọng bao gồm: hệ thống quản lý bản nháp, hỗ trợ đính kèm đa dạng định dạng tệp, bộ lọc và phân trang email, cập nhật trạng thái đồng bộ thời gian thực, và các cơ chế bảo mật như User-level isolation trong Vector Store. Đánh giá tổng quan cho thấy dự án có điểm mạnh về chất lượng mã nguồn (Clean Architecture), khả năng mở rộng và bảo mật cao. Các công nghệ nổi bật được sử dụng bao gồm FastAPI, LangGraph, Llama 3.3 (GroqAPI), Supabase (PostgreSQL) ở Backend và Next.js 16, TypeScript, Tailwind CSS 4 ở Frontend.

Ứng dụng “Trợ lý Email Thông minh” đã hoàn thành vượt mức các mục tiêu ban đầu. Đóng góp chính của đề tài là chứng minh tính khả thi của LLM Agent trong thực tế, áp dụng thành công RAG với cơ chế bảo mật dữ liệu người dùng, và xây dựng một pipeline xử lý khép kín từ Gmail API đến AI suggestion. Tuy nhiên, một số tính năng nâng cao như chọn văn phong, điều chỉnh các thông số của mô hình, tìm kiếm, quản lý email (lưu trữ, trạng thái đã đọc, đánh dấu...)... hay ứng dụng di động (mobile app) vẫn chưa được triển khai trong giai đoạn này. Đây là những hạng mục đã được dự kiến và sẽ được bổ sung trong các giai đoạn phát triển tiếp theo nhằm hoàn thiện và mở rộng phạm vi ứng dụng của hệ thống.

4.2. Đánh giá hệ thống

4.2.1. Ưu điểm

Hệ thống được xây dựng nhằm tăng tính thực tiễn và khả năng ứng dụng trong môi trường làm việc thực tế, phục vụ nhiều nhóm người dùng như nhân viên văn phòng, sinh viên, giảng viên... Việc tự động hóa quá trình gợi ý trả lời giúp giảm đáng kể thời gian soạn thảo các email lặp lại hoặc email xã giao. Nhờ áp dụng kỹ thuật RAG, hệ thống có khả năng tạo ra nội dung trả lời phù hợp cả về thông tin lẫn

văn phong cá nhân của từng người dùng, đảm bảo mức độ cá nhân hóa cao và tránh hiện tượng trả lời máy móc như ở các công cụ AI thông thường.

Ứng dụng đặc biệt chú trọng đến trải nghiệm người dùng thông qua thiết kế theo mô hình Single Page Application (SPA), mang lại trải nghiệm mượt mà và liên tục. Bố cục giao diện ba cột cho phép người dùng vừa đọc email vừa tham khảo nội dung trả lời trên cùng một màn hình, tạo nên quy trình làm việc liền mạch. Bên cạnh đó, các tính năng như chỉnh sửa trực tiếp trên nội dung gợi ý (inline editing), hiển thị trạng thái tải rõ ràng và thông báo lỗi thân thiện góp phần nâng cao hiệu quả và sự thoải mái khi sử dụng.

Về mặt kỹ thuật, hệ thống sử dụng Next.js (React) cho frontend và FastAPI (Python) cho backend, tận dụng khả năng render nhanh và sức mạnh xử lý bất đồng bộ cho các tác vụ AI. Việc ứng dụng LangGraph để xây dựng workflow AI theo dạng đồ thị có hướng giúp định nghĩa rõ ràng từng bước xử lý, từ đó cải thiện khả năng đọc hiểu mã nguồn, debug và mở rộng hệ thống. Đồng thời, các tác vụ tốn thời gian như đồng bộ và vector hóa email được xử lý bằng cơ chế Background Tasks của FastAPI, đảm bảo giao diện không bị block và người dùng nhận được phản hồi ngay lập tức.

Dự án lựa chọn triển khai mô hình Llama 3.3 70B Versatile trên Groq Cloud thay vì chạy cục bộ hoặc sử dụng các API AI thương mại nhằm đạt tốc độ suy luận cao nhờ phần cứng chuyên dụng, đồng thời giảm chi phí và loại bỏ gánh nặng hạ tầng GPU. Nhờ kiến trúc LPU của Groq, thời gian sinh một bản nháp trả lời email hoàn chỉnh (bao gồm cả RAG retrieval và generation) chỉ khoảng 30-60 giây, nhanh hơn so với chạy local trên GPU thông thường và vượt trội so với CPU inference (15-30 giây), qua đó đảm bảo trải nghiệm gần thời gian thực ngay cả khi tải cao. Hạ tầng Groq Cloud hỗ trợ tự động mở rộng, tránh các nút thắt về tài nguyên tính toán, trong khi mô hình Llama 3.3 cung cấp context window 128K tokens (tương đương khoảng 96.000 từ tiếng Anh hoặc hơn 200 luồng email dài), đủ để xử lý hiệu quả hầu hết các kịch bản email thực tế và các tác vụ đa ngôn ngữ. Hệ thống cũng cache Embeddings Model (all-MiniLM-L6-v2) để giảm loading time từ request thứ 2 trở đi, và sử

dụng temperature=0 để đảm bảo đầu ra nhất quán.

Hệ thống cũng được thiết kế sao cho không tự động gửi email mà chỉ đưa ra gợi ý, cho phép người dùng chỉnh sửa, yêu cầu tạo lại hoặc từ chối hoàn toàn. Ngoài ra, khả năng xử lý đa ngôn ngữ, chống nhại văn phong, phát hiện ngôn ngữ tự động và tích hợp sâu với Gmail API để hiểu đúng ngữ cảnh hội thoại giúp các email được tạo ra tự nhiên, chính xác và nhất quán trong từng chuỗi trao đổi.

4.2.2. Hạn chế

Bên cạnh những ưu điểm đạt được nêu trên, hệ thống vẫn tồn tại một số hạn chế đáng chú ý. Trước hết, hệ thống chưa nhận diện & xử lý triệt để các email spam, rác, quảng cáo..., chúng vẫn hiển thị và cho phép chọn để tạo câu trả lời bằng AI, thay vì phải lọc và không cần phải trả lời, gây nhiều giao diện và tiềm ẩn rủi ro khi AI xử lý nhầm ngữ cảnh. Song song đó, các chức năng hỗ trợ & cũng là nền tảng căn bản hình thành các hệ thống email như tìm kiếm, lọc theo thời gian, lưu trữ, đánh dấu, đồng bộ trạng thái đã đọc với Gmail,... chưa được triển khai đầy đủ trên ứng dụng khiến cho hệ thống còn sơ khai, chủ yếu tập trung vào trả lời tự động với trợ lý AI. Hơn nữa, hệ thống chỉ mới sử dụng một mô hình LLM cố định, đó là Llama 3.3-70B Versatile, chưa cho phép người dùng chuyển đổi giữa nhiều mô hình khác nhau để gia tăng sự đa dạng về câu trả lời cũng như giúp họ so sánh hiệu suất trực quan hơn phù hợp với nhu cầu họ.

Về hiệu năng, hệ thống hiện tại giới hạn email body ở 2500 ký tự để tối ưu prompt size và response time, đồng thời chỉ retrieve top-3 similar emails từ RAG để tránh context overload. Điều này có thể làm mất một số thông tin trong thread cực dài (30+ emails), nhưng đổi lại là tốc độ và tối ưu chi phí phần cứng.

Hệ thống hiện cũng chưa hỗ trợ hoàn thiện các tác vụ xử lý hàng loạt và cơ chế đánh giá chất lượng đầu ra của AI. Mặc dù giao diện cho phép chọn nhiều email, backend chưa được thiết kế để xử lý batch processing an toàn, nghĩa là chỉ xử lý & hoạt động ổn định trên từng email đơn lẻ, dễ dẫn đến quá tải tài nguyên, cạn kiệt bộ nhớ server hoặc trộn lẫn ngữ cảnh giữa các email. Bên cạnh đó, tính đồng thời của hệ thống vẫn chưa thực sự hoạt động, chẳng hạn người dùng không thể thực hiện xem

chi tiết một email hay bấm xem một trang khác khi AI đang tạo ra câu trả lời. Đồng thời, chất lượng câu trả lời của AI chưa đạt độ chính xác tuyệt đối, vẫn tồn tại các vấn đề như hallucination, lệch tone hoặc thiếu ngữ cảnh. Việc thiếu một feedback loop để đo lường mức độ sử dụng, mức chỉnh sửa của người dùng và đánh giá chất lượng draft khiến hệ thống khó tự cải thiện theo thời gian.

Tiếp theo, hiệu quả của hệ thống phụ thuộc mạnh vào dữ liệu email lịch sử của người dùng cả về số lượng và chất lượng, chưa có các template mẫu hay few-shot learning với các email mẫu chung cho ngành nghề cụ thể, dẫn đến vấn đề nghèo nàn ngữ cảnh, các câu trả lời thiếu tính cá nhân hóa và gần như không khác biệt so với việc dùng AI thuần túy đối với người dùng mới hoặc người ít sử dụng email. Bên cạnh đó, việc sử dụng Groq Cloud thay vì local LLM giúp loại bỏ hoàn toàn chi phí đầu tư GPU, vận hành và bảo trì phần cứng, nhưng đổi lại phụ thuộc vào uptime và rate limits của Groq (hiện tại free tier có limit 30 requests/minute). Hiện tại, hệ thống cũng chưa có cơ chế multi-tenancy, hạn chế khả năng phát triển thành sản phẩm SaaS quy mô lớn.

Ứng dụng này cũng chưa hỗ trợ nhiều nền tảng email và giới hạn khả năng mở rộng, chỉ tích hợp với Gmail thông qua Gmail API, chưa hỗ trợ các nền tảng email phổ biến khác như Microsoft Outlook/Exchange, Yahoo Mail, hay các email server tự host sử dụng IMAP/SMTP. Điều này giới hạn đối tượng người dùng tiềm năng, đặc biệt trong các doanh nghiệp lớn thường sử dụng Microsoft 365 với Outlook làm chuẩn. Việc mở rộng hỗ trợ cho các nền tảng khác đòi hỏi xây dựng thêm các adapter tương ứng (OutlookService, IMAPService) và xử lý sự khác biệt về API, format email, authentication flow.

Cuối cùng, ứng dụng chỉ mới tập trung với phiên bản dành cho máy tính, chưa có phiên bản sử dụng trên điện thoại trên các hệ điều hành iOS và Android. Bên cạnh đó, ứng dụng cũng chưa thực hiện triển khai (deploy) để có thể tiếp cận với các đối tượng sử dụng một cách dễ dàng hơn.

4.3. Hướng phát triển

Dựa trên những hạn chế hiện tại và xu hướng phát triển của các ứng dụng trả lời email tự động ngày nay, một lộ trình phát triển trong tương lai được đề xuất để nâng cao giá trị và khả năng cạnh tranh của sản phẩm.

Về mặt kỹ thuật, điều cần chú ý trước tiên là nâng cao chất lượng đầu ra của AI thông qua prompt engineering nâng cao, few-shot learning và cá nhân hóa giọng văn. Cụ thể, thay vì chỉ cung cấp 3 email tham khảo dạng raw text từ RAG, hệ thống nên trích xuất và highlight các đặc điểm văn phong quan trọng như độ dài câu trung bình, tỷ lệ sử dụng câu hỏi, mức độ sử dụng từ chuyên ngành, hoặc các cụm từ mở/đóng bài đặc trưng, & sau đó các metadata này sẽ được đưa vào prompt dưới dạng có cấu trúc để LLM nắm bắt đúng hơn phong cách của người dùng. Bên cạnh đó, ứng dụng cũng dự kiến cung cấp một dropdown menu với các lựa chọn về "Tone & Mood" cho phép người dùng điều chỉnh giọng văn của AI theo ngữ cảnh cụ thể, vì trong thực tế, cùng một nội dung trả lời nhưng tone có thể cần thay đổi tùy theo người nhận. Mỗi tone sẽ tương ứng với một prompt modifier được thiết kế kỹ lưỡng, và người dùng có thể preview sự khác biệt để chọn phiên bản phù hợp nhất.

Song song đó, ứng dụng cũng tiến đến việc triển khai caching thông minh với việc lưu trữ các mẫu thường xuất hiện để tái sử dụng, background processing tối ưu hơn, cũng như proactive AI để thực hiện pre-generate draft cho 5-10 email chưa đọc gần nhất sẽ giúp giảm đáng kể độ trễ, góp phần đưa người dùng tiến tới trải nghiệm gần như “zero-latency”. Các vấn đề khác như triển khai các tính năng còn thiếu như đã đề cập ở phần hạn chế, xử lý email spam giúp giảm gánh nặng cho cả người dùng và AI khi phải tiếp xúc với các email không cần trả lời, tính năng phân loại email giúp người dùng quản lý kho email tốt hơn & tự động giải mã các tiêu đề bị lỗi encoding trước khi trả về Frontend sẽ được triển khai trong phiên bản tiếp theo.

Để cải thiện độ chính xác và tính phù hợp của AI theo thời gian, việc xây dựng feedback loop để thu thập phản hồi người dùng, đánh giá chất lượng draft là điểm đáng chú ý tiếp theo. Hệ thống cần xây dựng một Feedback System toàn diện với nhiều cấp độ, như emoji rating 1-5 sao từ người dùng cho câu trả lời từ AI, hay

tracking mức độ chỉnh sửa của bản nháp,... & các dữ liệu này sẽ được lưu vào database kèm theo metadata của draft và được phân tích để mô hình có thể học từ dữ liệu thực tế. Bên cạnh đó, hệ thống cần được trang bị các công cụ monitoring, logging và load testing nhằm đảm bảo khả năng vận hành ổn định khi mở rộng quy mô.

Ở cấp độ hệ sinh thái, một trong những hạn chế lớn của hệ thống hiện tại là chỉ học được từ email lịch sử của chính người dùng, trong khi thực tế nhân viên văn phòng thường cần truy cập thông tin từ nhiều nguồn khác để trả lời email chính xác. Vì thế, một hướng phát triển chiến lược đó là mở rộng RAG để tích hợp với kho tri thức doanh nghiệp như Google Drive, Confluence, CRM hay các nền tảng chat nội bộ. Điều này cho phép AI không chỉ trả lời email đúng văn phong mà còn chính xác về mặt dữ liệu nghiệp vụ. Chẳng hạn khi người dùng cần trả lời email khách hàng hỏi về giá sản phẩm hoặc thông số kỹ thuật, AI sẽ không chỉ dựa vào văn phong của người dùng mà còn tự động tra cứu chính sách giá mới nhất từ Google Drive, kết hợp với thông tin khách hàng từ CRM để đưa ra câu trả lời vừa đúng số liệu vừa phù hợp về tone.

Hơn nữa, ứng dụng cũng nhắm đến mục tiêu phát triển phiên bản mobile application – điều rất cần thiết trong bối cảnh làm việc hybrid và remote ngày càng phổ biến, xây dựng dashboard phân tích hiệu suất làm việc thông qua các thống kê, cũng như hướng tới một hệ sinh thái plugin mở, tạo nền tảng cho sự phát triển lâu dài và hiệu ứng mạng lưới. Song song đó, hiện tại có rất nhiều mô hình LLM tiềm năng có thể áp dụng vào hệ thống này, vì thế trong tương lai hệ thống sẽ tích hợp nhiều loại mô hình để giúp người dùng lựa chọn (gồm cả trả phí và miễn phí) như các hệ thống chatbot phổ biến hiện tại, góp phần gia tăng sự đa dạng về câu trả lời, có nhiều mẫu trả lời hay hơn, cũng như họ có thể lựa chọn một mô hình & kế hoạch chi tiêu phù hợp với nhu cầu cuộc sống & công việc.

Về trải nghiệm người dùng, hệ thống cần bổ sung thêm các yếu tố như accessibility, đa ngôn ngữ, onboarding thông minh và contextual help giúp mở rộng đối tượng người dùng và nâng cao khả năng tiếp cận. Ngoài ra, các tính năng quản lý email nâng cao như follow-up reminder, thread summarization và smart snooze góp

phần biến hệ thống thành một công cụ quản lý công việc toàn diện, không chỉ đơn thuần là AI trả lời email.

Phát triển một ứng dụng di động trên hai hệ điều hành phổ biến hiện nay là iOS và Android là một mục tiêu dài hạn quan trọng. Việc sử dụng React Native chắc chắn sẽ là một lựa chọn hợp lý, cho phép tái sử dụng phần lớn mã nguồn và logic từ phiên bản web hiện có, giúp tiết kiệm thời gian cũng như chi phí phát triển.

Về triển khai sản phẩm, nhóm nghiên cứu đang dự định thực hiện theo một trong hai hướng. Một hướng là triển khai ứng dụng web, trong đó frontend (Next.js) được deploy trên Vercel để tận dụng CI/CD tự động và gói miễn phí phù hợp cho giai đoạn MVP; backend FastAPI chạy trên Railway/Render/DigitalOcean, còn phần AI sử dụng Groq Cloud API (không cần host GPU server riêng). Mô hình này cho phép truy cập đa thiết bị, dễ cập nhật và thuận lợi cho việc thương mại hóa theo hình thức đăng ký. Điều cần quan tâm đó là Chi phí chính là Groq API (nếu scale ra khỏi free tier) và Supabase storage (free tier: 500MB database + 2GB file storage). Hướng còn lại là phát triển tiện ích mở rộng Chrome tích hợp trực tiếp vào Gmail, inject panel AI để tạo phản hồi ngay trong workflow người dùng, tái sử dụng code React và dùng chung backend API, gọi Groq API từ backend. Cách tiếp cận này giảm ma sát sử dụng nhưng bị giới hạn bởi chính sách, hiệu năng và khả năng mở rộng tính năng. Tuy vậy, hai hướng này vẫn có thể triển khai song song, với web app là sản phẩm đầy đủ và extension đóng vai trò phiên bản nhẹ để tiếp cận người dùng sớm.

Tổng kết, lộ trình phát triển này mang tính linh hoạt, cho phép điều chỉnh theo phản hồi và sự tiến hóa nhanh chóng của công nghệ AI, từ đó giúp sản phẩm duy trì lợi thế cạnh tranh bền vững.

KẾT LUẬN

Đề tài “Ứng dụng LLM để phát triển ứng dụng trả lời Email tự động dựa trên nội dung email đã gửi” đã hoàn thành mục tiêu nghiên cứu và phát triển một hệ thống hỗ trợ tự động hóa quy trình trả lời email thông qua công nghệ AI tiên tiến. Hệ thống đã chứng minh được tính khả thi và hiệu quả cao trong việc giải quyết bài toán thực tiễn về năng suất làm việc của các đối tượng người dùng. Kết quả đạt được không chỉ đáp ứng đầy đủ các yêu cầu ban đầu mà còn vượt trội về khả năng cá nhân hóa và tính ứng dụng thực tế.

Báo cáo đã hệ thống hóa chặt chẽ quy trình xây dựng & phát triển ứng dụng, khởi đầu từ việc nghiên cứu cơ sở lý thuyết về xử lý ngôn ngữ tự nhiên, đánh giá lựa chọn mô hình AI cục bộ tối ưu, rồi việc hiện thực hóa thiết kế hệ thống thông qua các biểu đồ UML chi tiết, cho đến việc áp dụng kiến trúc Clean Architecture và phương pháp phát triển lặp lại. Cách tiếp cận này tuân thủ nghiêm ngặt quy trình phát triển phần mềm chuyên nghiệp, giúp sản phẩm cuối cùng đạt được sự cân bằng giữa hiệu năng xử lý của AI cũng như tính tiện dụng, bảo mật, giải quyết hiệu quả nhu cầu tự động hóa và cá nhân hóa email của người dùng.

Về mặt kỹ thuật, đề tài đã thành công trong việc xây dựng một kiến trúc LLM Agent hoàn chỉnh sử dụng LangGraph để tạo workflow xử lý email với bốn giai đoạn rõ ràng: fetch, retrieve, generate và create draft. Việc áp dụng kỹ thuật RAG với Supabase Vector Store và HuggingFace Embeddings đã cho phép hệ thống học hỏi hiệu quả từ lịch sử email của người dùng, tạo ra các câu trả lời không chỉ chính xác về nội dung mà còn duy trì được văn phong đặc trưng của từng cá nhân. Đặc biệt, việc sử dụng Groq Cloud Infrastructure với mô hình Llama 3.3 70B Versatile đã đảm bảo tốc độ cực nhanh nhờ LPU architecture, zero infrastructure maintenance, và auto-scaling, đồng thời tận dụng được mô hình mã nguồn mở với 70 tỷ parameters mà không cần đầu tư GPU đắt đỏ. Kiến trúc Clean Architecture phía backend được áp dụng nghiêm ngặt đã tạo nên một codebase dễ bảo trì, dễ mở rộng và dễ kiểm thử, thể hiện sự chuyên nghiệp trong thiết kế phần mềm.

Giao diện người dùng được thiết kế với bố cục 3 cột thông minh, cho phép người dùng vừa đọc email vừa xem gợi ý AI và chỉnh sửa trực tiếp trên cùng một màn hình, tạo ra trải nghiệm làm việc mượt mà và hiệu quả. Cơ chế Anti-Parroting và khả năng tự động phát hiện ngôn ngữ (tiếng Việt/tiếng Anh) đã giải quyết được những thách thức lớn trong việc tạo ra các email reply tự nhiên và phù hợp ngữ cảnh. Hơn nữa, việc tích hợp sâu với Gmail API, hỗ trợ thread conversation đúng cách và xử lý attachments cho thấy sự chú trọng đến các chi tiết quan trọng trong trải nghiệm người dùng thực tế.

Tuy nhiên, đề tài cũng nhận diện được một số hạn chế cần khắc phục trong tương lai. Thời gian xử lý 30-60 giây cho mỗi bản nháp email tuy nhanh hơn đáng kể so với triển khai LLM cục bộ nhưng vẫn có thể ảnh hưởng đến các kịch bản yêu cầu phản hồi tức thời. Việc phụ thuộc vào Groq Cloud tạo ra điểm lỗi đơn và chịu giới hạn free tier gây khó khăn khi mở rộng quy mô. Ngoài ra, vấn đề cold-start vẫn tồn tại khi người dùng có ít hơn 10–20 email trong Vector Store. Hệ thống hiện chỉ hỗ trợ Gmail và chưa mở rộng sang các nền tảng email khác như Outlook,... làm giới hạn đối tượng người dùng tiềm năng, đặc biệt trong các doanh nghiệp lớn sử dụng Microsoft 365. Vấn đề xử lý dữ liệu đầu vào như encoding, spam cũng cần được cải thiện để nâng cao chất lượng trải nghiệm tổng thể. Những hạn chế này đã được phân tích kỹ lưỡng và đề xuất các giải pháp cụ thể trong roadmap phát triển trong thời gian tới.

Nhìn về tương lai, đề tài đã vạch ra một lộ trình phát triển chiến lược với ba trụ cột chính: cải tiến kỹ thuật, mở rộng hệ sinh thái và tăng cường trải nghiệm người dùng. Đặc biệt, hướng phát triển tích hợp với enterprise knowledge base (Google Drive, CRM, Confluence) sẽ mở ra tiềm năng ứng dụng rộng lớn hơn, biến hệ thống từ một AI email assistant đơn thuần thành một intelligent work assistant toàn diện.

Như vậy, đề tài không chỉ đạt được thành công về mặt học thuật trong việc nghiên cứu và áp dụng các công nghệ AI vào bài toán thực tế, mà còn tạo ra một sản phẩm có giá trị ứng dụng cao. Sự kết hợp hài hòa giữa lý thuyết vững chắc và thực hành chuyên nghiệp đã tạo nên một nghiên cứu có ý nghĩa cả về mặt khoa học và

thực tiễn. Hệ thống Trợ lý email thông minh không chỉ là minh chứng cho sức mạnh của trí tuệ nhân tạo trong việc tối ưu hóa quy trình làm việc mà còn là bước đệm quan trọng hướng tới một tương lai nơi AI trở thành người đồng hành đáng tin cậy trong mọi khía cạnh công việc hàng ngày của con người. Đề tài đã đặt những viên gạch đầu tiên cho một sản phẩm có tiềm năng tạo ra tác động tích cực lớn đến năng suất lao động và chất lượng cuộc sống của nhiều người trên toàn thế giới.

Quan trọng hơn, việc hoàn thành đề tài không chỉ là cột mốc kết thúc một môn học mà còn là cơ hội để chúng em cọ xát với môi trường phát triển dự án thực tế. Đối mặt với các vấn đề phức tạp xuyên suốt từ hệ thống đến giao diện đã giúp chúng em rèn luyện tư duy giải quyết vấn đề và tác phong làm việc bài bản. Chúng em xin bày tỏ lòng biết ơn sâu sắc đến TS. Hàn Trung Định đã định hướng & hướng dẫn tận tình, cùng tập thể Thầy Cô Viện Công nghệ thông tin và Điện, Điện tử đã xây dựng nền tảng tri thức vững chãi, giúp chúng em có đủ tự tin để hoàn thành dự án này & chinh phục những thử thách sắp tới.

TÀI LIỆU THAM KHẢO

- [1] Kathrin Reinke; Tomas Chamorro-Premuzic, “When email use gets out of control: Understanding the relationship between personality and email overload and their impact on burnout and work engagement,” *Computers in Human Behavior*, pp. 502-509, 2014.
- [2] Vislavath Venkatesh; Durgesh Rathod; Lalitha; SV. Satya Krishna, “AUTO EMAIL RESPONDER USING AI,” *International Research Journal of Modernization in Engineering Technology and Science*, tập 07, số 07, pp. 357-360, 2025.
- [3] Kareem Darwish; Nizar Habash; Mourad Abbas; Hend Al-Khalifa; Huseein T. Al-Natsheh; Samhaa R. El-Beltagy; Houda Bouamor; Karim Bouzoubaa; Violetta Cavalli-Sforza; Wassim El-Hajj; Mustafa Jarrar; Hamdy Mubarak, “A Panoramic Survey of Natural Language Processing in the Arab World,” 2020.
- [4] Victor de Lamo Castrillo; Habtom Kahsay Gidey; Alexander Lenz; Alois Knoll, “Fundamentals of Building Autonomous LLM Agents,” 2025.
- [5] Kannan, Anjuli; Kurzweil, Ray; Balasubramanian, Sujith; Le, Quoc V.; Weinstein, Andrew, “Smart Reply: Automated Response Suggestion for Email,” trong *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, San Francisco, California, USA, 2016.
- [6] A. Bastola; H. Wang; J. Hembree; P. Yadav; Z. Gong; E. Dixon; A. Razi; N. McNeese, “LLM-based Smart Reply (LSR): Enhancing Collaborative Performance with ChatGPT-mediated Smart Reply System,” 2024.
- [7] Benjamin Towle; Ke Zhou, “End-to-End Autoregressive Retrieval via Bootstrapping for Smart Reply Systems,” tập Findings of the Association for Computational Linguistics: EMNLP 2023, p. 7610–7622, 2023.

- [8] Benjamin Towle; Ke Zhou, “Model-Based Simulation for Optimising Smart Reply,” trong *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023.
- [9] Strobe, B.; Kurzweil, R., “Efficient Smart Reply, now for Gmail,” Google, 17 May 2017. [Trực tuyến]. Available: <https://research.google/blog/efficient-smart-reply-now-for-gmail/>. [Đã truy cập 09 December 2025].